

INTRODUCTION TO

Java Programming

A Beginner's Course

STUDENT MANUAL

A complete, printable copy of every lesson in the course:
Part 1 (Modules 0–9), Part 2 (Modules 10–19, with the optional
Module 14b), and Appendix A.

Code examples, exercises and project starter files are in the course repository:
github.com/datlogix/Java4Beginners

Contents

Welcome: How This Course Works.....	3
Part 1: Java Foundations.....	7
Module 0: Setup (the JDK, VS Code, Git & GitHub).....	8
Module 1: First Steps.....	13
Module 2: Values, Types & Operators.....	21
Module 3: Variables, Input & Strings.....	27
Module 4: Making Decisions.....	33
Module 5: Loops.....	38
Module 6: Arrays.....	44
Module 7: ArrayList, HashMap & HashSet.....	50
Module 8: Methods.....	56
Module 9: Capstone 1, Text Adventure.....	62
Part 2: Object-Oriented Java.....	67
Module 10: Classes & Objects.....	68
Module 11: Inheritance & Polymorphism.....	75
Module 12: Interfaces, Enums & Records.....	82
Module 13: Exceptions & Debugging.....	89
Module 14: Files & Data.....	95
Module 14b: Databases with SQLite & JDBC (optional).....	100
Module 15: Generics, Collections & Lambdas.....	107
Module 16: Recursion & Algorithms.....	113
Module 17: Packages, Maven & JUnit Testing.....	118
Module 18: GUIs with JavaFX.....	124
Module 19: Capstone Project 2.....	132
Appendix: Optional Material.....	138
Appendix A: GUIs with Swing (optional).....	139

Welcome: How This Course Works

Welcome! This course takes you from **never having written a line of code** to **designing, testing, and building your own multi-file, object-oriented Java applications**, tracked in Git and pushed to GitHub. No prior programming experience is assumed.

The course comes in two parts:

- **Part 1: Modules 0–9.** From zero to a real program: how Java code is compiled and run, values and types, variables, input, decisions, loops, arrays, lists and maps, methods, and a team capstone (a text adventure game).
- **Part 2: Modules 10–19.** Moving from "I can write Java" to "I can build things in Java": classes and objects, inheritance and polymorphism, interfaces, enums and records, exceptions, files, generics and lambdas, recursion and algorithms, packages, Maven and automated testing with JUnit, graphical user interfaces with JavaFX, and a second, harder capstone. An optional module on databases (SQLite and JDBC) follows Module 14.

Part 2 assumes Part 1 is finished. Module 10 doesn't re-explain anything from Modules 0–9.

Built to work with or without a lecturer in the room

Every lesson is written so you can teach yourself from it directly. The *why* always comes before the *how*, every claim is backed by code you actually run, and nothing assumes a lecturer will fill in a gap out loud. If you have an instructor, they'll add depth, stories, and live debugging. If you don't, you can still get every concept from the page.

Every module starts with a hook

Before any theory, every module opens with a **Hook**: a short, genuinely fun demo you run *first*, before you understand how it works. Curiosity ("wait, how did it do that?") is the whole teaching strategy. It's a far stronger reason to keep going than "you'll need this later." By the end of each module you'll understand exactly how its hook worked.

How this course is built

Every module (after Setup) follows the same shape:

1. **README.md**: the lesson. A hook first, then concepts explained in plain language, with common beginner mistakes called out explicitly.
2. **examples/**: small, runnable `.java` files, each showing one idea. Run every single one yourself, then change something and run it again. Reading code you didn't run teaches you far less than watching your own changes compile (or refuse to).
3. **exercises/**: practice problems with starter files containing `TODO` markers. Reference solutions aren't included in this repo. Your instructor shares them after the deadline, so give each exercise a real attempt (at least 15 minutes) and check your own work against the sample runs in each file.

4. **project/**: a small, fun program that only uses what you've learned *so far*. This is the payoff for the module: something you'd actually want to show a friend.

In Part 2, every module project comes in three **tracks**: **Generic**, **Electrical/Electronic Engineering (EE)**, and **Biomedical Engineering**. They teach the same concept applied to different worlds. Pick the one that fits you, or try more than one.

Two more folders sit alongside the modules:

- **Student Manual/**: every lesson in one printable document (Word and PDF), for reading offline.
- **Slides/**: the slide deck for each module (PowerPoint).

Roadmap

Part 1: Java Foundations

#	Module	You will be able to...
0	Setup: the JDK, VS Code, Git & GitHub	Install your tools, run Java, and push your work to GitHub
1	First Steps	Compile and run a Java program, print output, draw in a window, and read an error message without panicking
2	Values, Types & Operators	Work with whole numbers, decimals, characters, text and true/false, and calculate with them correctly
3	Variables, Input & Strings	Store data, read what the user types with Scanner , and format text that talks back
4	Making Decisions	Make a program choose what to do with if / else if / else and switch
5	Loops	Repeat work with while , do-while and for , and add randomness with Random
6	Arrays	Store and process whole collections of values, including grids
7	ArrayList, HashMap & HashSet	Use collections that grow, look things up by name, count things, and remove duplicates
8	Methods	Break a program into reusable, named, testable pieces
9	Capstone 1: Text Adventure	Build a complete game as a team, in stages, with a real Git history

Part 2: Object-Oriented Java

#	Module	You will be able to...
10	Classes & Objects	Design your own types that bundle data and behaviour, and protect that data with encapsulation

#	Module	You will be able to...
11	Inheritance & Polymorphism	Build families of related classes, override methods, and write code that works with all of them at once
12	Interfaces, Enums & Records	Define contracts between classes, sort anything with <code>Comparable</code> and <code>Comparator</code> , and model fixed sets of values
13	Exceptions & Debugging	Handle bad input gracefully, create your own exceptions, and find bugs with the VS Code debugger
14	Files & Data	Make programs remember things using text and CSV files
14b	Databases with SQLite & JDBC (<i>optional</i>)	Store data in a real database, query it with SQL, and keep it safe from SQL injection
15	Generics, Collections & Lambdas	Write type-safe reusable code, choose the right collection, and transform data with streams
16	Recursion & Algorithms	Write methods that call themselves, and compare searching and sorting algorithms by speed
17	Packages, Maven & JUnit Testing	Organise code into packages, build it with Maven, and prove it works with automated tests
18	GUIs with JavaFX	Build windowed, event-driven programs with forms, tables, charts and animation
19	Capstone 2	Combine every Part 2 idea into one tested, persistent, track-based application

Appendix

	Module	You will be able to...
A	GUIs with Swing (<i>optional</i>)	Build windows with Swing, the toolkit built into the JDK, and understand every line of Module 1's drawing programs

Ground rules for how we'll work

- **Type the code yourself.** Copy-pasting defeats the purpose. Your fingers and your mistakes are how the syntax sticks.
- **Read every error message fully.** Java's compiler tells you the file, the line number, and what it expected. When a program crashes while running, the *top* of the stack trace names the problem and the first line that mentions *your* file shows where it happened.
- **Compile and run often.** Write 3–5 lines, then run. Don't write 50 lines and try to fix twenty compiler errors at once.

- **Commit often.** After finishing an example, exercise, or project step, commit it. Module 0 shows you exactly how. Small, frequent commits are a professional habit you're building from day one.

Prerequisites

None, other than curiosity and a willingness to make (and fix) mistakes. Mistakes are not a sign you're bad at this. They're the main way anyone learns to program. Start with Module 0: Setup.

Java Foundations

From your first line of code to a complete, team-built text adventure: compiling and running Java, values and types, variables and input, decisions, loops, arrays, lists, maps, sets and methods.

Module 0: Setup (the JDK, VS Code, Git & GitHub)

Hook: a language that pops up a window from one line

As soon as Java is installed (step 1 below), open a terminal, type `jshell` and press Enter. You'll see a `jshell>` prompt. Java is now waiting for you. Type these lines, pressing Enter after each:

```
jshell> "Java! ".repeat(5)
jshell> 365 * 24 * 60 * 60
jshell> javax.swing.JOptionPane.showMessageDialog(null, "Hello from Java!")
```

The first line repeats a piece of text five times. The second works out how many seconds there are in a year. The third opens a real window on your screen with a message and an **OK** button. One line of Java just reached outside the terminal and drew on your desktop. (It may open *behind* your terminal. Look for it, then click **OK**.)

Type `/exit` and press Enter to leave the `jshell>` prompt.

You don't know what `javax.swing.JOptionPane` means yet. That's fine: that's what the rest of this course is for. Every module from here replaces one piece of "I don't know why that worked" with "I know exactly why." (You'll build whole windowed programs in Module 18.)

1. Install the Java Development Kit (JDK)

Java is a **compiled** language. Before your code runs, a program called the **compiler** (`javac`) translates it into **bytecode**, and then the **Java Virtual Machine** (`java`) runs that bytecode. Module 1 explains this properly. For now, you need both tools, and they come together in the **JDK** (Java Development Kit).

Install a **JDK, version 21 or newer**. We recommend the free **Eclipse Temurin** build, which works the same on every operating system. Any LTS ("long-term support") version is a good choice: 21 or 25. Every program in this course also works on Java 17.

JDK, not JRE. A JRE (Java Runtime Environment) can *run* Java programs but can't *compile* them. It has no `javac`. Always install the JDK.

Windows

1. Go to adoptium.net, choose the latest **LTS** version, **Windows**, **x64**, **JDK**, and download the `.msi` installer.
2. Run it. On the *Custom Setup* screen, make sure **Add to PATH** and **Set JAVA_HOME variable** are both set to "Will be installed on local hard drive". `JAVA_HOME` is off by default. Turn it on: some tools you'll use later (Maven, in Module 17) need it.
3. Open a **new** terminal (PowerShell or Command Prompt) and check:

```
java -version
javac -version
```

macOS

1. Go to adoptium.net and download the latest **LTS JDK .pkg** installer for macOS (choose **aarch64** for an Apple Silicon Mac, M1 or later, and **x64** for an older Intel Mac).
2. Run it, then open a **new** Terminal window and check:

```
java -version
javac -version
```

Linux (Ubuntu/Debian)

```
sudo apt update && sudo apt install openjdk-21-jdk -y
java -version
javac -version
```

You should see something like **openjdk version "21.0.5"** and **javac 21.0.5**, not "command not found." The two version numbers should match.

2. Install VS Code

Download **Visual Studio Code** from code.visualstudio.com and install it. Then open the Extensions panel (**Ctrl+Shift+X**, or **Cmd+Shift+X** on a Mac), search for **Extension Pack for Java**, and install the one **by Microsoft**. It gives you colour-coded code, errors underlined in red *as you type* (before you even compile), a **Run** button, and the debugger you'll use in Module 13.

Check it works:

1. In VS Code, choose **File → Open Folder...** and open any empty folder.
2. Create a new file called **Test.java**. The file name matters in Java: it must match the class name inside it, capital letter included.
3. Type this in exactly:

```
public class Test {
    public static void main(String[] args) {
        System.out.println("VS Code works!");
    }
}
```

4. Click the  **Run** button in the top right corner (or the small **Run** link that appears just above **main**).

A terminal panel opens at the bottom and shows **VS Code works!**. The first time, the Java extension may take a minute to start up. You'll see "Java: Ready" in the status bar at the bottom when it's done.

3. Install and configure Git

Git keeps a complete history of every change you make to your code, so you can always go back, and it's how you'll hand in your work.

- **Windows:** install **Git for Windows** from git-scm.com. Accept the default options.
- **macOS:** open Terminal and type **git --version**. If Git isn't installed, macOS offers to install it for you. Say yes.
- **Linux:** **sudo apt install git -y**

Then tell Git who you are. This name and email are attached to every commit you make:

```
git config --global user.name "Your Full Name"
git config --global user.email "the-email-you-use-for-github@example.com"
```

4. Create a GitHub account

If you don't already have one, sign up at github.com. Choose a professional-looking username. This account will hold your work long after this course ends, and employers do look.

5. Get the course materials

The course itself lives in a public GitHub repository. **Clone** it (download a linked copy) once, to a folder where you keep your code:

```
git clone https://github.com/datlogix/Java4Beginners.git
```

This copy is **read-only for you**. Treat it as your textbook. Whenever your instructor says the course has been updated, get the latest version with:

```
cd Java4Beginners
git pull
```

6. Create your own coursework repository

Your own work goes in **your own repository**, not in the course copy.

1. On GitHub, click **+ → New repository**.
2. Name it **java-coursework**, set it to **Private**, and tick **Add a README file**. Under **Add .gitignore**, choose **Java**: it stops Git from saving the compiled **.class** files Java creates. Click **Create repository**.
3. Go to **Settings → Collaborators → Add people** and add your instructor's GitHub username (they'll tell you what it is). This is how your instructor sees and grades your work. Nothing is emailed or uploaded anywhere else.
4. Clone it next to the course folder:

```
git clone https://github.com/YOUR-USERNAME/java-coursework.git
```

You should now have two folders side by side:

```
Java4Beginners/    <- the course (read it, run it, don't edit it)
java-coursework/   <- your work (edit, commit, push)
```

For every module, create a matching folder in **java-coursework**, such as **01-first-steps/**, and copy that module's **exercises/** and **project/** starter files into it before you start.

7. Make your first commit

Open your **java-coursework** folder in VS Code (**File → Open Folder...**). Create a folder called **00-setup** and, inside it, a file called **Hook.java** containing:

```
public class Hook {
    public static void main(String[] args) {
        System.out.println("Hello from YOUR NAME");
        System.out.println("I just ran my first Java file.");
        javax.swing.JOptionPane.showMessageDialog(null, "Hello from YOUR NAME!");
    }
}
```

Replace both **YOUR NAME**s with your name (keep the quotation marks), then run it with the  button, or from the terminal:

```
java 00-setup/Hook.java
```

Two lines appear in the terminal, and a window pops up with your name in it. Click **OK** to close it and end the program.

Now save it to your Git history and send it to GitHub:

```
git add 00-setup/Hook.java
git commit -m "Complete Module 0 setup"
git push
```

The first time you push, Git asks you to sign in to GitHub. On Windows a browser window opens. On macOS/Linux, if it asks for a password, use a [personal access token](#) instead (your normal GitHub password won't work here).

Refresh your repository page on GitHub. You should see **00-setup/Hook.java** and your commit message. **This is your workflow for every module from now on:** edit code locally, **git add** the files you changed, **git commit** with a short description, then **git push**. Your instructor only sees what you've pushed.

Common setup problems

- **java or javac is "not recognized" or "command not found"**. On Windows, *Add to PATH* wasn't selected in the installer. Run the installer again and choose *Modify*. On any system, close and reopen the terminal (and VS Code) after installing.
- **java -version works but javac doesn't**. You installed a JRE, or an old Java that came with another program. Install the JDK from [adoptium.net](#) as above.
- **java -version and javac -version show different numbers**. You have more than one Java installed, and the older one comes first on your PATH. Uninstall the old one, or ask your instructor to help you fix the PATH.
- **VS Code says "Java runtime could not be located", or the  button never appears**. Make sure you opened a *folder* (not just a single file), and that the *Extension Pack for Java* is installed. Then press **Ctrl+Shift+P** (**Cmd+Shift+P** on a Mac), type **Java: Configure Java Runtime**, and check it found your JDK.
- **error: class Test is public, should be declared in a file named Test.java**. The file name and the class name don't match. Java is fussy about this, capital letters included.
- **git push rejects your password**. GitHub doesn't accept account passwords from the terminal. Use a personal access token, or install the [GitHub CLI](#) and run **gh auth login** once.

- **Nothing shows up on GitHub after `git push`.** Run `git status`. If it says "nothing to commit", you probably forgot `git add` before `git commit`.
- **You typed `java Hook.java` at the `jshell>` prompt** (or Java code straight into the terminal). The `jshell>` prompt (Java) and your terminal prompt (your operating system) are two different places. If you see `jshell>`, type `/exit` to get back to the terminal.

Try it yourself

1. Confirm `java -version`, `javac -version` and `git --version` all work.
2. Run the three hook lines at the `jshell>` prompt.
3. Clone the course repo, and create, clone, and share your `java-coursework` repo with your instructor.
4. Add `00-setup/Hook.java`, run it, commit, and push.
5. Confirm your commit appears on GitHub.

Module 1: First Steps

Hook: a spiral in a window

Open `examples/Ex00Spiral.java`, or better, create a file called `Spiral.java` in your `java-coursework/01-first-steps` folder and type it in yourself (don't worry about understanding it yet; just make sure every bracket and semicolon matches). Then run it:

```
java Ex00Spiral.java
```

A window opens and a glowing rainbow spiral appears on a black background. Close the window when you've enjoyed it.

Now find the line `heading = heading + 59;`, change `59` to `90`, and run it again. Then try `121`. Then `144`. One number changes the whole picture, because the program is a list of instructions, and the computer follows them *exactly*. By the end of this module you'll understand the shape of this program and how its drawing commands work, and in the module project you'll draw your own initials. (The `for` line, which repeats the indented lines 180 times, gets a whole module of its own: Module 5.)

If you typed the program yourself and got errors, good: keep them. This module teaches you how to read them.

What is a program, really?

A **program** is a list of instructions for a computer, written in a language both you and the computer can work with. Computers only truly understand **machine code**, long strings of numbers that are almost impossible for people to read. So we write in a **programming language** like Java, designed to be readable by humans, and let other programs translate it for us.

Java does this translation in **two steps**:

you write <code>Hello.java</code> (Java code)	--->	the compiler <code>javac</code> checks and translates it to <code>Hello.class</code> (bytecode)	--->	the Java Virtual Machine <code>java</code> runs the bytecode on your computer
---	------	---	------	---

1. The **compiler**, `javac`, reads your whole `.java` file, checks that it follows Java's grammar rules, and translates it into **bytecode**, saved in a `.class` file. If it finds a mistake, it refuses to produce anything, and tells you what's wrong.
2. The **Java Virtual Machine** (the **JVM**), started with the `java` command, runs that bytecode.

Why two steps? Bytecode isn't tied to one kind of computer. The same `Hello.class` file runs on Windows, macOS, Linux, a server or an Android-style device, as long as it has a JVM. Java's original slogan was "**write once, run anywhere.**" That's a big reason Java runs banking systems, Android apps, huge websites, and the software in devices from smart cards to Blu-ray players.

Other languages make other choices. C and C++ compile straight to machine code for one kind of computer. Python skips the separate compile step and runs your code directly. Java sits in between.

The shape of every Java program

Here's the smallest useful Java program, `examples/Ex01Hello.java`:

```
public class Ex01Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

There's more ceremony here than you might expect for one line of output. For now, treat the outer lines as a **frame** that every program needs, and learn what each part is for:

Part	What it means (for now)
<code>public class Ex01Hello</code>	Every piece of Java code lives inside a class . This one is called <code>Ex01Hello</code> . The file must be called <code>Ex01Hello.java</code> , with exactly the same spelling and capitals.
<code>{ ... }</code>	Braces mark where something starts and ends. The class's braces hold everything in the class.
<code>public static void main(String[] args)</code>	The main method : the place your program starts. When you run the program, Java looks for exactly this line and runs what's between its braces.
<code>System.out.println(...)</code>	An instruction (a statement) that prints a line of text.
<code>;</code>	Every statement ends with a semicolon , like a full stop at the end of a sentence.

You'll understand `public`, `static`, `void` and `String[] args` properly by Modules 8 and 10. Until then, type them exactly as shown. VS Code can type the frame for you: in a new, empty `.java` file, type `main` and press Enter to get the `main` line and its braces.

Indentation (the spaces at the start of lines) doesn't change what a Java program does: Java uses braces, not spaces, to see where things start and end. But it's how *humans* see the structure, so always indent the inside of each pair of braces by four spaces. VS Code does it for you, and **Shift+Alt+F** (Windows/Linux) or **Shift+Option+F** (Mac) tidies a whole file.

Three ways to run Java

1. jshell, for quick experiments

Type `jshell` in a terminal. You get the `jshell>` prompt you met in Module 0:

```
jshell> 2 + 3
$1 ==> 5
jshell> System.out.println("Hello!")
Hello!
jshell> 7 * 6
$2 ==> 42
```

jshell reads what you type, evaluates it, prints the result, and loops back for more (a **REPL**: Read-Evaluate-Print Loop). You don't need a class or a `main` method, and you can even leave out the semicolon. It's perfect for "what happens if I do this?" Nothing you type there is saved. Type `/exit` to leave.

2. One command: `java File.java`

For anything you want to keep, write your code in a `.java` file and run it:

```
java Ex01Hello.java
```

This compiles the file in memory and runs it straight away. It's the quickest way to run a single-file program, and it's what VS Code's  button does for you. **Almost everything in Part 1 runs this way.**

3. The classic two steps: `javac`, then `java`

This is what's happening behind the scenes, and what real projects do:

```
javac Ex01Hello.java    # compile: creates Ex01Hello.class
java Ex01Hello          # run the class (no .java, no .class on the end)
```

Try it, and look in the folder afterwards: a new `Ex01Hello.class` file appeared. That's the bytecode. Open it in VS Code if you're curious: it's not meant for humans. From Module 10 onwards, when programs are split across several files, you'll use these two steps.

The #1 beginner mix-up: the `jshell>` prompt and your terminal's prompt are two different places. `java Ex01Hello.java` is a terminal command. Typed at `jshell>`, it's an error. If you see `jshell>`, type `/exit` first.

Your first instructions: printing

```
System.out.println("Hello, world!");
```

- `System.out` is the **standard output**: the terminal.
- `println` ("print line") is a **method**: a named, ready-made piece of code that does a job. Its job is to show something and then move to a new line.
- The **parentheses** `( )` are how you *call* (use) a method. Whatever you put inside them is what you're handing to it.
- `"Hello, world!"` is a **string**: a piece of text. The double quotes tell Java "this is text, not code." In Java, text **must** use double quotes.

You can join text and other values together with `+`:

```
System.out.println("I am " + 17 + " years old");    // I am 17 years old
System.out.println("2 + 3 = " + (2 + 3));           // 2 + 3 = 5
System.out.println("2 + 3 = " + 2 + 3);             // 2 + 3 = 23 (!)
System.out.println(7 * 6);                          // 42
System.out.println();                              // an empty line
```

Notice that `2 + 3` *without* quotes is calculated, while `"2 + 3 = "` *with* quotes is printed exactly as written. Quotes are the difference between "say this" and "work this out." And look closely at the third

line. Java works from left to right: `"2 + 3 = " + 2` joins the text and the 2 into `"2 + 3 = 2"`, and then `+ 3` joins a 3 on the end. The brackets in the second line make Java add the numbers *first*. Module 2 explains exactly why.

`print` (without `ln`) prints *without* moving to a new line:

```
System.out.print("Loading");
System.out.print("...");
System.out.println("done!");           // Loading...done!
```

Some characters can't be typed directly inside a string. A backslash `\` gives the next character a special meaning, called an **escape sequence**:

Escape	Means	Example	Prints
<code>\n</code>	new line	<code>"one\ntwo"</code>	one and two on separate lines
<code>\t</code>	tab	<code>"Name:\tAma"</code>	Name: Ama
<code>\"</code>	a double quote	<code>"say \"hi\""</code>	say "hi"
<code>\\</code>	a backslash	<code>"C:\\Users"</code>	C:\Users

See [examples/Ex02PrintManyThings.java](#).

Comments: notes for humans

Java ignores comments completely. They're for the people reading your code, including you, three weeks from now. There are three kinds:

```
// A line comment: everything after // on this line is ignored.

/* A block comment can
   span several lines. */

/** A Javadoc comment, which documentation tools read (Module 8). */
```

Good comments explain **why**, not what. `System.out.println("Welcome!"); // prints Welcome` adds nothing. `// Greet first so the user knows the program started` is useful. A handy trick: put `//` in front of a line to switch it off temporarily without deleting it ("commenting it out"). In VS Code, **Ctrl+/** (**Cmd+/** on a Mac) comments out the current line. See [examples/Ex03Comments.java](#).

Programs run top to bottom

Java runs the statements inside `main` **one at a time, from the first to the last**. Order matters:

```
System.out.println("1. Put on socks");
System.out.println("2. Put on shoes");
```

Swap those lines and the computer happily tells you to put your shoes on first. It doesn't know what socks are. It just follows the order you gave it. See [examples/Ex04TopToBottom.java](#).

Errors are messages, not failures

You *will* see errors constantly. Every programmer does, every day. An error message is Java telling you exactly what confused it and where. Learning to read them is one of the most valuable skills in this whole course. Java has two very different kinds.

Compile-time errors: nothing runs

The compiler checks your *whole* file before any of it is allowed to run. Here's a file with a missing semicolon:

```
public class Ex05CompileError {
    public static void main(String[] args) {
        System.out.println("Line one");
        System.out.println("Line two is missing something")
        System.out.println("Line three");
    }
}
```

Running it gives:

```
Ex05CompileError.java:11: error: ';' expected
    System.out.println("Line two is missing something")
                                   ^
1 error
error: compilation failed
```

Read it piece by piece:

1. **Ex05CompileError.java:11**: the file, and the **line number** (line 11 of the real example file, which starts with comments).
2. **error: ';' expected**: what the compiler wanted and didn't find.
3. The line of code itself, with a **caret** pointing at the exact spot.

Notice that **Line one** did **not** print, even though line one was fine. A compile-time error means **nothing runs at all**. VS Code underlines these mistakes in red as you type, before you even press . Hover the mouse over the red squiggle to read the message.

Runtime errors (exceptions): the program crashes partway

Some problems can only be discovered while the program is running. This file compiles perfectly:

```
System.out.println("Sharing 10 sweets between 0 friends...");
System.out.println("Each friend gets " + (10 / 0));
System.out.println("Finished.");
```

But running it gives:

```
Sharing 10 sweets between 0 friends...
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Ex06RuntimeError.main(Ex06RuntimeError.java:12)
```

This is a **stack trace**. Java calls a runtime error an **exception**:

1. **The first line** says *what* went wrong: an **ArithmeticException**, because you can't divide by zero.

2. The **at** line says *where*: in the **main** method of **Ex06RuntimeError**, file **Ex06RuntimeError.java**, line 12.

Notice that the first line *did* print. Java ran it, then stopped the moment it hit the bad line. **Finished.** never printed.

Errors you'll meet this week

Error	Kind	Usually means	Example
<code>';' expected</code>	compile	A statement is missing its semicolon. The caret often points at the <i>end</i> of the line.	<code>System.out.println("hi")</code>
<code>cannot find symbol</code>	compile	A misspelt name. The symbol : line under the message says which name.	<code>System.out.printn("hi");</code>
<code>package system does not exist</code>	compile	A capital letter is missing: it's System , not system .	<code>system.out.println("hi");</code>
<code>unclosed string literal</code>	compile	A string is missing its closing <code>"</code> (or used <code>'</code> at one end).	<code>println("hi");</code>
<code>class X is public, should be declared in a file named X.java</code>	compile	The class name and the file name don't match.	<code>public class Greeting</code> in <code>Hello.java</code>
<code>ArithmeticException: / by zero</code>	runtime	A whole number was divided by zero.	<code>10 / 0</code>

See `examples/Ex05CompileError.java` and `examples/Ex06RuntimeError.java`. Both are broken on purpose. Run each, read the message, then fix it.

Several errors at once? The compiler reports every problem it can find, and one mistake can confuse it into reporting several, like a missing quote making it misread the rest of the line. **Always fix the first error first, then run again.** Often the others disappear.

Drawing in a window

The hook drew in a window using Java's built-in graphics tools, which come with every JDK.

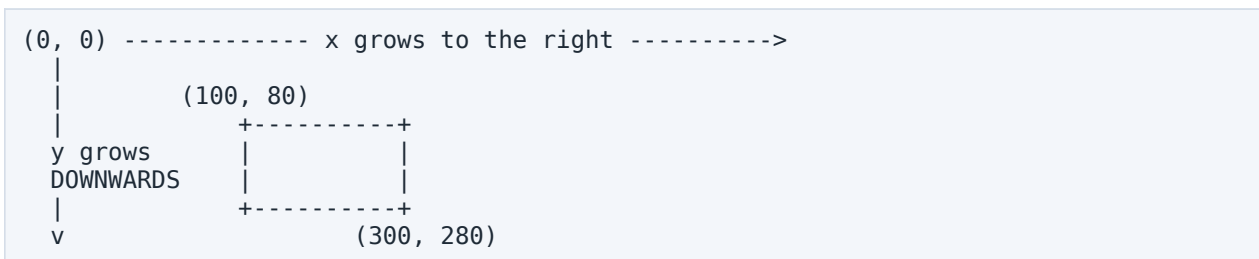
`examples/Ex07DrawingSquare.java` shows the pattern you'll use. All your drawing goes inside a **draw** section, and the code below it (which opens the window) stays the same:

```
void draw(Graphics2D pen) {
    pen.setColor(Color.BLUE);
    pen.setStroke(new BasicStroke(6));           // a thicker pen
    pen.drawRect(100, 80, 200, 200);             // x, y, width, height
    pen.setColor(Color.ORANGE);
    pen.fillOval(150, 130, 100, 100);           // a filled circle
}
```

pen is the drawing tool, and every command is a method you call on it:

Command	What it does
<code>pen.drawLine(x1, y1, x2, y2)</code>	Draw a straight line from <code>(x1, y1)</code> to <code>(x2, y2)</code>
<code>pen.drawRect(x, y, w, h) /</code> <code>pen.fillRect(...)</code>	A rectangle outline / a filled rectangle. <code>(x, y)</code> is its top-left corner
<code>pen.drawOval(x, y, w, h) /</code> <code>pen.fillOval(...)</code>	An oval that fits inside that rectangle. Make <code>w</code> and <code>h</code> equal for a circle
<code>pen.drawArc(x, y, w, h, start, angle)</code>	Part of an oval: start at <code>start</code> degrees and sweep <code>angle</code> degrees
<code>pen.setColor(Color.RED)</code>	Change the colour. Try <code>BLUE</code> , <code>GREEN</code> , <code>ORANGE</code> , <code>MAGENTA</code> , <code>BLACK</code> ... or mix your own with <code>new Color(255, 140, 0)</code>
<code>pen.setStroke(new BasicStroke(5))</code>	Change the line thickness
<code>pen.setFont(new Font("SansSerif",</code> <code>Font.BOLD, 24))</code>	Choose a font, a style and a size for text
<code>pen.drawString("Hi", x, y)</code>	Write text, starting at <code>(x, y)</code>

The window's **coordinates** work like this:



`(0, 0)` is the **top-left corner**, and `y` grows *downwards*. Most graphics systems work this way, because screens are drawn line by line from the top.

Don't worry yet about the `import` lines, `extends JPanel`, or why some code says `pen.` and some says `Color.` For now, copy the pattern. Modules 10 and 11 explain most of it, and the optional Appendix A explains every line. Re-read the hook now: you can recognise `setColor`, `setStroke` and `drawLine`, and the frame around them. The `for` loop and the maths that turns a heading into `x` and `y` arrive in Modules 2 and 5.

Common beginner mistakes

- **Capital letters.** Java is **case-sensitive**: `System` works, `system` doesn't; `String` works, `string` doesn't. `main` must be lower case.
- **A missing semicolon** at the end of a statement. Lines that end with `{` or `}` don't need one; statements do.
- **The file name doesn't match the class name.** `public class Hello` must live in `Hello.java`, capital H included.
- **Single quotes around text.** `'Hello'` is an error in Java. Text uses double quotes: `"Hello"`. (Single quotes are for single characters, as Module 2 explains.)
- **Mismatched braces or brackets.** Every `{` needs a matching `}` and every `(` a matching `)`. VS Code highlights the partner when you click next to one.

- **Forgetting the brackets around a sum** when joining it to text: `"Total: " + 2 + 3` prints `Total: 23`.
- **Forgetting to save** before running. VS Code shows a dot ● on the file's tab when it has unsaved changes. Turn on **File → Auto Save** to stop this happening.
- **Changing the code, but the output doesn't change.** If you're using the two-step method, `java Ex01Hello` runs the *old* `.class` file until you run `javac` again. Recompile after every change (or use `java Ex01Hello.java`, which always compiles first).
- **Trying to fix every error message at once.** Fix the *first* one, run again, repeat.

Try it yourself

1. Run every file in `examples/`, including the two broken ones (fix them after you've read their error messages). Run `Ex01Hello` both ways: `java Ex01Hello.java`, and `javac` then `java`.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: your initials in Java art.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 1: first steps"
git push
```

Module 2: Values, Types & Operators

Hook: six lines of arithmetic that lie to you (or do they?)

Before you run anything, copy these six lines onto paper and write down what you think each one prints:

```
System.out.println(7 / 2);
System.out.println(0.1 + 0.2);
System.out.println(2147483647 + 1);
System.out.println('A' + 1);
System.out.println(1 + 2 + "3");
System.out.println("1" + 2 + 3);
```

Now run `examples/Ex00MathSurprises.java`:

```
java Ex00MathSurprises.java
```

```
3
0.30000000000000004
-2147483648
66
33
123
```

Seven divided by two is three? Adding one to a positive number gives a huge *negative* number? A letter plus one is 66? None of these are bugs in Java. Every one follows a precise rule, and by the end of this module you'll be able to explain all six, and predict lines like them before you run them. Programmers who can't are the ones who ship banking software that loses pesewas.

Every value has a type

A **value** is a single piece of data: `42`, `3.75`, `true`, `'J'`, `"Java"`. Every value in Java has a **type**, which decides what the value is allowed to be and what you can do with it.

Type	What it holds	Examples
<code>int</code>	Whole numbers, from about -2.1 billion to +2.1 billion	<code>42</code> , <code>-7</code> , <code>0</code> , <code>1_000_000</code>
<code>long</code>	Much bigger whole numbers (end the literal with <code>L</code>)	<code>8_000_000_000L</code>
<code>double</code>	Numbers with a decimal point	<code>3.75</code> , <code>-0.5</code> , <code>2.0</code> , <code>6.02e23</code>
<code>boolean</code>	Exactly two values: <code>true</code> or <code>false</code>	<code>true</code> , <code>false</code>
<code>char</code>	One character, in single quotes	<code>'J'</code> , <code>'7'</code> , <code>'?'</code> , <code>' '</code>
<code>String</code>	Text: zero or more characters, in double quotes	<code>"Java"</code> , <code>"Room 101"</code> , <code>""</code>

The first five are Java's **primitive types**: simple, built into the language, written in lower case. `String` is different (notice its capital S): it's a **class**, a more complex type with lots of built-in abilities. Module 3 explores strings properly. (Java has three more primitives you'll rarely need: `byte`, `short` and `float`.)

The underscores in `1_000_000` are only there to help humans read the number; Java ignores them.

Storing a value: a first look at variables

To keep a value and use it later, you put it in a **variable**: a named box that holds one value of one type. In Java you must say the type when you create the variable:

```
int age = 17;
double temperature = 36.6;
boolean isRaining = false;
char grade = 'A';
String name = "Ama";

System.out.println(name + " is " + age);    // Ama is 17
```

Read `int age = 17;` as "make a box for an `int`, call it `age`, and put 17 in it." Once a box has a type, it can only ever hold that type: you can't put text in an `int` box. Module 3 covers variables in depth. For now, it's enough to know that the type comes first. See `examples/Ex01Types.java`.

Arithmetic

Operator	Meaning	Example	Result
<code>+</code>	add	<code>7 + 2</code>	<code>9</code>
<code>-</code>	subtract	<code>7 - 2</code>	<code>5</code>
<code>*</code>	multiply	<code>7 * 2</code>	<code>14</code>
<code>/</code>	divide	<code>7 / 2</code>	<code>3 (!)</code>
<code>/</code>	divide	<code>7.0 / 2</code>	<code>3.5</code>
<code>%</code>	remainder	<code>7 % 2</code>	<code>1</code>

Integer division: the first hook surprise explained

When **both** sides of `/` are whole numbers (`int` or `long`), Java does **integer division**: it works out how many *whole* times one number goes into the other and **throws the decimal part away**. It doesn't round: `7 / 2` is `3`, and `99 / 100` is `0`.

If **either** side is a `double`, you get ordinary division with a decimal answer: `7.0 / 2`, `7 / 2.0` and `7.0 / 2.0` are all `3.5`.

This catches everyone. Working out an average of whole-number marks?

```
System.out.println((70 + 85 + 90) / 3);    // 81, the .666... is gone
System.out.println((70 + 85 + 90) / 3.0);  // 81.66666666666667
```

`%`: the remainder

`%` (called **modulo** or **mod**) gives the **remainder** after integer division. Together, `/` and `%` split a number into "how many whole groups" and "how many left over", which turns out to be useful all the time:

```
System.out.println(200 / 60);    // 3   whole hours in 200 minutes
System.out.println(200 % 60);    // 20  minutes left over
System.out.println(4071 % 10);   // 1   the last digit of a number
System.out.println(17 % 2);      // 1   odd numbers leave 1, even numbers leave 0
```

See `examples/Ex02Arithmetic.java` and `examples/Ex03DivisionAndRemainder.java`.

Order of operations

Java follows the same rules as school maths: `*`, `/` and `%` happen before `+` and `-`, and operators on the same level go **left to right**. Brackets always win:

```
2 + 3 * 4      // 14, not 20
(2 + 3) * 4    // 20
20 / 2 * 5     // 50: left to right, so (20 / 2) * 5
```

If you have to stop and think about the order, add brackets. They cost nothing and make your intention obvious. See `examples/Ex04Precedence.java`.

Doubles are (nearly always) slightly wrong

The second hook surprise: `0.1 + 0.2` prints `0.30000000000000004`.

Computers store a **double** in **binary** (base 2). Just as one third can't be written exactly in decimal (`0.3333...` goes on forever), most decimal fractions, including `0.1`, can't be written exactly in binary. Java stores the nearest value it can, which is a tiny bit off, and occasionally the tiny errors show up when you print the answer. Every programming language that uses standard decimals behaves exactly the same way.

Three practical rules:

1. **Round when you display** a double. `Math.round(x)` gives the nearest whole number. To keep two decimal places, multiply by 100, round, then divide by `100.0`:

```
System.out.println(Math.round(2.4567 * 100) / 100.0);    // 2.46
```

Module 3 shows a neater way to format numbers.

2. **Never compare doubles with `==`**. `0.1 + 0.2 == 0.3` is **false**.
3. **For money, count the smallest unit as an `int`**: store 1050 pesewas, not 10.50 cedis. Whole numbers are always exact.

See `examples/Ex05DoubleSurprises.java`.

Overflow: when an `int` runs out of room

The third surprise: `2147483647 + 1` gives `-2147483648`.

An **int** is stored in exactly 32 binary digits, so it has a **largest possible value**: `2147483647`, available as `Integer.MAX_VALUE`. Go one past it, and the number **wraps around** to the most negative value, like a car's mileage counter rolling over from 99999 to 00000. This is called **overflow**, and Java doesn't warn you.

```
System.out.println(Integer.MAX_VALUE + 1);    // -2147483648
System.out.println((long) Integer.MAX_VALUE + 1); // 2147483648
```

If numbers could get bigger than about two billion (populations, bank balances in pesewas, milliseconds since 1970), use **long**. Its limit is about 9.2 quintillion (9,200,000,000,000,000).

Text, characters, and what + really does

When at least one side of `+` is a **String**, `+` **joins** (concatenates) instead of adding. Any number on the other side is turned into text first:

```
"Java" + "Script"    // "JavaScript"
"Room " + 101        // "Room 101"
```

And because Java works **left to right**, when the String appears matters. That explains the last two hook lines:

```
1 + 2 + "3"          // 1 + 2 is 3 (both ints), then 3 + "3" joins: "33"
"1" + 2 + 3           // "1" + 2 joins to "12", then "12" + 3 joins: "123"
"1" + (2 + 3)         // brackets first: "1" + 5 is "15"
```

A **char** is a single character in **single** quotes. Behind the scenes, each character is stored as a number: its position in the **Unicode** table, which lists every character in every writing system. `'A'` is 65, `'B'` is 66, `'a'` is 97. So the fourth hook surprise, `'A' + 1`, is `65 + 1`, which is the **int** 66. To get a character back, **cast** it: `(char) ('A' + 1)` is `'B'`.

Strings also have useful abilities, called with a dot:

```
"=" .repeat(30)       // "=====
"Ghana".length()      // 5
```

See [examples/Ex06StringsAndChars.java](#).

Comparisons give booleans

A **comparison** asks a yes/no question, and the answer is a **boolean**:

Operator	Means	Example	Result
<code>==</code>	equal to	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	not equal to	<code>5 != 5</code>	<code>false</code>
<code>< / ></code>	less than / greater than	<code>3 < 5</code>	<code>true</code>
<code><= / >=</code>	less than or equal / greater than or equal	<code>5 >= 5</code>	<code>true</code>

Note the **double** equals sign: `==` asks "are these equal?", while a single `=` stores a value in a variable. You can combine booleans with `&&` (and), `||` (or) and `!` (not):

```
int age = 17;
System.out.println(age >= 13 && age <= 19);    // true: a teenager
```

Strings are the exception: compare them with `.equals()`, as in `"java".equals("java")`, not with `==`. Module 4 explains why, and uses all of these to make decisions. See [examples/Ex07Comparisons.java](#).

Converting between types

Java is **strict** about types. It will automatically convert a value to a type that can hold it *safely* (**widening**), such as `int` to `double`. But it refuses to do anything that might lose information unless you ask:

```
double d = 7;           // fine: 7 becomes 7.0
int i = 36.9;           // ERROR: possible lossy conversion from double to int
int i = (int) 36.9;      // fine, but i is 36: casting CHOPS, it doesn't round
```

Writing a type in brackets before a value, like `(int)` or `(double)`, is called a **cast**. It's you telling Java "I know, convert it anyway." A common use is getting a decimal answer from int division:

```
int total = 7, count = 2;
System.out.println((double) total / count); // 3.5
```

To turn **text into a number** (you'll need this for user input), use `Integer.parseInt` or `Double.parseDouble`. To turn anything into text, use `String.valueOf`, or just join it to a String:

```
int n = Integer.parseInt("42");           // the number 42
double p = Double.parseDouble("19.99");   // the number 19.99
String s = String.valueOf(42);            // the text "42"
```

`Integer.parseInt("forty-two")` crashes with a `NumberFormatException`. Module 13 shows how to handle that gracefully. See `examples/Ex08Casting.java`.

The Math class

Java comes with a `Math` class full of ready-made maths methods:

Method	Gives	Example	Result
<code>Math.sqrt(x)</code>	square root	<code>Math.sqrt(144)</code>	12.0
<code>Math.pow(a, b)</code>	a to the power b	<code>Math.pow(2, 10)</code>	1024.0
<code>Math.abs(x)</code>	distance from zero	<code>Math.abs(-7)</code>	7
<code>Math.max(a, b) / Math.min(a, b)</code>	the larger / smaller	<code>Math.max(4, 9)</code>	9
<code>Math.round(x)</code>	the nearest whole number	<code>Math.round(2.5)</code>	3
<code>Math.floor(x) / Math.ceil(x)</code>	round down / up	<code>Math.ceil(2.1)</code>	3.0
<code>Math.PI</code>	π	<code>Math.PI</code>	3.141592653589793

Java has no `^` "power" operator (`^` means something else entirely), so use `Math.pow`, or just multiply: `r * r`. See `examples/Ex09MathClass.java`. Now go back to the Module 1 hook: `Math.cos` and `Math.sin` turned the spiral's heading into how far to move in `x` and `y`.

Common beginner mistakes

- **Integer division by accident.** `5 / 9 * (f - 32)` is always 0, because `5 / 9` is 0. Write `5.0 / 9`.

- **Expecting a cast to round.** `(int) 2.99` is 2. Use `Math.round(2.99)` to get 3.
- **Joining when you meant to add.** `"Total: " + 2 + 3` is `"Total: 23"`. Add brackets: `"Total: " + (2 + 3)`.
- **Single quotes around text.** `'Hello'` is an error: single quotes are for one `char`. Text needs double quotes.
- **= versus ==.** `=` stores, `==` compares.
- **Comparing doubles with ==,** or printing them without rounding.
- **Writing ^ for "to the power of".** Use `Math.pow(a, b)`.
- **string instead of String.** Primitive types are lower case (`int`, `double`); `String` is a class and starts with a capital.

Try it yourself

1. Run every file in `examples/`. Before each line runs, predict its output. Keep a tally of how many you get right.
2. Explain all six hook lines, in one sentence each, to someone else (or in a comment in your own copy of the hook).
3. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
4. Build the module project: a conversion cheat sheet.
5. Commit and push your work:

```
git add .
git commit -m "Complete Module 2: values, types and operators"
git push
```

Module 3: Variables, Input & Strings

Hook: a program that knows things about you

Run `examples/Ex00NameAnalyser.java` and answer its two questions:

```
java Ex00NameAnalyser.java
```

```
What's your first name? Ama
What year were you born? 2007
```

```
Hello, AMA!
Your name has 3 letters. It starts with 'A' and ends with 'a'.
Backwards, it's amA.
You're about 19, so you've been alive for roughly 599,184,000 seconds.
In the year 2050 you'll be 43.
```

Run it again with a friend's name. Every program so far has printed the same thing every time. This one **listens**: it stores what you type, works things out from it, and talks back in neatly formatted sentences. That's three ideas, and they're this module: **variables** (storing values), **input** (reading the keyboard with a `Scanner`), and **strings** (taking text apart and formatting it).

Variables: named boxes

You met variables briefly in Module 2. A **variable** is a named box that holds one value. In Java, every box has a **type** that never changes:

```
String name = "Kofi";
int age = 19;
double heightInMetres = 1.78;

System.out.println(name + " is " + age + " years old.");
```

`int age = 19;` does two things: it **declares** the variable (makes a box for an `int` called `age`) and **initialises** it (puts 19 in it). You can do them separately:

```
int marks;           // declare: an empty box
marks = 74;          // assign: put a value in
```

Java won't let you use a box before something has been put in it. `System.out.println(marks);` straight after `int marks;` is the compile-time error `variable marks might not have been initialized`.

Once a variable exists, you use its **name** wherever you'd use its value. Java looks inside the box each time. See `examples/Ex01Variables.java`.

Changing a variable

A single `=` means **assign**: "work out the right-hand side, then store it in the box on the left." So this, which looks wrong in maths, is perfectly normal in programming:

```
score = score + 5;    // read score, add 5, store the answer back in score
```

It's so common that Java has shortcuts:

Shortcut	Same as
<code>score += 10;</code>	<code>score = score + 10;</code>
<code>score -= 3;</code>	<code>score = score - 3;</code>
<code>score *= 2;</code>	<code>score = score * 2;</code>
<code>score /= 4;</code>	<code>score = score / 4;</code>
<code>score++;</code>	<code>score = score + 1;</code>
<code>score--;</code>	<code>score = score - 1;</code>

`++` and `--` will be everywhere once you meet loops in Module 5. See

`examples/Ex02UpdatingVariables.java`, which also shows how to **swap** two variables (you need a third, temporary box).

Naming variables

The rules (break them and the program won't compile):

- Letters, digits, `_` and `$` only, and the name can't **start** with a digit. `player2` is fine, `2ndPlayer` isn't.
- No spaces. `my score` is two names.
- Not a **reserved word** that already means something in Java: `class`, `int`, `public`, `new`, `if`...
- Case matters: `age`, `Age` and `AGE` are three different variables.

The **style** every Java programmer uses (break this and your code compiles, but looks wrong to everyone who reads it):

- **camelCase** for variables: start lower case, and capitalise each new word. `numberOfStudents`, `averageMark`, `isLoggedIn`.
- Choose names that say what the value **means**. `speedKmPerHour` beats `s`, `x` or `data`.
- Booleans read well as questions: `isFull`, `hasPaid`, `canVote`.

Constants: values that must never change

Add `final` and the variable can be assigned only once. Try to change it and the compiler refuses. By convention, constants are written in **UPPER_SNAKE_CASE**:

```
final double VAT_RATE = 0.15;
final int SECONDS_PER_HOUR = 3600;

VAT_RATE = 0.2;    // error: cannot assign a value to final variable VAT_RATE
```

Remember the Module 2 project, where you typed `0.621371` again and again? A constant fixes that: type the number once, give it a name, and if it ever changes, you change it in one place.

var: let Java work out the type

Inside a method, you can write `var` instead of the type, and Java works out the type from the value:

```
var town = "Kumasi";    // Java decides: String
var count = 3;          // Java decides: int
count = "three";        // still an error: count is an int forever
```

`var` saves typing, but the type is just as fixed as before. This course mostly writes types out in full, because while you're learning, seeing the type is helpful. See [examples/Ex03NamingAndConstants.java](#).

Input: reading the keyboard with Scanner

To read what the user types, you use a **Scanner**. It isn't automatically available like `System` and `Math`, so you **import** it at the very top of the file, above the class:

```
import java.util.Scanner;

public class Greeter {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);    // one Scanner for the keyboard

        System.out.print("What's your name? ");
        String name = in.nextLine();            // waits for the user to press Enter

        System.out.println("Hello, " + name + "!");
    }
}
```

- `import java.util.Scanner;` says "I'll be using the **Scanner** class, which lives in the `java.util` package." (Module 17 explains packages.)
- `new Scanner(System.in)` creates a Scanner that reads from the keyboard. Create **one**, near the top of `main`, and reuse it.
- `print` (not `println`) leaves the cursor on the same line as the question, which looks nicer.

The Scanner can read different types:

Method	Reads	Example input
<code>in.nextLine()</code>	A whole line of text, up to Enter	Kofi Boateng
<code>in.nextInt()</code>	One whole number	19
<code>in.nextDouble()</code>	One decimal number	1.78
<code>in.next()</code>	One word (stops at a space)	Kofi

If the user types `nineteen` when `nextInt()` expects a number, the program crashes with an `InputMismatchException`. That's fine for now. Module 13 shows how to recover. See [examples/Ex04Input.java](#).

The `nextLine` trap

Run [examples/Ex05TheNextLineTrap.java](#) and watch it **skip** a question:

```
System.out.print("Age: ");
int age = in.nextInt();      // you type 19 and press Enter
System.out.print("Name: ");
String name = in.nextLine(); // doesn't wait! name is ""
```

When you type **19** and press Enter, the keyboard sends **19** followed by an invisible "end of line" character. `nextInt()` takes the **19** and **leaves the end of line behind**. Then `nextLine()` reads up to the next end of line, finds one immediately, and returns an empty string.

There are two fixes. You can call `in.nextLine()`; on its own after each `nextInt()` to throw the leftover away. Or, simpler and safer, **always read whole lines, and convert them yourself**:

```
System.out.print("Age: ");
int age = Integer.parseInt(in.nextLine().trim());
```

This course uses the second pattern everywhere: `nextLine()` for every answer, then `Integer.parseInt` or `Double.parseDouble` for numbers. The `.trim()` removes any spaces the user accidentally typed around the number.

Formatting output with `printf`

Joining strings with `+` gets messy, and it can't line things up or control decimal places. `printf` ("print formatted") can. You write the text once with **placeholders**, and list the values to drop in:

```
String item = "Notebook";
int quantity = 3;
double price = 4.5;

System.out.printf("%d x %s costs GHS %.2f%n", quantity, item, quantity * price);
// 3 x Notebook costs GHS 13.50
```

Placeholder	For	Example	Gives
<code>%s</code>	a String (or anything)	<code>printf("%s", "Ama")</code>	Ama
<code>%d</code>	a whole number	<code>printf("%d", 42)</code>	42
<code>%.2f</code>	a decimal, 2 places (rounded)	<code>printf("%.2f", 3.14159)</code>	3.14
<code>%,d</code>	a whole number with commas	<code>printf("%,d", 34000000)</code>	34,000,000
<code>%n</code>	a new line		
<code>%%</code>	a real percent sign	<code>printf("15%%")</code>	15%

A number before the letter sets a minimum **width**, so columns line up. A `-` means "line up on the left":

```
System.out.printf("[%10s]%n", "right"); // [      right]
System.out.printf("[%10s]%n", "left");  // [left      ]
System.out.printf("[%8.2f]%n", 3.14159); // [    3.14]
```

`printf` doesn't add a new line by itself, so remember the `%n`. The values must match the placeholders in number, order and type: `%d` with a `double` crashes with an `IllegalFormatConversionException`.

`String.format` works exactly like `printf`, but **gives back** the text instead of printing it, so you can store it in a variable:

```
String label = String.format("%s (%d)", item, quantity);    // "Notebook (3)"
```

See [examples/Ex06Printf.java](#).

What strings can do

A **String** comes with dozens of built-in **methods**, called with a dot:

Method	Gives	<code>"Hello, Java! ".trim()</code> then...
<code>s.length()</code>	the number of characters	12
<code>s.toUpperCase()</code> / <code>s.toLowerCase()</code>	a copy in capitals / small letters	"HELLO, JAVA!"
<code>s.trim()</code>	a copy without spaces at the ends	"Hello, Java!"
<code>s.contains(t)</code>	does <code>s</code> contain <code>t</code> ?	<code>contains("Java")</code> is <code>true</code>
<code>s.startsWith(t)</code> / <code>s.endsWith(t)</code>	does it start / end with <code>t</code> ?	<code>endsWith("!")</code> is <code>true</code>
<code>s.indexOf(t)</code>	where <code>t</code> first appears, or -1	<code>indexOf("Java")</code> is 7
<code>s.replace(a, b)</code>	a copy with every <code>a</code> replaced by <code>b</code>	<code>replace("Java", "Ghana")</code>
<code>s.equals(t)</code>	is it exactly the same text?	
<code>s.equalsIgnoreCase(t)</code>	the same text, ignoring capitals?	
<code>s.isEmpty()</code>	is it ""?	false

Strings never change

Strings are **immutable**: no method changes the original string. Each one *gives back a new string*, and if you don't store it, it's lost:

```
String shout = "quiet";  
shout.toUpperCase();           // the new string is thrown away  
System.out.println(shout);     // quiet  
shout = shout.toUpperCase();    // store the result  
System.out.println(shout);     // QUIET
```

See [examples/Ex07StringMethods.java](#).

Characters and pieces: `charAt` and `substring`

Each character in a string has a position, called its **index**. Indexes start at **0**, not 1:

E	N	G	I	N	E	E	R
0	1	2	3	4	5	6	7

```
String word = "ENGINEER";  
word.charAt(0)           // 'E'  the first character  
word.charAt(word.length() - 1) // 'R' the last character (index 7, not 8!)  
word.substring(0, 3)      // "ENG" from 0 up to, but NOT including, 3  
word.substring(5)         // "EER" from 5 to the end
```

`charAt` gives a `char`; `substring` gives a `String`. Asking for an index that doesn't exist, like `word.charAt(8)`, crashes with a `StringIndexOutOfBoundsException`.

The "up to but not including" rule seems odd at first, but it's handy: `substring(0, 3)` gives exactly 3 characters, and `substring(3, 6)` carries on from where it stopped. Combine `indexOf` and `substring` to split text:

```
String fullName = "Abena Owusu";  
int space = fullName.indexOf(" ");           // 5  
String first = fullName.substring(0, space);  // "Abena"  
String last = fullName.substring(space + 1);  // "Owusu"
```

See `examples/Ex08CharAtAndSubstring.java`, and then `examples/Ex09Receipt.java`, which puts the whole module together. Now re-read the hook: you can explain every line except `java.time.Year.now()` (which asks the computer's clock for the current year) and `new StringBuilder(name).reverse()` (a ready-made way to reverse text, which Module 5 shows you how to do by hand).

Common beginner mistakes

- **Using a variable before giving it a value:** "variable might not have been initialized".
- **Declaring the same variable twice.** `int age = 19;` then later `int age = 20;` is an error ("variable age is already defined"). The second time, leave out the type: `age = 20;`.
- **The `nextLine` trap:** a question gets skipped after `nextInt()`. Read every answer with `nextLine()` and convert it.
- **Forgetting `import java.util.Scanner;`**: "cannot find symbol: class Scanner".
- **Calling a `String` method and not keeping the answer.** `name.trim();` on its own does nothing useful. Write `name = name.trim();`.
- **Off-by-one indexes.** The first character is at index 0; the last is at `length() - 1`.
- **`printf` without `%n`,** so the next output runs onto the same line.
- **Comparing strings with `==`.** Use `.equals()`. Module 4 explains why `==` sometimes seems to work and then suddenly doesn't.

Try it yourself

1. Run every file in `examples/`. In `Ex05TheNextLineTrap.java`, make sure you understand why the first name comes out empty.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: a Mad Libs story generator.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 3: variables, input and strings"  
git push
```

Module 4: Making Decisions

Hook: the birthday detective

Run `examples/Ex00BirthdayDetective.java` and type in your date of birth:

```
java Ex00BirthdayDetective.java
```

```
Day you were born (1-31): 15
Month (1-12): 3
Year: 2008
```

```
You were born on a SATURDAY.
Your Akan day name would be Kwame or Ama.
A weekend baby: your parents didn't have to take a day off work!
2008 was a leap year, with 366 days.
```

Check it against your real Akan day name (or a family member's). Then try the birthday `29, 2, 2000`. The program says something extra that it didn't say before. Every program you've written so far ran the same lines every time, from top to bottom. This one **chooses**: it looks at the data and decides which lines to run. By the end of this module you'll be able to write every decision in it, including the leap-year rule that trips up professional programmers every four years.

if: run some code only when a condition is true

```
if (temperature > 28) {
    System.out.println("It's hot. Drink plenty of water.");
}
System.out.println("This line always runs.");
```

- The **condition** goes in brackets after `if`. It must be a **boolean**: something that is either `true` or `false`, usually a comparison from Module 2 (`>`, `<`, `>=`, `<=`, `==`, `!=`).
- The **block** in braces `{ ... }` runs only when the condition is `true`. When it's `false`, Java skips straight past the closing brace.
- There's **no semicolon** after `if (...)`. (See the mistakes section for what happens if you add one.)

Indent everything inside the braces by four spaces, so you can see at a glance which lines belong to the `if`. Java doesn't need the indentation, but your readers do. See `examples/Ex01If.java`.

else: one path or the other

```
if (number % 2 == 0) {
    System.out.println(number + " is even.");
} else {
    System.out.println(number + " is odd.");
}
```

Exactly one of the two blocks runs, never both, never neither. See `examples/Ex02IfElse.java`.

else if: choosing between many paths

```
String grade;
if (mark >= 80) {
    grade = "A";
} else if (mark >= 70) {
    grade = "B";
} else if (mark >= 60) {
    grade = "C";
} else if (mark >= 50) {
    grade = "D";
} else {
    grade = "F";
}
```

Java checks the conditions **in order, top to bottom**, runs the block of the **first** one that's true, and skips the rest. So when Java reaches `mark >= 70`, it already knows the mark is below 80. That's why you don't need to write `mark >= 70 && mark < 80`.

This means **order matters**. Put `mark >= 50` first, and a mark of 95 gets a D, because 95 is indeed ≥ 50 , and Java never looks any further. See `examples/Ex03ElseIfChain.java`, which includes the broken version.

The final `else` (with no condition) catches everything that's left. It's optional, but it's usually a good idea: it's where unexpected values end up.

Combining conditions: &&, || and !

Operator	Name	True when...	Example
<code>&&</code>	and	both sides are true	<code>hr >= 60 && hr <= 100</code>
<code> </code>	or	at least one side is true	<code>hr < 60 hr > 100</code>
<code>!</code>	not	the condition is false	<code>!isNormal</code>

```
if (heartRate >= 60 && heartRate <= 100) {
    System.out.println("Normal resting heart rate.");
}
```

In maths you might write $60 \leq hr \leq 100$, but Java can't chain comparisons like that: `60 <= hr <= 100` doesn't compile. Write it as two comparisons joined by `&&`.

Java is lazy in a useful way: `&&` and `||` stop as soon as they know the answer. In `count != 0 && total / count > 50`, if `count` is 0 the left side is false, so the whole thing must be false, and Java never attempts the division that would crash. This is called **short-circuiting**. See `examples/Ex04LogicalOperators.java`.

Nested decisions

An `if` can go inside another `if`. The inner decision only happens if the outer one lets you in:

```

if (ticket.equalsIgnoreCase("yes")) {
    if (age >= 16) {
        System.out.println("Enjoy the film!");
    } else {
        System.out.println("Sorry, this film is for ages 16 and over.");
    }
} else {
    System.out.println("Please buy a ticket first.");
}

```

Each `else` belongs to the nearest `if` above it in the same pair of braces. Good indentation makes this obvious; bad indentation makes it a trap. See [examples/Ex05NestedIf.java](#).

Comparing strings: `.equals()`, never `==`

Run [examples/Ex06StringEquality.java](#) and type `yes`:

```

String answer = in.nextLine();    // you type: yes
if (answer == "yes") { ... }      // FALSE!
if (answer.equals("yes")) { ... } // true

```

Why? A `String` isn't a primitive like `int`. A `String` variable holds a **reference**: the address of where the text is stored in memory. `==` compares the *addresses*, asking "are these the very same object?" The text you typed and the `"yes"` written in your code are stored in two different places, so `==` says no, even though the characters are identical. `.equals()` compares the **characters** themselves, which is what you almost always want. (Module 10 explains references properly.)

To make matters more confusing, `==` *sometimes* seems to work with strings that are both typed into your code, because Java reuses identical string literals. Don't let that fool you. **Always** compare strings with `.equals()`, or `.equalsIgnoreCase()` to accept `YES`, `Yes` and `yes` alike.

switch: choosing by value

When you're comparing **one value** against a list of possibilities, a `switch` is often clearer than a long `else if` chain. Java has two forms.

The classic `switch` statement

```

switch (colour) {
    case "red":
        System.out.println("Stop.");
        break;
    case "amber":
        System.out.println("Get ready to stop.");
        break;
    default:
        System.out.println("That's not a traffic light colour.");
}

```

Java jumps to the matching `case` and runs from there **until it hits a `break`**. Forget the `break`, and it **falls through** into the next case's code too. That's occasionally useful (several cases sharing one block), but far more often it's a bug. `default` catches everything else. See [examples/Ex07Switch.java](#).

The modern arrow switch

Since Java 14 there's a neater form, using `->`:

```
switch (month) {
    case 12, 1, 2 -> System.out.println("Harmattan season.");
    case 4, 5, 6, 7 -> System.out.println("Major rainy season (in the south).");
    default -> System.out.println("Some other part of the year.");
}
```

No `break`, no fall-through, and several values can share a case. Even better, an arrow switch can be a **switch expression** that works out to a value you can store:

```
int days = switch (month) {
    case 4, 6, 9, 11 -> 30;
    case 2 -> 28;
    default -> 31;
}; // note the semicolon: this whole thing is one statement
```

A switch expression must cover **every** possible value, so it usually needs a `default`. This course uses the arrow form from now on. You can switch on `int`, `char` and `String` values (and on enums, in Module 12), but not on `double` or `boolean`. See `examples/Ex08SwitchExpressions.java`.

The ternary operator: a tiny if-else that gives a value

```
int daysInFebruary = leap ? 29 : 28;
```

Read it as "is `leap` true? If so, 29, otherwise 28." The pattern is `condition ? valueIfTrue : valueIfFalse`. It's handy for short, simple choices. For anything longer, an `if` is clearer.

`examples/Ex09LeapYear.java` writes the leap-year rule three ways: as an `else if` chain, as one boolean expression, and with a ternary. The rule: a year is a leap year if it divides by 4, **except** century years, which must also divide by 400. So 2024 and 2000 were leap years, but 1900 wasn't, and 2100 won't be:

```
boolean leap = (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
```

Now re-read the hook. You can explain every decision in it. The only new thing is `LocalDate`, a class from `java.time` that knows the calendar: `LocalDate.of(year, month, day).getDayOfWeek()` works out the day of the week for any date.

Common beginner mistakes

- **A semicolon after the condition.** `if (age >= 18);` ends the `if` right there, with an empty statement. The block after it then runs *every time*. No error, just a very confusing bug.
- **`=` instead of `==`.** `if (x = 5)` is a compile error in Java ("incompatible types: int cannot be converted to boolean"). At least Java catches this one.
- **Comparing strings with `==`.** Use `.equals()` or `.equalsIgnoreCase()`.
- **Chaining comparisons.** `0 <= mark <= 100` doesn't compile. Write `mark >= 0 && mark <= 100`.

- The wrong order in an `else if` chain, so a later case can never be reached.
- Missing `break` in a classic `switch`, so one case falls through into the next. The arrow form avoids this.
- Leaving out the braces. Java allows `if (x > 0) doSomething();` without braces, but only the one statement after it belongs to the `if`. Add a second indented line and it runs every time. Always use braces.
- Forgetting the user might type capitals or spaces. Use `trim()` and `equalsIgnoreCase()` (or `toLowerCase()`) on answers.

Try it yourself

1. Run every file in `examples/` more than once, with input that sends it down **every** branch.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: a choose-your-own adventure story.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 4: making decisions"
git push
```

Module 5: Loops

Hook: a million dice rolls in the blink of an eye

How often do you get a 7 when you roll two dice? You could roll them a few hundred times and keep a tally. Or run `examples/Ex00DiceMillion.java`:

```
java Ex00DiceMillion.java
```

```
Rolled two dice 1,000,000 times in 40 ms.
```

2	*****	2.74%
3	*****	5.54%
4	*****	8.34%
5	*****	11.08%
6	*****	13.86%
7	*****	16.69%
8	*****	13.89%
9	*****	11.14%
10	*****	8.39%
11	*****	5.55%
12	*****	2.78%

A million rolls, counted and charted, in less time than it takes you to blink. Run it again: the numbers wobble a little, but the shape never changes. (7 is the most common total because more pairs of dice add up to 7 than to anything else: 1+6, 2+5, 3+4, 4+3, 5+2 and 6+1.)

The program is only a few lines long, because of one idea: a **loop**, which tells the computer to repeat some code. Repetition is where computers leave humans far behind. They never get bored, never lose count, and do millions of repetitions a second. This module covers Java's three kinds of loop, and the patterns that make them useful.

while: repeat as long as a condition is true

```
int count = 10;
while (count > 0) {
    System.out.println(count + "...");
    count--;
}
System.out.println("Liftoff!");
```

A **while** loop looks like an **if**, and it starts the same way: it checks the condition, and if it's **true**, it runs the block. The difference is what happens at the closing brace: instead of carrying on, Java **goes back up and checks the condition again**. It keeps going round until the condition is **false**, then carries on after the loop.

Each trip round the loop is called an **iteration**. For a loop to finish, something inside it must eventually make the condition false. Here, that's `count--`. Delete that line and the condition stays **true** forever: an **infinite loop**. If that happens, press **Ctrl+C** in the terminal to stop the program. See

`examples/Ex01WhileCountdown.java`, which also uses `Thread.sleep(300)` to pause between numbers.

Why throws `InterruptedException`? `Thread.sleep` can, in principle, be interrupted early, and Java insists that you either handle that or declare that `main` might pass the problem on. Module 13 explains exactly what this means. For now, add it to `main` whenever you use `Thread.sleep`.

Pattern: keep asking until the answer is valid

```
System.out.print("Rate the course from 1 to 10: ");
int rating = Integer.parseInt(in.nextLine().trim());

while (rating < 1 || rating > 10) {
    System.out.print("That's not between 1 and 10. Try again: ");
    rating = Integer.parseInt(in.nextLine().trim());
}
```

The loop condition describes a **bad** answer: "keep looping while it's wrong." When the user finally types something valid, the loop ends. See `examples/Ex02InputValidation.java`.

Pattern: the sentinel loop

When you don't know in advance how many values the user will type, let them type a special **sentinel** value, such as `done`, to say they've finished:

```
System.out.print("Price: ");
String entry = in.nextLine().trim();
while (!entry.equalsIgnoreCase("done")) {
    total += Double.parseDouble(entry);
    count++;
    System.out.print("Price: ");
    entry = in.nextLine().trim();
}
```

Notice the shape: read once **before** the loop, and again at the **end** of the loop body. That way the condition always checks the most recent thing the user typed. See `examples/Ex03SentinelLoop.java`.

do-while: run first, check afterwards

```
String choice;
do {
    System.out.println("1) Say hello    2) Tell me the time    3) Quit");
    System.out.print("Choose: ");
    choice = in.nextLine().trim();
    // ... act on the choice ...
} while (!choice.equals("3"));
```

A **do-while** loop runs its block **first** and checks the condition **afterwards**, so the block always runs at least once. That's exactly right for a menu: you always want to show it at least once. Note the semicolon after the final `}`, and that `choice` is declared *before* the loop, so the condition at the bottom can see it. See `examples/Ex04DoWhile.java`.

for: counting loops

Most loops count: "do this 10 times", "for every number from 1 to 100". You can count with `while`:

```
int i = 1;           // start
while (i <= 5) {     // condition
    System.out.print(i + " ");
    i++;             // step
}
```

But this is so common that Java has a loop that puts all three parts on one line:

```
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");    // 1 2 3 4 5
}
```

Read the `for` line as: "start with `i` at 1; keep going while `i <= 5`; after each iteration, do `i++`." The order of events is: start (once), then check, body, step, check, body, step... until the check is false.

Loop	Gives
<code>for (int i = 0; i < 5; i++)</code>	0 1 2 3 4 (five numbers, starting at 0: the most common form)
<code>for (int i = 1; i <= 5; i++)</code>	1 2 3 4 5
<code>for (int i = 0; i <= 100; i += 10)</code>	0 10 20 ... 100
<code>for (int i = 10; i >= 1; i--)</code>	10 9 8 ... 1

The variable `i` is created by the loop and **only exists inside it**. Try to use it after the loop and you get "cannot find symbol". (`i` is the traditional name for a loop counter, but use a more meaningful name when there is one: `row`, `day`, `student`.)

Which loop should I use? Use `for` when you know how many times to repeat (or you're counting through a range). Use `while` when you're waiting for something to happen (a valid answer, a sentinel, a target being reached). Use `do-while` when the body must run at least once. See [examples/Ex05ForLoop.java](#).

Looping over a String

The indexes of a string go from 0 to `length() - 1`, which is exactly what the standard `for` loop counts through:

```
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    // ... do something with c ...
}
```

[examples/Ex06ForOverString.java](#) counts vowels, spells a word out with dashes, and builds a reversed copy by looping **backwards** from the last index. That's what `new StringBuilder(name).reverse()` did in the Module 3 hook.

Accumulator patterns

Many loops build up an answer, one iteration at a time, in a variable created *before* the loop. These patterns come up constantly:

```
int total = 0;                // a running total starts at 0
int count = 0;                // so does a counter
int largest = Integer.MIN_VALUE; // the smallest possible int, so anything beats it
long product = 1;             // a product starts at 1 (0 would ruin it)

for (int i = 0; i < 10; i++) {
    int mark = /* the next value */;
    total += mark;
    count++;
    largest = Math.max(largest, mark);
}
```

After the loop, `total / (double) count` gives the average. See [examples/Ex07Accumulators.java](#).

break and continue

- **break** leaves the loop **immediately**, skipping the rest of the body and any remaining iterations.
- **continue** skips the rest of **this** iteration, and goes straight on to the next one.

```
for (int n = 1001; n < 2000; n++) {
    if (n % 17 == 0 && n % 23 == 0) {
        found = n;
        break;           // we have our answer: stop looking
    }
}
```

A common shape is `while (true) { ... if (done) { break; } ... }`, a loop whose exit is in the middle. Use these sparingly: a loop with one clear condition is usually easier to read. See [examples/Ex08BreakContinue.java](#).

Nested loops

A loop can go inside another loop. **The inner loop runs completely, every time the outer loop goes round once.** That's how you work with anything that has rows and columns:

```
for (int row = 1; row <= 6; row++) {
    for (int col = 1; col <= 6; col++) {
        System.out.printf("%4d", row * col);
    }
    System.out.println();           // end of the row
}
```

With 6 rows and 6 columns, the inner body runs $6 \times 6 = 36$ times. The inner loop can even depend on the outer one: in a triangle of stars, row `r` has `r` stars. See [examples/Ex09NestedLoops.java](#).

Random numbers

Games, simulations and tests all need randomness. Java's `Random` class (import `java.util.Random`) provides it:

```
Random random = new Random();           // create one, and reuse it
int dice = random.nextInt(6) + 1;        // 1 to 6
boolean heads = random.nextBoolean();    // true or false
double chance = random.nextDouble();    // 0.0 up to (not including) 1.0
```

`random.nextInt(n)` gives a whole number from 0 up to `n - 1`. For any range from `low` to `high` (both included), use `random.nextInt(high - low + 1) + low`. `Math.random()` also gives a double from 0.0 to just under 1.0, if you'd rather not create a `Random`.

If you give `Random` a number when you create it, like `new Random(42)`, it produces the **same** "random" sequence every run. That's called a **seed**, and it's very useful for testing. See `examples/Ex10Random.java`, then play `examples/Ex11ComputerGuesses.java`: think of a number from 1 to 100, and the computer finds it in at most 7 guesses by always guessing the middle of what's left. You'll see why that works so well in Module 16.

Loops and drawing

Loops and the drawing tools from Module 1 make a great team.

`examples/Ex12DrawingWithLoops.java` draws concentric circles, a set of fading squares, and a checkerboard (nested loops!), each from a few lines of code.

Now go back to the Module 1 spiral hook. You can read the whole of its `paintComponent` method: a `for` loop runs 180 times, each line is `step * 2` pixels long (so they get longer and longer), the colour cycles through six colours using `step % 6`, and the heading turns by 59° each time. `Math.cos` and `Math.sin` turn "move this far in this direction" into how far to move in `x` and `y`. The only piece you haven't met is `Color[] colours = {...}`, a list of values called an **array**. That's next.

Common beginner mistakes

- **An infinite loop:** the condition never becomes false, because nothing in the body changes it. Press **Ctrl+C** to stop it, then check the variable in the condition.
- **Off-by-one errors:** `i <= 10` versus `i < 10`, or starting at 1 versus 0. Ask: "what's the first value, what's the last value, and how many times does it run?"
- **A semicolon after the loop line.** `for (int i = 0; i < 5; i++);` loops five times doing *nothing*, then runs the "body" once. `while (x > 0);` is even worse: an infinite loop that does nothing.
- **Resetting an accumulator inside the loop.** `int total = 0;` must go *before* the loop, or it's reset to 0 every iteration.
- **Using the loop variable after the loop.** It only exists inside.
- **Creating a new `Random` (or `Scanner`) inside a loop.** Create one before the loop and reuse it.
- **Forgetting to read again** at the end of a sentinel loop, so it checks the same old value forever.

Try it yourself

1. Run every file in `examples/`. In `Ex01WhileCountdown`, delete `count--`, run it, then stop it with Ctrl+C.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: a games arcade.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 5: loops"  
git push
```

Module 6: Arrays

Hook: a picture made of numbers

Run `examples/Ex00PixelArt.java`:

```
java Ex00PixelArt.java
```

A window opens with a green space invader, and the same invader is printed in the terminal in `#` characters. Now open the file and look at the picture's source:

```
int[][] grid = {
    {0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0},
    {0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 1, 0, 0},
    {0, 1, 1, 2, 1, 1, 1, 2, 1, 1, 0},
    ...
}
```

The whole picture is a **grid of numbers**: 0 is black, 1 is green, 2 is white. Squint and you can see the invader in the numbers. Change a few 0s to 1s, run it again, and you've edited the picture. Add a `3` and a fourth colour to the `palette` line, and you've added a colour.

Every photo on your phone works the same way: millions of numbers in a grid. Until now, each variable held **one** value. This module is about holding **many values under one name**, which is what made this picture possible: an **array**.

Why arrays?

Suppose you want to store the marks of a class of 40 students. With what you know so far, you'd need 40 variables: `mark1`, `mark2`, ... `mark40`. Working out the average would need a 40-term sum, and adding a 41st student would mean editing the program. An array solves all of this:

```
int[] marks = {72, 85, 64, 90, 58};
```

`int[]` (say "int array") is the type: an array of `ints`. The variable `marks` holds the whole collection, in order. One name, any number of values, and a loop can visit them all.

Creating arrays

There are two ways:

```
// 1. List the values, in braces, when you know them already
int[] marks = {72, 85, 64, 90, 58};
String[] days = {"Mon", "Tue", "Wed", "Thu", "Fri"};

// 2. Say how many values, and fill them in later
int[] scores = new int[40];           // 40 ints
String[] names = new String[3];       // 3 Strings
```

With **new**, Java fills every slot with a **default value**: **0** for numbers, **false** for booleans, **'\0'** (an invisible "nothing" character) for chars, and **null** for Strings. **null** means "no object at all", and calling a method on it, like `names[0].length()`, crashes with a **NullPointerException**.

Two important facts:

- **All the values have the same type.** An `int[]` can't hold a `String`.
- **The size is fixed** when the array is created. An array of 5 values stays at 5. There's no "add". (Module 7's `ArrayList` can grow.)

`marks.length` gives the size. Note: `length` with **no brackets** for arrays, but `length()` **with** brackets for Strings. Everyone mixes these up at first; the compiler will remind you.

Printing an array directly gives something like `[I@5cdd8682`: Java's code for "an array of ints, stored at this address". To see the values, use `Arrays.toString(marks)`, after `import java.util.Arrays;`. See `examples/Ex01CreatingArrays.java`.

Indexes: reading and changing values

Each value has a position, its **index**, starting at **0**, exactly like the characters of a String:

```
String[] planets = {"Mercury", "Venus", "Earth", "Mars"};
//                0         1         2         3

planets[0]           // "Mercury"
planets[planets.length - 1] // "Mars": the last one
planets[3] = "MARS";    // replace a value
```

`array[index]` works on both sides of `=`: on the right it **reads** a value, on the left it **stores** one. And it works with every operator you know: `counts[2]++`, `totals[i] += price`.

Ask for an index that doesn't exist and the program crashes:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 4 out of
bounds for length 4
    at Ex02IndexingAndChanging.main(Ex02IndexingAndChanging.java:28)
```

The message is very helpful: you asked for index 4, but the array only has 4 values (indexes 0 to 3). See `examples/Ex02IndexingAndChanging.java`.

Looping over arrays

Arrays and **for** loops were made for each other. There are two styles:

```
// The index loop: you get the position i, and can read or change fruits[i]
for (int i = 0; i < fruits.length; i++) {
    System.out.println((i + 1) + ". " + fruits[i]);
}

// The for-each loop: "for each String fruit in fruits"
for (String fruit : fruits) {
    System.out.println(fruit.toUpperCase());
}
```

The **for-each** loop is shorter and impossible to get off-by-one, so use it when you just need each **value**. Use the **index** loop when you need the **position**, when you're working with two arrays at once, or when you want to **change** the values: assigning to **fruit** inside a for-each loop changes the loop variable, not the array. See [examples/Ex03LoopingOverArrays.java](#).

The algorithms you'll use again and again

Most of what programs do with arrays is built from a few patterns:

```
// Total (and so the average)
double total = 0;
for (double t : temperatures) {
    total += t;
}
double average = total / temperatures.length;

// The maximum, and WHERE it is: track the index, not just the value
int hottest = 0;
for (int i = 1; i < temperatures.length; i++) {
    if (temperatures[i] > temperatures[hottest]) {
        hottest = i;
    }
}

// Linear search: where is a value? (-1 means "not found")
int position = -1;
for (int i = 0; i < days.length; i++) {
    if (days[i].equals("Fri")) {
        position = i;
        break;
    }
}
```

Counting the values that match a condition is the counter pattern from Module 5, inside a for-each loop. See [examples/Ex04ArrayAlgorithms.java](#).

The Arrays class

`java.util.Arrays` has ready-made versions of the jobs you'd otherwise write by hand:

Method	Does
<code>Arrays.toString(a)</code>	Gives a readable String like <code>[3, 7, 19]</code>
<code>Arrays.sort(a)</code>	Sorts the array in place : numbers smallest first, Strings alphabetically
<code>Arrays.copyOf(a, n)</code>	Gives a new array of length <code>n</code> with <code>a</code> 's values (padded with defaults if <code>n</code> is bigger)
<code>Arrays.fill(a, v)</code>	Sets every value to <code>v</code>
<code>Arrays.equals(a, b)</code>	Do two arrays hold the same values in the same order?

`Arrays.sort` changes the original array. If you still need the original order, sort a copy. See [examples/Ex05ArraysClass.java](#).

Arrays are references: the aliasing gotcha

Run `examples/Ex06ReferencesAndAliasing.java`:

```
int[] original = {1, 2, 3};
int[] alias = original;
alias[0] = 99;
System.out.println(Arrays.toString(original)); // [99, 2, 3] !!
```

Changing `alias` changed `original`. Why? An `int` variable holds a number directly. But an array variable holds a **reference**: the address of where the array lives in memory. `alias = original` copies the address, so both names point at the **same** array:

```
original  ┌───┐
           │   │
           └───┴───> [ 99 | 2 | 3 ]
alias ───┬───┐
          │   │
```

This is the same reason `==` on two Strings compares addresses (Module 4), and why `==` on two arrays with the same values is `false`. To get an independent copy, use `Arrays.copyOf`. To compare values, use `Arrays.equals`. You'll meet references again with every object you create in Part 2.

Parallel arrays

Values at the same index in two arrays can belong together:

```
String[] students = {"Akua", "Kwesi", "Nana"};
int[] marks = {78, 64, 91}; // Akua got 78, Kwesi 64, Nana 91
```

This works, and `examples/Ex07ParallelArrays.java` uses it to print a report. But it's fragile: sort one array and the names and marks no longer line up. Module 10 shows the proper solution: a class that keeps a student's name and mark together.

Splitting text into arrays

`split` cuts a String wherever a separator appears, and gives back an array of the pieces. It's how you'll read data files in Module 14:

```
String[] fields = "Kofi,19,Kumasi,1.75".split(",");
String name = fields[0]; // "Kofi"
int age = Integer.parseInt(fields[1]); // 19

String[] words = "the quick brown fox".split(" ");
String.join(" - ", words) // "the - quick - brown - fox"
```

`split(" ")` gives empty strings when there are several spaces in a row. `split(" +")` treats any run of spaces as one separator. (The argument is a **regular expression**, a mini-language for patterns: `+` means "one or more".) See `examples/Ex08SplitAndJoin.java`.

Two-dimensional arrays: grids

An array can hold other arrays. A **2D array** is an array of rows, and each row is an array of values:

```
char[][] seats = {
    {'X', '.', '.', 'X', 'X'},
    {'.', '.', '.', '.', '.'},
    {'X', 'X', '.', 'X', '.'},
};

seats[0][3]          // row 0, column 3: 'X'
seats[1][2] = 'X';    // book row 1, seat 2
seats.length         // 3: the number of rows
seats[0].length       // 5: the number of columns in row 0
```

Always **row first, then column**: `grid[row][col]`. To visit every cell, use nested loops, rows on the outside:

```
for (int row = 0; row < seats.length; row++) {
    for (int col = 0; col < seats[row].length; col++) {
        System.out.print(seats[row][col] + " ");
    }
    System.out.println();
}
```

`new int[4][4]` creates an empty 4 × 4 grid of zeros. See [examples/Ex09Grids2D.java](#), then re-read the hook: `grid[row][col]` picks a number, `palette[...]` turns the number into a colour, and nested loops draw one square per cell.

[examples/Ex10CardDealer.java](#) puts the module together: it builds a 52-card deck with nested loops, shuffles it, and deals two hands.

String[] args, explained at last

Since Module 1, every `main` method has said `String[] args`. Now you can read it: `args` is an **array of Strings**. It holds any words you type after the program's name when you run it:

```
java Ex11CommandLineArgs.java Ama 3
```

Here `args[0]` is `"Ama"` and `args[1]` is `"3"`. These are called **command-line arguments**, and they let you give a program its input without any questions. See [examples/Ex11CommandLineArgs.java](#).

Common beginner mistakes

- **Going one past the end**: `for (int i = 0; i <= a.length; i++)` crashes on the last iteration. It's `<`, not `<=`.
- **length versus length()**: arrays use `a.length`; Strings use `s.length()`.
- **Printing an array with `println(a)` and getting `[I@...`**. Use `Arrays.toString(a)`.
- **Thinking `b = a` copies an array**. It copies the reference. Use `Arrays.copyOf`.
- **Comparing arrays with `==`**. Use `Arrays.equals`.
- **Using a slot of a new `String[n]` before filling it in**, causing a `NullPointerException`.
- **Mixing up rows and columns in a 2D array**. It's always `grid[row][col]`, like reading a book: down to the line, then along.

- **Expecting an array to grow.** Its size is fixed. Make a bigger copy, or use an `ArrayList` (next module).

Try it yourself

1. Run every file in `examples/`. In the hook, design your own picture (a heart, a robot, your initials) in the grid.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: two-player Tic-Tac-Toe.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 6: arrays"  
git push
```

Module 7: ArrayList, HashMap & HashSet

Hook: what does this text say most?

Run `examples/Ex00WordFrequency.java`:

```
java Ex00WordFrequency.java
```

```
80 words, 43 different ones.
```

```
Top 8:
the      18 *****
robot    4 ****
it       4 ****
lab      3 ***
found    3 ***
...
```

```
Used only once: [a, arm, asleep, back, bench, broken, but, by, corner, ...]
```

In a fraction of a second, Java counted every word in a paragraph, ranked them, and listed every word used only once, in alphabetical order. Now paste something of your own into the `TEXT` at the top of the file: a speech, an essay, the lyrics of a song you like. Which word wins?

Doing this with arrays from Module 6 would be painful. You don't know in advance how many different words there will be, and to count a word you'd have to search a whole array for it every time. This module gives you three new tools that make it easy: a list that **grows** (`ArrayList`), a table that **looks things up by name** (`HashMap`), and a collection that **never holds duplicates** (`HashSet`). Together they're called **collections**, and real Java programs use them far more than plain arrays.

ArrayList: a list that grows

```
import java.util.ArrayList;

ArrayList<String> queue = new ArrayList<>();    // an empty list of Strings
queue.add("Ama");                               // add to the end
queue.add("Kojo");
queue.add(0, "Yaw");                             // insert at index 0
System.out.println(queue);                       // [Yaw, Ama, Kojo]
```

The type in angle brackets, `<String>`, says what the list holds. It's called a **type parameter**, and it means Java will stop you from putting anything else in the list. The `<>` on the right (the "diamond") tells Java to fill in the same type again. Unlike arrays, lists **print nicely** with plain `println`.

Method	Does	Array equivalent
<code>list.add(x)</code>	Add <code>x</code> to the end	(impossible: fixed size)
<code>list.add(i, x)</code>	Insert <code>x</code> at index <code>i</code> , moving the rest along	
<code>list.get(i)</code>	The value at index <code>i</code>	<code>a[i]</code>

Method	Does	Array equivalent
<code>list.set(i, x)</code>	Replace the value at index <code>i</code>	<code>a[i] = x</code>
<code>list.size()</code>	How many values	<code>a.length</code>
<code>list.remove(i)</code>	Remove the value at index <code>i</code> (and give it back)	
<code>list.remove(x)</code>	Remove the first <code>x</code> (gives <code>true</code> if it was there)	
<code>list.contains(x)</code>	Is <code>x</code> in the list?	(a search loop)
<code>list.indexOf(x)</code>	Where is <code>x</code> ? (<code>-1</code> if it isn't)	(a search loop)
<code>list.isEmpty()</code> / <code>list.clear()</code>	Is it empty? / Remove everything	

Indexes start at 0, just like arrays, and a bad index crashes with an `IndexOutOfBoundsException`. See [examples/Ex01ArrayListBasics.java](#).

Lists of numbers: wrapper classes

Collections can only hold **objects**, not primitives. So `ArrayList<int>` doesn't compile. Instead, each primitive type has a matching **wrapper class**:

Primitive	Wrapper
<code>int</code>	<code>Integer</code>
<code>double</code>	<code>Double</code>
<code>boolean</code>	<code>Boolean</code>
<code>char</code>	<code>Character</code>

```
ArrayList<Integer> marks = new ArrayList<>();
marks.add(72);           // Java wraps the int in an Integer: "autoboxing"
int first = marks.get(0); // ...and unwraps it again: "unboxing"
```

This mostly happens without you noticing. But it causes two famous traps:

1. **`remove(int)` removes by index.** On an `ArrayList<Integer>`, `numbers.remove(1)` removes whatever is at **index 1**, not the number 1. To remove the value, write `numbers.remove(Integer.valueOf(1))`.
2. **Compare `Integers` with `.equals()`, not `==`.** They're objects, so `==` compares references, just like Strings. It *seems* to work for small numbers (Java reuses objects from `-128` to `127`), and then fails for bigger ones.

See [examples/Ex02WrapperClasses.java](#).

Looping, and removing safely

Loop over a list exactly as you would an array: `for-each` when you just need the values, an index loop (with `size()` and `get(i)`) when you need positions.

But **never add or remove items inside a `for-each` loop over the same list**. Java detects it and crashes with a `ConcurrentModificationException`. To remove matching items, either loop **backwards** by

index (so removing an item doesn't shift the ones you haven't visited yet), or use an `Iterator`, which has its own safe `remove()`:

```
for (int i = tasks.size() - 1; i >= 0; i--) {
    if (tasks.get(i).startsWith("buy")) {
        tasks.remove(i);
    }
}
```

(Module 15 shows a one-line way: `tasks.removeIf(...)`.) See [examples/Ex03LoopingAndRemoving.java](#).

The Collections class

`java.util.Collections` does for lists what `Arrays` does for arrays: `Collections.sort(list)`, `reverse`, `shuffle`, `max`, `min` and `frequency(list, x)` (how many times `x` appears).

`List.of(1, 2, 3)` creates a list in one line, but it's **fixed**: trying to add to it crashes. To get a list you can change, copy it: `new ArrayList<>(List.of(1, 2, 3))`. See [examples/Ex04CollectionsClass.java](#).

HashMap: look things up by key

A list finds things by **position**. But often you want to find things by **name**: the capital of Kenya, the price of an LED, the number of votes for Akua. A **map** stores **key** → **value** pairs:

```
import java.util.HashMap;

HashMap<String, String> capitals = new HashMap<>(); // keys: String, values:
String
capitals.put("Ghana", "Accra");
capitals.put("Kenya", "Nairobi");

capitals.get("Kenya")           // "Nairobi"
capitals.get("Mali")           // null: no such key
capitals.getOrDefault("Mali", "?") // "?"
```

Think of a real dictionary: you look up a **word** (the key) to find its **meaning** (the value). Each **key appears only once**: `put` with an existing key **replaces** its value. Values can repeat.

Method	Does
<code>map.put(key, value)</code>	Add a pair, or replace the value for an existing key
<code>map.get(key)</code>	The value for <code>key</code> , or <code>null</code> if it isn't there
<code>map.getOrDefault(key, d)</code>	The value for <code>key</code> , or <code>d</code> if it isn't there
<code>map.containsKey(key)</code> / <code>map.containsValue(v)</code>	Is that key / value there?
<code>map.remove(key)</code>	Remove a pair (gives back the old value, or <code>null</code>)
<code>map.size()</code> , <code>map.isEmpty()</code>	How many pairs / are there none?

Finding a key in a `HashMap` is **fast**, even with millions of entries: it doesn't search one by one. (The "hash" in the name is the trick it uses: Module 11 gives you a peek at how it works.) See [examples/Ex05HashMapBasics.java](#).

Looping over a map

```
for (String name : ages.keySet()) { ... }           // every key
for (int age : ages.values()) { ... }              // every value
for (Map.Entry<String, Integer> e : ages.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue()); // both together
}
```

A `HashMap` keeps **no particular order**: entries come out in an order that looks random, and can change as the map grows. If order matters, choose a different map. They all work the same way:

Map	Order
<code>HashMap</code>	None (the fastest)
<code>TreeMap</code>	Keys in sorted order (alphabetical, or smallest number first)
<code>LinkedHashMap</code>	Keys in the order you added them

You'll often see a map's variable declared with the general type `Map`, as in `Map<String, Integer> ages = new HashMap<>()`. That says "I need a map; this one happens to be a `HashMap`." Module 12 explains why that's good style. See [examples/Ex06LoopingOverMaps.java](#).

The counting pattern

The single most useful thing to do with a map is **count** things:

```
Map<String, Integer> tally = new TreeMap<>();
for (String vote : votes) {
    tally.put(vote, tally.getOrDefault(vote, 0) + 1);
}
```

Read it as: "look up this vote's count (or 0 if we haven't seen it), add 1, and put it back." That one line is the heart of the hook. You'll also see `tally.merge(vote, 1, Integer::sum)`, which does the same job: "put 1, or *add* 1 to what's already there." (`Integer::sum` is a ready-made "add two numbers" method, passed in by name. Module 15 explains this `::` syntax.) See [examples/Ex07Counting.java](#).

Collections inside collections

A map's values can be lists, and a list can hold maps:

```
Map<String, List<Integer>> marks = new TreeMap<>();
marks.put("Akua", new ArrayList<>(List.of(70, 82, 91)));
marks.get("Akua").add(65);           // add a mark to Akua's list
```

Before adding to a list inside a map, make sure the list exists: if `marks.get("Esi")` is `null`, calling `.add` on it crashes with a `NullPointerException`. See [examples/Ex08NestedCollections.java](#).

HashSet: no duplicates allowed

A **set** holds values with **no duplicates** and no positions. Adding something that's already there does nothing (and `add` gives back `false` to tell you):

```
Set<String> visited = new HashSet<>();
visited.add("Accra");           // true
visited.add("Accra");           // false: already there
visited.contains("Accra")       // true, and fast, like a map's containsKey
```

Sets are perfect for "have I seen this before?" (voter IDs, visited rooms, used usernames) and for removing duplicates in one line: `new TreeSet<>(list)`. As with maps, `HashSet` has no order, `TreeSet` keeps values sorted, and `LinkedHashSet` keeps them in the order added.

Sets also do the set operations from maths:

Operation	Method	Gives
Intersection	<code>a.retainAll(b)</code>	Keeps only what's in both
Union	<code>a.addAll(b)</code>	Everything in either
Difference	<code>a.removeAll(b)</code>	What's in a but not b

These **change a**, so make a copy first if you still need it. See `examples/Ex09HashSet.java`.

Which collection should I use?

You need...	Use
A fixed number of values, or a grid	an array
An ordered list that grows and shrinks	ArrayList
To look values up by a key	HashMap (TreeMap for sorted keys)
To know whether you've seen something, or to remove duplicates	HashSet (TreeSet for sorted)

`examples/Ex10PhoneBook.java` puts the module together in a menu-driven phone book. Now re-read the hook. You can explain all of it except two lines. `split("[^a-z']+")` splits the text at anything that isn't a letter or an apostrophe (another regular expression). And `entries.sort((a, b) -> b.getValue() - a.getValue())` sorts the entries by count, biggest first, using a **lambda**: a tiny function written inline. You'll write your own in Module 12.

Common beginner mistakes

- **ArrayList<int>**. Collections need wrapper classes: **ArrayList<Integer>**.
- **remove(1)** on a list of **Integers** removes index 1, not the value 1.
- Comparing **Integers** (or any objects) with **==**. Use **.equals()**.
- Removing from a list inside a for-each loop over it: **ConcurrentModificationException**. Loop backwards, or use an **Iterator**.
- Forgetting that **get** can return **null**. Use **getOrDefault**, or check **containsKey** first.

- Expecting a **HashMap** or **HashSet** to keep things in order. Use **TreeMap/TreeSet** (sorted) or **LinkedHashMap/LinkedHashSet** (insertion order).
- Thinking **put** adds a second entry for the same key. It replaces the first.
- Forgetting the imports. Everything in this module lives in **java.util**. **import java.util.*;** imports all of it at once.

Try it yourself

1. Run every file in **examples/**. Put your own text into the hook.
2. Complete **exercises/Exercise1.java** and **exercises/Exercise2.java**.
3. Build the module project: a vocabulary quiz.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 7: ArrayList, HashMap and HashSet"  
git push
```

Module 8: Methods

Hook: one method, a whole village

Run `examples/Ex00Village.java`:

```
java Ex00Village.java
```

A window opens showing a village: nine houses of different sizes and colours, a row of trees, grass and a sun. Now look at the code. The instructions for drawing a house (walls, roof, door, window) are written **once**, in a block called `drawHouse`:

```
static void drawHouse(Graphics2D pen, int x, int y, int size, Color wall) {
    pen.setColor(wall);
    pen.fillRect(x, y - size, size, size);    // walls
    ...
}
```

Then the village is drawn by **calling** it nine times, with a different position, size and colour each time:

```
drawHouse(pen, 20 + i * 85, 330 + random.nextInt(60), size, walls[i %
walls.length]);
```

Change the loop to draw 20 houses. Change the roof colour inside `drawHouse`, and every house changes at once. That's the power of a **method**: a named, reusable piece of code that you write once and use as often as you like. You've been *using* methods since Module 1 (`println`, `nextLine`, `Math.sqrt`). This module is about *writing your own*, which is how every large program is built.

Why methods?

Look at `examples/Ex01WhyMethods.java`. It prints the same kind of banner several times. Without a method, that means copying and pasting three lines for each banner. With a method, you write those lines once and give them a name:

```
static void printBanner(String title) {
    System.out.println("+" + "-".repeat(title.length() + 4) + "+");
    System.out.println("| " + title + " |");
    System.out.println("+" + "-".repeat(title.length() + 4) + "+");
}
```

Then `printBanner("Results");` prints a banner. Methods give you:

- **No repetition.** Fix a bug once, and it's fixed everywhere.
- **Names.** `printBanner("Report")` says what it does. Three lines of `println` and `repeat` make the reader work it out.
- **Smaller pieces.** A 300-line `main` is impossible to hold in your head. Twenty 15-line methods, each doing one job, are not.
- **Testing.** You can check a small method on its own, with chosen inputs, before you trust it inside a bigger program.

Defining and calling a method

```
public class Ex02DefiningAndCalling {

    static void sayHello() {
        System.out.println("Hello!");
    }

    public static void main(String[] args) {
        sayHello();           // call it
        sayHello();           // and again
    }
}
```

- A method is defined **inside the class**, but **outside** every other method. Its order in the class doesn't matter: `main` can call a method written below it.
- `static` means the method belongs to the class itself. (Every method in Part 1 is `static`. Module 10 shows what non-static methods are for.)
- `void` means the method doesn't give back a value. It just does something.
- `sayHello` is its name: camelCase, usually a **verb**, saying what it does.
- The brackets `()` hold its **parameters** (none here), and the braces hold its **body**.

Defining a method does nothing by itself. Its code only runs when the method is **called**. When Java reaches a call, it jumps into the method, runs the whole body, then **comes back** to exactly where it left off. Run `examples/Ex02DefiningAndCalling.java` and follow its output: `main` calls `countToThree`, which calls `sayHello`, and each one returns to its caller in turn. Java keeps track of who called whom in the **call stack**. That's what the `at` lines in a stack trace are: the chain of calls that led to the crash, most recent first.

Parameters: handing values to a method

```
static void greet(String name) {
    System.out.println("Akwaaba, " + name + "!");
}

greet("Afua");           // Akwaaba, Afua!
greet("Kwabena");        // Akwaaba, Kwabena!
```

`name` is a **parameter**: a variable that belongs to the method, filled in each time the method is called.

`"Afua"` is the **argument**: the actual value given in the call. A method can have several parameters, of any types, separated by commas, and each needs its own type:

```
static void printReceiptLine(String item, int quantity, double price) { ... }

printReceiptLine("Breadboard", 2, 25.0);
```

The arguments must match the parameters in **number, order and type**. `printReceiptLine(2, "Breadboard", 25.0)` doesn't compile. An argument can be anything that gives the right type: a literal, a variable, a calculation, even another method call. See `examples/Ex03Parameters.java`.

Return values: getting an answer back

Many methods calculate an answer. `return` sends it back to the caller:

```
static double celsiusToFahrenheit(double celsius) {  
    return celsius * 9 / 5 + 32;  
}  
  
double f = celsiusToFahrenheit(37);    // f is 98.6
```

The type before the method's name (`double` here, `void` before) is its **return type**: the type of value it gives back. Think of the call as being **replaced by its result**, so you can use it anywhere a value of that type fits: in a variable, in a calculation, in a `println`, or in an `if`:

```
static boolean isPrime(int n) {  
    if (n < 2) {  
        return false;  
    }  
    for (int d = 2; d * d <= n; d++) {  
        if (n % d == 0) {  
            return false;  
        }  
    }  
    return true;  
}  
  
if (isPrime(97)) { ... }
```

`return` **ends the method immediately**, even in the middle of a loop. That's why `isPrime` can stop as soon as it finds a divisor. A method with a return type must return a value on **every** path through it. Miss one and the compiler says `missing return statement`.

Methods that answer yes/no questions return `boolean`, and read best with names like `isPrime`, `hasPaid` or `canBorrow`. See `examples/Ex04ReturnValues.java`.

Print or return? A method that *works something out* should usually **return** it, and leave printing to whoever called it. That way the caller decides what to do with the answer: print it, store it, compare it, or pass it to another method. `celsiusToFahrenheit` that prints instead of returning is useless for a temperature table that needs `printf` columns.

Overloading: same name, different parameters

Two methods can share a name, as long as their parameter lists differ:

```
static double area(double radius) { return Math.PI * radius * radius; }  
static double area(double width, double height) { return width * height; }  
  
area(2)        // calls the first one  
area(3, 4)     // calls the second one
```

Java chooses which one to run from the arguments. You've relied on this all along: `println` has separate versions for `String`, `int`, `double`, `char` and `boolean`. See `examples/Ex050overloading.java`.

Scope: where variables live

A variable exists only inside the braces where it was created. That's its **scope**:

```
static double withVat(double price) {  
    double vat = price * VAT_RATE;    // vat only exists inside withVat  
    return price + vat;  
}
```

- Variables created inside a method (including its parameters) are **local** to that method. Other methods can't see them, and they disappear when the method returns.
- Two methods can each have a local variable with the same name. They're completely separate boxes.
- A **constant** declared in the class, outside every method, as `static final double VAT_RATE = 0.15;`, is visible to every method in the class. That's the right place for values the whole program shares.

See `examples/Ex06Scope.java`.

Java passes copies

When you call a method, each parameter gets a **copy** of the argument's value. So a method can't change the caller's variables:

```
static void tryToChange(int number) {  
    number = 999;    // changes the local copy only  
}  
  
int number = 5;  
tryToChange(number);  
System.out.println(number);    // still 5
```

But remember Module 6: an array variable holds a **reference**. The method gets a copy of the *reference*, which points at the **same array**. So a method *can* change the contents of an array (or list) you pass it:

```
static void doubleAll(int[] values) {  
    for (int i = 0; i < values.length; i++) {  
        values[i] *= 2;    // changes the caller's array  
    }  
}
```

Assigning a whole new array to the parameter, though, only changes the local copy of the reference. See `examples/Ex07PassByValue.java`. If a method needs to give back new data, the clearest way is to **return** it, as in `examples/Ex08ArraysAndListsInMethods.java`.

Documenting and checking methods

A **Javadoc** comment, `/** ... */`, written just above a method, describes what it does, its parameters (`@param`) and its result (`@return`):

```

/**
 * Works out the body mass index (BMI) from weight and height.
 *
 * @param weightKg the person's weight in kilograms
 * @param heightM the person's height in metres
 * @return the BMI, which is weight divided by height squared
 */
static double bmi(double weightKg, double heightM) {
    return weightKg / (heightM * heightM);
}

```

Hover over a call to `bmi` in VS Code and this comment pops up as help. It's also how the official Java documentation at docs.oracle.com is generated.

Because a method takes inputs and returns a result, you can **check** it on its own.

`examples/Ex09JavadocAndChecks.java` has a small `check` method that prints PASS or FAIL for chosen inputs, including the **boundaries** (is a BMI of exactly 18.5 "underweight" or "healthy"?), which is where bugs love to hide. Module 17 turns this idea into professional automated testing with JUnit.

Designing with methods

When a program has a clear structure, `main` reads like a **table of contents**:

```

public static void main(String[] args) {
    printWelcome();
    int count = askInt("How many temperatures? ", 1, 10);
    double[] temps = readTemperatures(count);
    printReport(temps);
}

```

Each method does **one** job, its name says what that job is, and it's short enough to see all at once. A good rule of thumb: if you can't describe a method without using the word "and", it should probably be two methods. See `examples/Ex10TableOfContents.java`.

Now re-read the hook. `drawHouse` and `drawTree` take the drawing pen and the position as parameters, and `draw` calls them in loops. The `/** ... */` above each one is a Javadoc comment. You can explain every line.

Common beginner mistakes

- **Defining a method inside another method** (such as inside `main`). Methods go side by side inside the class.
- **Forgetting the brackets in a call**: `sayHello;` doesn't compile. It's `sayHello();`.
- **Forgetting to use the return value**. `celsiusToFahrenheit(20);` on its own line calculates the answer and throws it away.
- **missing return statement**: some path through a non-`void` method doesn't return. Check every `if` branch.
- **Printing inside a method that should return**. Return the answer and let the caller print it.
- **Expecting a method to change an `int` you passed it**. It gets a copy. Return the new value instead: `score = addBonus(score);`.

- **Arguments in the wrong order** when two parameters have the same type, like `(width, height)`. The compiler can't catch this one. Good parameter names and Javadoc help.
- **Giant methods.** If a method doesn't fit on the screen, split it.

Try it yourself

1. Run every file in `examples/`. In the hook, write a `drawCar` or `drawCloud` method and add some to the village.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project: Tic-Tac-Toe, restructured into methods, with a computer opponent.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 8: methods"  
git push
```

Module 9: Capstone 1, Text Adventure

Everything from Part 1 comes together here: variables, input, strings, decisions, loops, arrays, lists, maps, sets, and methods, combined into one real program. You'll build a **text adventure game**, the kind of game people played before graphics: the player explores a world by typing commands, picks up items, solves puzzles, and tries to reach an ending.

```
=== THE LOST ROBOT OF MAKERSPACE ===

WORKSHOP
Soldering irons cool on the benches. A half-built robot slumps in the
corner, missing its battery. Doors lead north and east.
You see: screwdriver

> take screwdriver
You pick up the screwdriver.

> go north
STORE ROOM
Shelves of components stretch into the dark. The door to the south is
the way back.
You see: battery, torch

> help
Commands: go <direction>, take <item>, drop <item>, use <item>,
look, inventory, help, quit
```

It's built in a team, in stages, with every stage committed to GitHub, so by the end you'll have a real project with a real history in your portfolio.

1. How teams are formed

Your instructor announces the class size before this module starts, which sets your group size:

Class size	Group size	Ratio
Up to 20 students	Pairs	2:1
More than 20 students	Groups of 4	4:1
Approved solo ("daring") students, any class size	Individual	1:1

Going solo ("daring" track): any student may ask to build the capstone alone instead of in a group. It isn't a smaller workload. It's the same core requirements (§4) **plus every stretch goal** in §5, since you don't have teammates sharing the work. Ask your instructor *before* Stage 0 (§6). Going solo is a good fit if you're comfortable with everything through Module 8 and want the extra challenge. It isn't a way to avoid working with others.

2. Setting up your team's repository

1. One team member creates a new **private** GitHub repository named `text-adventure-<teamname>` (e.g. `text-adventure-robo-rangers`), with a README and the Java `.gitignore`.
2. They go to **Settings** → **Collaborators** and add every teammate *and* the instructor.
3. Every teammate clones the **same** repository:

```
git clone https://github.com/OWNER-USERNAME/text-adventure-teamname.git
cd text-adventure-teamname
```

4. Copy `Adventure.java` into the repository, run it (`java Adventure.java`), commit it, and push. That's your Stage 0 starting point.
5. Everyone commits under their **own** Git identity (`git config user.name / user.email` from Module 0). This is how your instructor sees who contributed what. A history where one teammate made every commit is a problem to fix early, not at the deadline.

Working together without overwriting each other

When two people edit the same file, Git has to combine their changes. Keep it painless:

- Always **git pull** before you start working, and before you push.
- **Split the work by method.** One person writes `takeItem` while another writes `showInventory`. Methods are what make teamwork possible. Agree on each method's name, parameters and return type first (write its Javadoc comment together), then each person fills in their own.
- **Commit small and often**, with clear messages: "`Add take command`" beats "`stuff`".
- If **git pull** reports a **merge conflict**, Git marks the clashing lines in the file with `<<<<<<`, `=====` and `>>>>>>`. Open the file, decide what the code should be (often you keep both changes), delete the markers, make sure it still compiles, then **git add**, **git commit**, and **git push**. It's normal. Ask your instructor the first time it happens.

3. How the game is built: data + methods

The trick to a text adventure is that **the world is data**. Every room has a short **key** (like "`workshop`"), and maps look up everything about a room by its key:

```
static final Map<String, String> ROOM_NAMES = new HashMap<>();
static final Map<String, String> DESCRIPTIONS = new HashMap<>();
static final Map<String, Map<String, String>> EXITS = new HashMap<>(); // room ->
(direction -> room)
static final Map<String, List<String>> ITEMS = new HashMap<>(); // room ->
items lying there
```

The starter's `buildWorld` method fills these in, using three small helper methods:

```
addRoom("workshop", "Workshop", "Soldering irons cool on the benches...");
addRoom("store room", "Store Room", "Shelves of components stretch into the
dark...");
addExit("workshop", "north", "store room");
addExit("store room", "south", "workshop");
addItem("store room", "battery");
```

The player's state is a few simple variables in `main`: the key of the `currentRoom` (a `String`), an `inventory` (a `List<String>`), and perhaps a `Set<String>` of rooms they've visited. Moving north is just looking up the exit:

```
currentRoom = EXITS.get(currentRoom).get("north");
```

The **game loop** reads a command, splits it into a verb and a noun ("`take battery`" → "`take`" and "`battery`"), and calls the right method. Add a room or an item by adding *data*, not by writing new `if` statements. `Adventure.java` sets up exactly this structure, with two rooms and working `go`, `look`, `help` and `quit` commands.

Keeping one room's details in four separate maps works, but it's a little clumsy, a lot like the parallel arrays of Module 6. In Module 10 you'll see how a `Room` class keeps everything about a room together. For now, the maps are exactly right.

4. Requirements for every team

Your finished game must have:

The world

- **At least 6 rooms**, each with a name, a description, exits and items, all created in `buildWorld`.
- **At least 5 items** the player can pick up.
- **At least one locked or blocked route** that needs a specific item (a key opens a door, a torch lights a dark room, a battery powers a lift...).
- **A win condition** (reach a room carrying certain items, fix the robot, escape the building...) and **at least one way to lose**, or a move or time limit.

The commands

- `go <direction>` (and short forms such as `n`, `s`, `e`, `w`).
- `take <item>` and `drop <item>`.
- `inventory` (or `i`), `look`, `help`, and `quit`.
- At least **one command of your own design** (`use`, `read`, `talk`, `open`...).
- Unknown commands, impossible moves ("You can't go west from here."), and missing items get a helpful message. **No crashes, ever**. Not on empty input, extra spaces, capital letters, or a number where a word was expected.

The code

- **At least 8 methods you wrote yourselves**, each with a Javadoc comment and one clear job, such as `describeRoom`, `move`, `takeItem`, `dropItem`, `showInventory`, `showHelp`, `parseCommand`, and `hasWon`.
- A `main` method containing the game loop, and **no static variables except constants and the world data**. Keep the player's state (current room, inventory) in `main`, pass it into methods, and return results.
- Use of a **list** (the inventory), a **map** (rooms and exits), and a **set** (e.g. visited rooms, or the items needed to win).

- Clean, consistent style: `camelCase` names, `UPPER_SNAKE_CASE` constants, comments where the why isn't obvious, consistent indentation, and no copy-pasted blocks that should be methods.

The project

- A `README.md` in your repository explaining what the game is, how to run it (`java Adventure.java`), how to win (spoilers below a warning line are fine), and **who built which parts**.
- A **map** of your world: a drawing (photo or image file) or an ASCII diagram in the README.

5. Stretch goals

Pick as many as you like. **Solo/daring students must complete all of them.**

1. **Scoring:** points for visiting rooms and finding items, shown with a `score` command and at the end.
2. **A non-player character** who says different things depending on what you're carrying, or gives you an item in exchange for another.
3. **A move limit** with a countdown ("The building's power fails in 12 moves...") that creates tension and a losing ending.
4. **Smarter parsing:** ignore filler words ("`go to the north`", "`take the battery`") and accept synonyms (`get` = `take`, `walk` = `go`). A `Set<String>` of filler words and a `Map<String, String>` of synonyms make this neat.
5. **A `map` command** that prints which rooms the player has visited and the exits from the current room.
6. **Random events:** a power cut, a wandering cat, or a random item location each game, using `Random`.

6. Stages and timeline

Stage	Focus	Git evidence expected
Stage 0	Team formed, repository set up, world designed on paper first (rooms, exits, items, puzzle, ending)	First commit: <code>Adventure.java</code> , plus your map and plan in the README
Stage 1	All rooms in <code>buildWorld</code> ; <code>go</code> , <code>look</code> , <code>help</code> , <code>quit</code> working	Several small commits; every room reachable and described
Stage 2	Items: <code>take</code> , <code>drop</code> , <code>inventory</code> ; items appear in room descriptions	Commits adding each command as its own method
Stage 3	The locked route, your own command, and the win/lose conditions	A commit where the game can be won from start to finish
Stage 4	Playtesting and polish: try to break your game (empty input, nonsense, capitals), fix what you find, stretch goals	Commits named after the specific bugs you fixed
Final	README finished, every method has a Javadoc comment, last push before the deadline	Your final commit <i>is</i> your submission

Commit at the end of every working session, even if it isn't finished, as long as it **compiles**. Never push code that doesn't compile: your teammates will pull it and their game will stop working too. A history with one giant commit the night before the deadline is a red flag, not a shortcut.

Playtest each other's games. Before the final stage, swap with another team. Play their game without reading its code, and write down everything that confused you or broke. Then fix what they found in yours.

7. How it's assessed

- **It works.** It compiles, it can be won, and it handles bad input without crashing. Every requirement in §4 is met.
- **It uses Part 1 well.** Maps, lists, sets, loops, `switch` and methods used because they fit the problem, not bolted on to tick a box. Be ready to explain any line of your code.
- **It's well organised.** Short methods with clear names and Javadoc comments, no repeated blocks, no unnecessary `static` variables.
- **It's fun and well written.** Descriptions that create a sense of place, a puzzle that makes sense, an ending worth reaching.
- **Git history.** Regular, meaningful commits from **every** team member, under their own name.
- **Documentation.** A README a stranger could use to run and play your game without asking you anything.

Try it yourself

1. Confirm your team (or solo status) with your instructor.
2. Create and share the team repository, and push `Adventure.java`.
3. Design your world on paper: map, items, puzzle, ending.
4. Work through the stages in §6, committing and pushing regularly.
5. Submit by pushing your final commit before the deadline. There is no separate submission step.

You've built a real game from nothing but text and logic. Put the repository link on your CV, and get ready for Part 2, starting with **Module 10: Classes & Objects**, where you'll learn to design your own types, starting with a `Room` class that would have made this game a lot tidier.

Object-Oriented Java

From "I can write Java" to "I can build things in Java": classes and objects, inheritance and polymorphism, interfaces, enums and records, exceptions and debugging, files, generics and streams, recursion and algorithms, packages, Maven and JUnit, graphical user interfaces with JavaFX, and a second capstone. Module 14b, on databases with SQLite and JDBC, is optional.

Module 10: Classes & Objects

Hook: adopt a virtual pet (or two)

Run `examples/Ex00VirtualPet.java`:

```
java Ex00VirtualPet.java
```

```
Name your first pet: Kwaku
Name your second pet: Adjoa

Kwaku   hunger [###       ] happiness [#####  ] content
Adjoa   hunger [###       ] happiness [#####  ] content

Who do you want to look after? (1/2, or q to quit) 1
What will you do with Kwaku? (f)eed or (p)lay: p
Kwaku chases a ball around the yard!
```

Feed them, play with them, and ignore one of them for a while. Every turn, both pets get hungrier and a little less happy, but **each pet keeps track of its own hunger and happiness**. Play with Kwaku all day and Adjoa sulks. Feed Adjoa and Kwaku starves.

Here's how a pet is described in the code:

```
static class Pet {
    String name;
    int hunger = 3;
    int happiness = 7;

    Pet(String name) {
        this.name = name;
    }

    void feed() {
        hunger = Math.max(0, hunger - 3);
        System.out.println(name + " munches happily. Yum!");
    }
    ...
}
```

`class Pet` defines a brand-new **type**, just as `String` and `ArrayList` are types. It bundles a pet's **data** (name, hunger, happiness) with the **behaviour** that changes it (feed, play). Then `new Pet("Kwaku")` and `new Pet("Adjoa")` create two separate **objects** of that type, each with its own data. Java is an **object-oriented** language: designing classes like this is what Java programmers spend most of their time doing. By the end of this module, you'll design your own classes to model a library, a circuit, or a hospital ward.

You've been using objects all along

Every `String`, `Scanner`, `Random` and `ArrayList` you've used is an **object**, and you've been calling their **methods** since Module 3:

```
String name = "kware";
name.toUpperCase();           // a String object's method
Scanner in = new Scanner(System.in); // new creates a Scanner object...
in.nextLine();                // ...which has a nextLine method
ArrayList<Integer> scores = new ArrayList<>();
scores.add(91);               // THIS list's add method
```

The dot means "the thing on the left's own...". `scores.add(91)` changes `scores` and no other list. Each object carries its own **data** and its own **methods**. So far you've only used classes other people designed. Now you'll design your own.

Classes and objects

A **class** is a **blueprint**. It describes what data every object of that type has, and what it can do. An **object** (also called an **instance**) is one actual thing built from the blueprint:

```
class Dog {
    String name;           // FIELDS: the data every Dog has
    String breed;
    int age;
}

Dog rex = new Dog();      // new builds an object from the blueprint
rex.name = "Rex";         // the dot reaches into THIS object's fields
Dog bingo = new Dog();    // a second, completely separate object
bingo.name = "Bingo";
```

The variables declared inside the class, but outside any method, are its **fields** (also called instance variables). Every object gets its own copy of every field, starting at the default values from Module 6 (`0`, `false`, `null`) unless you give them a value.

Class names use **CapitalisedWords** (`Dog`, `BankAccount`, `VitalReading`), unlike the camelCase used for variables and methods. So you can always tell `Pet` (a class) from `pet` (an object). See [examples/Ex01FirstClass.java](#).

One class per file (and why the examples cheat)

In real Java programs, **each class goes in its own file**, named after the class: `Dog` lives in `Dog.java`, marked `public class Dog`. The lesson examples break this rule for convenience: each example file keeps its classes *inside* the main class, marked `static class`, so that `java Ex01FirstClass.java` still runs it as a single file. In your projects, use one file per class, as in `examples/multi-file/`, and compile and run them in two steps:

```
cd examples/multi-file
javac -d out *.java          # compile every .java file; put the .class files in
out/                          # out/
java -cp out BankApp         # run the class with main; -cp out says where to find
classes
```

The Java `.gitignore` you chose in Module 0 tells Git to ignore `.class` files, so compiled code never clutters your repository. In VS Code, the  button on `BankApp.java` does all of this for you.

Constructors: setting up a new object

Setting every field by hand after `new` is tedious and easy to forget. A **constructor** is a special method that runs automatically when an object is created:

```
class Dog {
    String name;
    String breed;
    int age;

    Dog(String name, String breed, int age) {
        this.name = name;        // this.name is the FIELD; name is the PARAMETER
        this.breed = breed;
        this.age = age;
    }
}

Dog rex = new Dog("Rex", "Labrador", 3);
```

- A constructor has **the same name as the class** and **no return type** (not even `void`).
- `this` means "the object being created (or used) right now". When a parameter has the same name as a field, `this.name` is the field and plain `name` is the parameter.
- Constructors can be **overloaded**, and one can call another with `this(...)`. In `examples/Ex02Constructors.java`, `new Dog("Patch", 1)` calls the three-argument constructor with `"mixed breed"`.

If you write no constructor at all, Java gives you an empty one for free. As soon as you write one, the free one disappears, so `new Dog()` stops compiling. That's usually what you want: it forces everyone to supply the data a Dog needs.

Instance methods: behaviour

The methods in Module 8 were all `static`. Methods *without* `static` are **instance methods**: they belong to each object, and work on that object's fields:

```
class Counter {
    String label;
    int count;

    Counter(String label) {
        this.label = label;
    }

    void click() {
        count++;                // THIS counter's count
    }

    boolean isOver(int limit) {
        return count > limit;
    }
}

Counter visitors = new Counter("Visitors");
Counter cars = new Counter("Cars");
visitors.click();              // changes visitors.count, not cars.count
```

Inside an instance method, a field's name on its own means "this object's field". You can write `this.count++` if you want to be explicit. An instance method is always called **on an object**: `visitors.click()`. That's the whole difference from `static`: a static method like `Math.sqrt` belongs to the class and doesn't need an object; an instance method needs one, because it works on that object's data. See `examples/Ex03Methods.java`.

toString: how an object prints

Print an object of your own class and you get something like `Ex04ToString$Plain@7bc1a03d`: the class name and a code for where it lives in memory. Add a `toString` method, and Java uses it **whenever the object is turned into text**: in `println`, when it's joined with `+`, and when a list of them is printed:

```
@Override
public String toString() {
    return "(" + x + ", " + y + ")";
}
```

It must be written exactly like this: `public`, returning a `String`, with no parameters. `@Override` tells Java "I'm replacing the standard version every object has". Module 11 explains where that standard version comes from. Give every class you write a `toString`: it makes debugging enormously easier. See `examples/Ex04ToString.java`.

Encapsulation: objects that protect themselves

With public fields, anyone can put nonsense into an object: `account.balance = -1_000_000`; **Encapsulation** means hiding an object's data and only allowing changes through methods that check them:

```
class BankAccount {
    private final String owner;
    private double balance;

    BankAccount(String owner, double openingBalance) {
        if (openingBalance < 0) {
            throw new IllegalArgumentException("Opening balance can't be
negative.");
        }
        this.owner = owner;
        this.balance = openingBalance;
    }

    double getBalance() {                // a "getter": read-only access
        return balance;
    }

    void withdraw(double amount) {
        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds.");
        }
        balance -= amount;
    }
}
```

- **private** means only code inside this class can use the field. `account.balance = 1_000_000`; elsewhere is a compile error: "balance has private access".

- **final** on a field means it's set once, in the constructor, and never changes. Use it for everything that shouldn't change, like an owner's name or an ID.
- **Getters** (`getBalance()`) give read access. Only add a **setter** (`setBalance(...)`) if outside code genuinely needs to change a value, and make it check the new value. Often, as here, a meaningful method like `withdraw` is better than a setter.
- **throw new IllegalArgumentException("message")** stops the method immediately and reports the problem. If nothing catches it, the program crashes with that message, which is far better than silently carrying on with bad data.

The rule of thumb: **make every field private**, and design methods that keep the object valid at all times. To turn a thrown exception into a friendly message instead of a crash, the caller wraps the call in `try/catch`:

```
try {
    account.withdraw(500);
} catch (IllegalArgumentException e) {
    System.out.println("Refused: " + e.getMessage());
}
```

That's all you need for now. Module 13 covers exceptions in depth. See [examples/Ex05Encapsulation.java](#).

static members: shared by the whole class

Sometimes data belongs to the class as a whole, not to any one object: a counter of how many tickets have been sold, or a constant price. Mark it **static**, and there's just **one copy**, shared by every object:

```
class Ticket {
    private static int nextNumber = 1;    // one copy for the whole class
    static final double PRICE = 15.0;

    private final int number;             // every ticket has its own

    Ticket(String holder) {
        this.number = nextNumber;
        nextNumber++;
    }

    static int ticketsSold() {
        return nextNumber - 1;
    }
}

Ticket.ticketsSold()    // called on the CLASS, not on a ticket
```

That's what `Math.PI`, `Math.sqrt` and `Integer.MAX_VALUE` are: static members of a class. A static method can't use instance fields directly, because it isn't running on any particular object. See [examples/Ex06StaticMembers.java](#).

Objects are references

Exactly like arrays in Module 6, a variable of a class type holds a **reference** to an object, not the object itself:

```
Lamp kitchen = new Lamp("kitchen");
Lamp sameLamp = kitchen;           // two names, ONE object
sameLamp.on = true;
System.out.println(kitchen.on); // true
```

- Passing an object to a method passes a copy of the reference, so the method can change the object.
- `==` asks "are these the **same object**?" Two different objects with identical data are not `==`. (Module 11 shows how to write an `equals` method that compares data.)
- `null` means "no object at all". Calling a method or reading a field through a `null` reference crashes with a `NullPointerException`. The message names the variable or expression that was `null`, which is your first clue. Methods often return `null` to mean "not found", so check before you use the result.

See [examples/Ex07ReferencesAndNull.java](#).

Objects working together

Real programs are made of objects that **contain** other objects. A `Course` has a list of `Students`; a `Library` has a map of `Books`. This is called **composition**:

```
class Course {
    private final String code;
    private final List<Student> students = new ArrayList<>();

    void enrol(Student s) { ... }
    Student find(String id) { ... } // returns null if there's no such student
    double average() { ... }
    Student top() { ... }
}
```

Each class has **one job**: a `Student` knows about itself; a `Course` manages a group of students. The code that uses them reads almost like English: `course.enrol(new Student("S01", "Akua", 78))`, `course.top()`. See [examples/Ex08ObjectsTogether.java](#).

A list of objects also fixes the problem with parallel arrays from Module 6. Each product's name, price and stock live together inside one `Product` object, so sorting the list can never separate them. See [examples/Ex09ObjectsInLists.java](#), and [examples/Ex10Room.java](#), which rebuilds the Module 9 text adventure with a `Room` class: each room's exits lead directly to other `Room` objects, and the four separate maps disappear.

Designing a class

When you design a class, answer three questions:

1. **What does it know?** Those are its fields. Make them `private`, and `final` where possible.
2. **What can it do?** Those are its methods. Name them after what they achieve (`lend`, `withdraw`, `needsAttention`), not how.
3. **What must always be true?** A balance is never negative; a mark is 0–100; a bed number exists on the ward. Those rules go in the constructor and the methods, so the object enforces them itself.

Common beginner mistakes

- **Forgetting `new`.** `Dog rex;` creates a variable, not a dog. `rex.name = "Rex";` then fails ("might not have been initialized").
- **Giving a constructor a return type.** `void Dog(String name)` is a *method* called `Dog`, not a constructor, so the object never gets set up.
- **Forgetting `this`.** when a parameter has the same name as a field. `name = name;` assigns the parameter to itself, and the field stays `null`.
- **Calling an instance method from `static main` without an object.** "non-static method cannot be referenced from a static context" means you need an object first: `pet.feed()`, not `feed()`.
- **Public fields.** Make them `private`, and provide only the methods that are really needed.
- **A file name that doesn't match its public class**, or two public classes in one file.
- **Comparing objects with `==`** when you mean "same data".
- **Using a `null` result**, such as from `find`, without checking it first.

Try it yourself

1. Run every file in `examples/`, and compile and run the `multi-file` example with `javac` and `java`. In the hook, give `Pet` a third ability (`sleep?`) and a new mood.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: a library, a circuit analyser, or a ward monitor.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 10: classes and objects"
git push
```

Module 11: Inheritance & Polymorphism

Hook: one line of code, twelve different drawings

Run `examples/Ex00ShapeParade.java`:

```
java Ex00ShapeParade.java
```

A window opens with a parade of twelve shapes: circles, squares, stars and rings, in random colours. Now look at the code that draws them all:

```
for (Shape shape : shapes) {  
    shape.draw(pen);  
}
```

That's it. One loop, one method call. There's no `if (it's a star)`, no `switch` on the kind of shape. Yet stars come out as stars and rings as rings, because **each kind of shape knows how to draw itself**:

```
static class Star extends Shape {  
    void draw(Graphics2D pen) { ... ten points around a centre ... }  
}  
  
static class Ring extends Circle {  
    void draw(Graphics2D pen) {  
        super.draw(pen);                // draw the circle first...  
        pen.setColor(Color.WHITE);  
        pen.fillOval(x - 20, y - 20, 40, 40); // ...then cut a hole in it  
    }  
}
```

Add a new kind of shape, a **Heart** or a **Hexagon**, and the drawing loop doesn't change at all. This is **polymorphism** ("many forms"), and it rests on **inheritance**: `Star extends Shape` means "a Star is a kind of Shape". This module explains both, and they're the two ideas that make large object-oriented programs manageable.

Why inheritance?

Look at the first half of `examples/Ex01WhyInheritance.java`. `DogBefore` and `CatBefore` both have a `name`, an `age`, a constructor and a `describe` method, all identical, copied and pasted. Fix a bug in one, and you must remember to fix it in the other.

Inheritance lets you write the shared parts **once**, in a **superclass**, and have other classes **extend** it:

```

class Animal {
    protected final String name;
    protected final int age;

    Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    String describe() {
        return name + ", aged " + age;
    }
}

class Dog extends Animal {
    Dog(String name, int age) {
        super(name, age);
    }
    String sound() { return "Woof"; }
    void fetch() { System.out.println(name + " fetches the stick!"); }
}

```

Dog is a **subclass** of **Animal** (also called a child class or derived class). It **inherits** every field and method of **Animal**, so `rex.describe()` works even though **Dog** never defines `describe`, and it **adds** its own: `sound` and `fetch`.

The test for whether inheritance fits is the phrase "**is a**": a Dog *is an* Animal; a Savings Account *is an* Account; a Professor *is an* Employee. If the phrase sounds wrong, inheritance is the wrong tool (see "Composition" below).

A Java class can extend only **one** superclass, but chains can be as long as you like: a **Trotro** is a **Car**, which is a **Vehicle**.

Constructors and `super(...)`

A subclass object is built in **layers**: the superclass part first, then the subclass part. So a subclass constructor must start by calling the superclass constructor with `super(...)`:

```

class Car extends Vehicle {
    private final int seats;

    Car(String registration, int seats) {
        super(registration, 4);    // MUST come first: build the Vehicle part
        this.seats = seats;        // then the Car part
    }
}

```

If you leave out `super(...)`, Java tries to call the superclass's constructor with no arguments, and if there isn't one, you get a compile error: "constructor Vehicle in class Vehicle cannot be applied to given types". `examples/Ex02ExtendsAndSuper.java` prints a message from each constructor so you can watch the layers being built, from **Vehicle** down to **Trotro**.

Overriding: changing inherited behaviour

A subclass can **override** an inherited method: provide its own version, with exactly the same name, parameters and return type:

```
class Manager extends Employee {
    private final double allowance;
    ...
    @Override
    double monthlyPay() {
        return super.monthlyPay() + allowance;    // the original, plus a bit
    }
}
```

- **@Override** asks the compiler to check that you really are overriding something. Misspell the method's name, or get a parameter type wrong, and you get an error instead of a silent new method. Always use it.
- **super.monthlyPay()** calls the superclass's version. Use it when the subclass wants to **extend** the behaviour rather than replace it completely, as **Ring.draw** does in the hook.
- You've already done this: every **toString** you've written overrides the one every class inherits (more on that below).

There's a subtle and important point in `examples/Ex030overriding.java`: `Employee.payslip()` calls `monthlyPay()`. For a **Manager** object, that call runs **Manager's monthlyPay**, even though the call is written inside **Employee's** code. Java always runs the version belonging to the object's **actual** class. That's the engine behind polymorphism.

Polymorphism

A variable of a **superclass** type can hold an object of **any subclass**:

```
Animal pet = new Dog("Rex");    // an Animal variable holding a Dog
System.out.println(pet.speak()); // Rex says Woof!
pet = new Cat("Tom");           // the same variable, now holding a Cat
System.out.println(pet.speak()); // Tom says Meow.
```

When you call a method, Java looks at the **object** (Dog, Cat), not the **variable** (Animal), to decide which version to run. This is called **dynamic dispatch**. It means you can write code that works with *every* kind of Animal, including kinds that don't exist yet:

```
static void chorus(List<Animal> animals) {
    for (Animal a : animals) {
        System.out.println(a.speak());
    }
}
```

The variable's type still matters for one thing: it decides **which methods you're allowed to call**. Through an **Animal** variable, you can only call methods that **Animal** declares. `pet.fetch()` doesn't compile, even when `pet` holds a Dog, because not every Animal can fetch. See `examples/Ex04Polymorphism.java`.

Abstract classes

In the hook, what would a plain **Shape** look like if you drew it? There's no such thing. **Shape** is an *idea*: the shared part of every real shape. Mark it **abstract**:

```
abstract class Shape {
    private final String name;

    Shape(String name) { this.name = name; }

    abstract double area();           // no body: every subclass MUST provide one
    abstract double perimeter();

    String describe() {               // a normal method can USE the abstract ones
        return String.format("%-10s area %7.2f", name, area());
    }
}
```

- You **can't create** an object of an abstract class: `new Shape("blob")` is a compile error.
- An **abstract method** has no body, just a semicolon. Every (non-abstract) subclass **must** override it, or it won't compile. That's a guarantee: anything that is a **Shape** definitely has an `area()`.
- An abstract class can still have fields, constructors and normal methods, which subclasses inherit.

See `examples/Ex05AbstractClasses.java`, where **Rectangle**, **Circle** and **Triangle** each work out their own area, and one loop adds them all up.

Access modifiers: who can see what

Modifier	Visible to
<code>private</code>	Only code inside the same class
<code>(none)</code>	Code in the same package (for now: the same folder)
<code>protected</code>	The same package, plus subclasses anywhere
<code>public</code>	Everyone

Subclasses **can't** see their superclass's `private` fields directly. Usually that's fine: they use the superclass's methods (or `super(...)` in the constructor). Use `protected` when subclasses genuinely need direct access to a field, as `balance` does in `examples/Ex06AccessModifiers.java`. The rule of thumb stays the same as Module 10: fields `private` unless there's a reason, methods `public` when other classes need them.

Object: the class at the top of every family

Every class you write **secretly extends** **Object**. `class Dog` really means `class Dog extends Object`. So every object in Java inherits a few methods from **Object**, including:

Method	Object 's version	Override it to...
<code>toString()</code>	the class name and a memory code	describe the object (Module 10)

Method	<code>Object</code> 's version	Override it to...
<code>equals(Object other)</code>	the same as <code>==</code> (the same object?)	compare data
<code>hashCode()</code>	a number based on the memory address	match your <code>equals</code>

This is why `println` works with every object (it calls `toString`), why a method parameter of type `Object` accepts anything, and why the checks in Module 8 could compare any two values with `.equals`.

To make two objects with the same data count as equal, override `equals`, and **always** override `hashCode` with it:

```
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Point)) return false;
    Point p = (Point) other;
    return x == p.x && y == p.y;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);    // built from the SAME fields as equals
}
```

`HashSet` and `HashMap` (Module 7) use `hashCode` to decide where to store an object, and `equals` to check for duplicates. If equal objects had different hash codes, a `HashSet` would happily store "duplicates" in different places. The rule: **equal objects must have equal hash codes**. `Objects.hash(...)` builds a good one from your fields. See `examples/Ex070ObjectClass.java`.

instanceof and casting

Occasionally you have a superclass variable and need to know what kind of object it holds. `instanceof` asks:

```
for (Device d : devices) {
    if (d instanceof Phone phone) {        // is it a Phone? If so, call it phone
        phone.call("Ama");                // now Phone's methods are available
    }
}
```

The `Phone phone` part (Java 16+) both tests and gives you a variable of the subclass type. In older code you'll see the two-step version: test with `instanceof`, then **cast** with `(Phone) d`. A cast to the wrong type compiles but crashes with a `ClassCastException`. See `examples/Ex08InstanceofAndCasting.java`.

Use `instanceof` sparingly. If you find yourself writing `if (x instanceof A) ... else if (x instanceof B) ...` to decide how to do something, that "something" should usually be an overridden method instead. That's exactly what the hook avoids.

Composition, and final

Inheritance is powerful, but it's not the only way to reuse code, and often not the best. The alternative is **composition**: give a class a field holding another object, and let it do part of the work. The test is "has a": a Car *has an* Engine; it isn't one.

```
class Car {
    private final Engine engine = new Engine(132);
    void start() {
        engine.start();        // delegate the job to the part that knows how
    }
}
```

`class Car extends Engine` would compile, but it would be nonsense: every Engine method would become a Car method. **Prefer composition** unless "is a" is clearly true.

Finally, **final** stops inheritance:

- A **final class** can't be extended. `String` is final, so nobody can make a "broken String".
- A **final method** can't be overridden.

See `examples/Ex09CompositionVsInheritance.java`, and then

`examples/Ex10PayrollReport.java`, which puts the module together: an abstract `Employee`, three kinds of staff paid in three different ways, and one polymorphic loop that prints the whole payroll.

Common beginner mistakes

- Forgetting `super(...)`, or not putting it first in the constructor.
- Misspelling an overriding method (`toString`, `ToString`) without `@Override`, which creates a new method that is never called. Always write `@Override`.
- Trying to use a subclass method through a superclass variable. The variable's type decides which methods are available.
- Using inheritance for "has a". A `Library` has books; it isn't a `List`.
- Overriding `equals` without `hashCode`, so `HashSet` and `HashMap` misbehave.
- `equals(Point other)` instead of `equals(Object other)`. That *overloads* rather than *overrides*, and collections never call it. `@Override` catches this too.
- Long `instanceof` chains where an overridden method would do.
- Trying to create an object of an abstract class.

Try it yourself

1. Run every file in `examples/`. In the hook, add a new kind of shape (a `Heart`? a `Hexagon`? a `Smiley`?) and put a few in the parade.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: a staff payroll, AC circuit components, or a medical device fleet.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 11: inheritance and polymorphism"  
git push
```

Module 12: Interfaces, Enums & Records

Hook: five rankings, five lines of code

Run `examples/Ex00Leaderboard.java`:

```
java Ex00Leaderboard.java
```

```
By score, highest first:
Kojo   PRESEC      92    47.9 s
Akosua ACHIMOTA      92    44.1 s
Ama    WESLEY_GIRLS 88    41.2 s
...

Score, then fastest as the tie-breaker:
Akosua ACHIMOTA      92    44.1 s
Kojo   PRESEC      92    47.9 s
Efua   ACHIMOTA      88    39.5 s
...
```

The same six results, ranked five different ways. Each ranking is **one line**:

```
Comparator.comparingInt(Result::score).reversed()
Comparator.comparing(Result::name)
Comparator.comparingDouble(Result::seconds)
Comparator.comparing(Result::school).thenComparing(Result::score,
Comparator.reverseOrder())
Comparator.comparingInt(Result::score).reversed().thenComparingDouble(Result::second
s)
```

And look how the data is defined: a school is one of a **fixed list** of values, and a result is a single line:

```
enum School { ACHIMOTA, PRESEC, WESLEY_GIRLS, MFANTSIPIIM, ST_PETERS }

record Result(String name, School school, int score, double seconds) { }
```

That `record` line gives you a complete class with a constructor, getters, `equals`, `hashCode` and `toString`, all written by Java. This module covers the three tools in the hook: **interfaces** (which is what a `Comparator` is), **enums**, and **records**.

Interfaces: a contract

An **interface** lists methods that a class **promises** to provide, without saying how:

```
interface Chargeable {
    int batteryPercent();
    void charge(int minutes);
}

class Phone implements Chargeable {
    private int battery = 20;
    public int batteryPercent() { return battery; }
    public void charge(int minutes) { battery = Math.min(100, battery + minutes); }
}
```

- **implements Chargeable** signs the contract. The class **must** provide every method in the interface, or it won't compile.
- Interface methods are automatically **public** and **abstract**, so the implementing methods must be **public**.
- You can't create an interface (**new Chargeable()**), but you can use it as a **type**: a **Chargeable** variable can hold a Phone, an electric bike, a robot, anything that implements it.

```
static void chargeAll(List<Chargeable> things, int minutes) {
    for (Chargeable c : things) {
        c.charge(minutes);
    }
}
```

This is polymorphism again, as in Module 11, but **without inheritance**. A phone, a bike and a robot have nothing else in common, and they don't need to. See [examples/Ex01FirstInterface.java](#).

Interfaces versus abstract classes

	Abstract class	Interface
A class can...	extend one	implement many
Fields	any	only constants
Constructors	yes	no
Methods with code	yes	only default and static methods
Use it for	a family that shares state and code ("is a")	a capability unrelated classes share ("can do")

Because a class can implement any number of interfaces, they're how Java lets one class play several roles: **class Player extends GameCharacter implements Drawable, Movable, Named**. See [examples/Ex02MultipleInterfaces.java](#).

Default and static methods

An interface can include a **default** method, with a body that every implementing class gets for free (and may override), and **static** helper methods, called on the interface itself:

```
interface Sensor {
    double read();
    String unit();

    default String describe() {
        return String.format("%.2f %s", read(), unit());
    }

    static double celsiusToFahrenheit(double c) {
        return c * 9 / 5 + 32;
    }
}
```

See [examples/Ex03DefaultAndStaticMethods.java](#).

Coding to the interface

You've already been using interfaces. `List`, `Map` and `Set` are interfaces; `ArrayList`, `HashMap` and `HashSet` are classes that implement them. That's why good Java style declares variables with the interface type:

```
List<String> names = new ArrayList<>();
Map<String, Integer> counts = new HashMap<>();
```

The rest of the program only relies on `List`'s contract, so switching to a different implementation later means changing one line.

Comparable: a natural order

`Collections.sort`, `TreeSet`, `TreeMap` and `Collections.max` need to know how to order your objects. A class gives itself a **natural order** by implementing the `Comparable` interface, which has one method:

```
class Student implements Comparable<Student> {
    ...
    @Override
    public int compareTo(Student other) {
        return id.compareTo(other.id);    // natural order: by ID
    }
}
```

`a.compareTo(b)` returns a **negative** number if `a` comes first, a **positive** number if `b` comes first, and **0** if they're equal in this order. `String`, `Integer`, `Double` and enums already implement it, so the easiest way to write a `compareTo` is usually to call theirs: `id.compareTo(other.id)`, or `Double.compare(gpa, other.gpa)` for primitives. (Don't subtract numbers to compare them: with large values the subtraction can overflow.) See [examples/Ex04Comparable.java](#).

Lambdas

An interface with exactly **one** abstract method is called a **functional interface**. Instead of writing a whole class to implement it, you can write a **lambda**: a tiny function, written inline:

```
interface Operation {
    double apply(double a, double b);
}

Operation multiply = (a, b) -> a * b;
multiply.apply(12, 4)           // 48.0
```

Read `(a, b) -> a * b` as "given `a` and `b`, give back `a * b`". The parameter types come from the interface, so you don't write them. A lambda with several statements uses braces and `return`:

```
Operation safeDivide = (a, b) -> {
    if (b == 0) {
        return 0;
    }
    return a / b;
};
```

A lambda is an object like any other: you can store it in a variable, pass it to a method, or put it in a list. Java has many built-in functional interfaces, such as `Runnable` (no parameters, no result: `() -> System.out.println("Hi")`). Module 15 introduces the most useful ones. See [examples/Ex05Lambdas.java](#).

Comparator: as many orders as you like

`Comparable` gives a class **one** natural order, built in. A `Comparator` is an order that lives **outside** the class, so you can have as many as you want. It's a functional interface, so a lambda can be one:

```
phones.sort((a, b) -> Double.compare(a.getPrice(), b.getPrice()));
```

Even neater, `Comparator` has static methods that build an order from a **key**, the thing to compare by:

```
phones.sort(Comparator.comparingDouble(Phone::getPrice));           // cheapest
first
phones.sort(Comparator.comparingDouble(Phone::getPrice).reversed()); // dearest
first
phones.sort(Comparator.comparingInt(Phone::getStorageGb).reversed()
    .thenComparingDouble(Phone::getPrice));           // most
storage, then cheapest
```

`Phone::getPrice` is a **method reference**: a short way to write the lambda `p -> p.getPrice()`. Use `comparing` for keys that are objects (`String`, enums), and `comparingInt` / `comparingDouble` for numbers. `reversed()` flips the order; `thenComparing` breaks ties. Now you can read every line of the hook. See [examples/Ex06Comparator.java](#).

Enums: a fixed set of values

Some types have a small, fixed set of possible values: the days of the week, the colours of a traffic light, the grades A to F. You *could* use Strings (`"MONDAY"`), but then nothing stops a typo like `"MONDYA"`. An **enum** (enumeration) makes the set official:

```
enum TrafficLight { RED, AMBER, GREEN }

TrafficLight light = TrafficLight.RED;
```

- The compiler rejects any value that isn't on the list. `TrafficLight.BLUE` doesn't compile.
- Enum values **can** be compared with `==`: there's only ever one `TrafficLight.RED` object.
- They work beautifully with `switch`, and a switch expression over an enum needn't have a `default` if it covers every value (the compiler checks).
- `TrafficLight.values()` gives every value, in the order declared; `ordinal()` gives a value's position; `TrafficLight.valueOf("RED")` turns text into a value (and throws `IllegalArgumentException` for anything else).
- Enums are `Comparable`, in declaration order, and have a ready-made `toString` that gives the value's name.

See `examples/Ex07Enums.java`.

Enums with data

An enum is really a special kind of class, so each value can carry its own data, through fields and a constructor:

```
enum Grade {
    A(80, 4.0), B_PLUS(75, 3.5), B(70, 3.0), C(60, 2.0), D(50, 1.0), F(0, 0.0);

    final int minimumMark;
    final double point;

    Grade(int minimumMark, double point) {
        this.minimumMark = minimumMark;
        this.point = point;
    }

    static Grade forMark(int mark) {
        for (Grade g : values()) {
            if (mark >= g.minimumMark) {
                return g;
            }
        }
        throw new IllegalArgumentException("Bad mark: " + mark);
    }
}
```

Compare this with the Module 4 grade exercise: the whole grading table now lives in one place, as data. See `examples/Ex08EnumsWithFields.java`, which also works out your weight on four planets.

Records: data classes in one line

Many classes just **hold data**: a point, a reading, a result. Writing private final fields, a constructor, getters, `equals`, `hashCode` and `toString` for each is repetitive. A **record** (Java 16+) does it all:

```
record Point(int x, int y) { }
```

```
Point p = new Point(3, 4);
p.x()           // 3: the getter has the field's name, no "get"
System.out.println(p);      // Point[x=3, y=4]
p.equals(new Point(3, 4))    // true: compares the data
```

Records are **immutable**: their fields are `final`, and there are no setters. To "change" one, make a new one. That makes them safe to share and perfect as keys in a `HashMap` or values in a `HashSet`.

A record can still check its data, in a **compact constructor** (no parameter list; the fields are assigned automatically afterwards), and can have extra methods:

```
record Reading(String sensor, double value, String unit) {
    Reading {
        if (sensor == null || sensor.isBlank()) {
            throw new IllegalArgumentException("A reading needs a sensor name.");
        }
    }

    String describe() {
        return sensor + ": " + value + " " + unit;
    }
}
```

Use a **record** for plain data that shouldn't change. Use a **class** when an object has state that changes over time (a bank balance, a pet's hunger) or behaviour that protects it. See [examples/Ex09Records.java](#).

An abstract data type

[examples/Ex10StackInterface.java](#) puts interfaces to classic use. A **Stack** interface says **what** a stack does (push, pop, peek: last in, first out), and **ArrayStack** says **how**, with an array that doubles in size when it fills up. A bracket checker uses the stack without knowing anything about arrays. Swap in a different implementation and the checker doesn't change. Describing a data structure by its operations, separately from how it's stored, is called an **abstract data type**, and interfaces are how Java expresses one.

Common beginner mistakes

- **Forgetting `public`** on methods that implement an interface: "attempting to assign weaker access privileges".
- **A `compareTo` that isn't consistent**, such as returning `1` for both `a.compareTo(b)` and `b.compareTo(a)`. Use `Integer.compare`, `Double.compare` and `String.compareTo` rather than inventing your own.
- **Sorting by subtracting numbers** (`return a.gpa - b.gpa;` doesn't even compile for doubles, and overflows for big ints).
- **Calling `valueOf` on text with the wrong capitals**. Enum names are usually UPPER_CASE: `Day.valueOf(text.toUpperCase())`.
- **Trying to change a record's field**. Create a new record instead.
- **Using a record for something that changes**, like a stock level or a balance. That's a job for a class.
- **Using Strings for a fixed set of categories** when an enum would catch typos at compile time.
- **Writing `p.getX()` for a record**. Record getters are named after the field: `p.x()`.

Try it yourself

1. Run every file in [examples/](#). In the hook, add a sixth ranking of your own.
2. Complete [exercises/Exercise1.java](#) and [exercises/Exercise2.java](#).

3. Build the module project in the track of your choice: a media library, a logic gate simulator, or an emergency triage queue.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 12: interfaces, enums and records"  
git push
```

Module 13: Exceptions & Debugging

Hook: try to break this calculator

Run `examples/Ex00CrashProofCalculator.java` and do your worst:

```
java Ex00CrashProofCalculator.java

=== ROUND 1: no safety net ===
Sum: 7 / 0
CRASH! java.lang.ArithmeticException: You can't divide by zero.
(Without a try/catch, the program would end here.)

=== ROUND 2: crash-proof (type quit to stop) ===
Sum: ten + 2
Those don't look like numbers. Try: 12 * 4
Sum: 7 / 0
You can't divide by zero.
Sum: 2 ^ 10
Unknown operator: ^
Sum:
Type a sum like: 12 * 4
Sum: 12 * 4
= 48.0
```

In Round 1, a single bad answer ends everything: that's what every program you've written so far does. Round 2 runs the **same** calculation code, but it survives letters, division by zero, unknown operators and empty lines, and explains each problem in plain English. The difference is a few lines of `try` and `catch`.

Real software is used by real people, who type anything. A program that crashes on bad input is a broken program. This module shows you how to handle problems gracefully with **exceptions**, how to create your own, and how to hunt down the bugs that *don't* crash, using the **debugger**.

What an exception is

When something goes wrong while a program is running (a division by zero, text where a number should be, an index past the end of an array), Java **throws an exception**: it creates an object describing the problem, stops what it was doing, and looks for code that will **catch** it. If nothing catches it, the program ends and prints a **stack trace**.

You've met plenty already: `ArithmeticException`, `NumberFormatException`, `ArrayIndexOutOfBoundsException`, `NullPointerException`, and the `IllegalArgumentException`s you've been throwing since Module 10.

Reading a stack trace

Run `examples/Ex01ReadingStackTraces.java`:

```
Akua: average 80
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Ex01ReadingStackTraces.average(Ex01ReadingStackTraces.java:13)
    at Ex01ReadingStackTraces.report(Ex01ReadingStackTraces.java:17)
    at Ex01ReadingStackTraces.main(Ex01ReadingStackTraces.java:22)
```

1. **The first line** says *what*: the exception's type (`ArithmeticException`) and its message (`/ by zero`).
2. **The `at` lines** say *where*, as a chain of method calls with the **most recent first**: the crash happened in `average` at line 13, which was called by `report` at line 17, which was called by `main` at line 22. This is the **call stack** from Module 8, printed out.
3. **Look for the first line that mentions your own code.** In bigger programs the top lines are often inside Java's library (at `java.base/...`). The bug is almost always where *your* code called it.

Here the crash is in `average`, but the *cause* is at line 22: `main` passed it an empty array. Reading the whole chain tells you both.

try and catch

Put code that might throw in a `try` block, and say what to do if it does in a `catch` block:

```
try {
    int age = Integer.parseInt(text.trim());
    System.out.println("Next year you'll be " + (age + 1));
} catch (NumberFormatException e) {
    System.out.println("'" + text + "' isn't a whole number.");
}
System.out.println("The program carries on either way.");
```

- If nothing goes wrong, the `try` block runs to the end and the `catch` is skipped.
- If a line throws, Java **jumps immediately** to the matching `catch`. The rest of the `try` block is skipped (so there's no "next year" line).
- After the `catch`, the program **carries on** normally.
- `e` is the exception object. `e.getMessage()` gives its message, and printing `e` gives its type and message.

Combine `try/catch` with a loop, and you get the most useful pattern in this module, **keep asking until it works**:

```
while (true) {
    System.out.print("Type a whole number: ");
    try {
        number = Integer.parseInt(in.nextLine().trim());
        break; // only reached if parseInt succeeded
    } catch (NumberFormatException e) {
        System.out.println(" Not a whole number. Try again.");
    }
}
```

See `examples/Ex02TryCatch.java`.

Several catches

A `try` can have several `catch` blocks, one per kind of problem. Java uses the **first** one that matches:

```
try {
    ...
} catch (NumberFormatException e) {
    System.out.println("Please type whole numbers.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("There's no such box.");
} catch (ArithmeticException e) {
    System.out.println("Nobody to share with!");
}
```

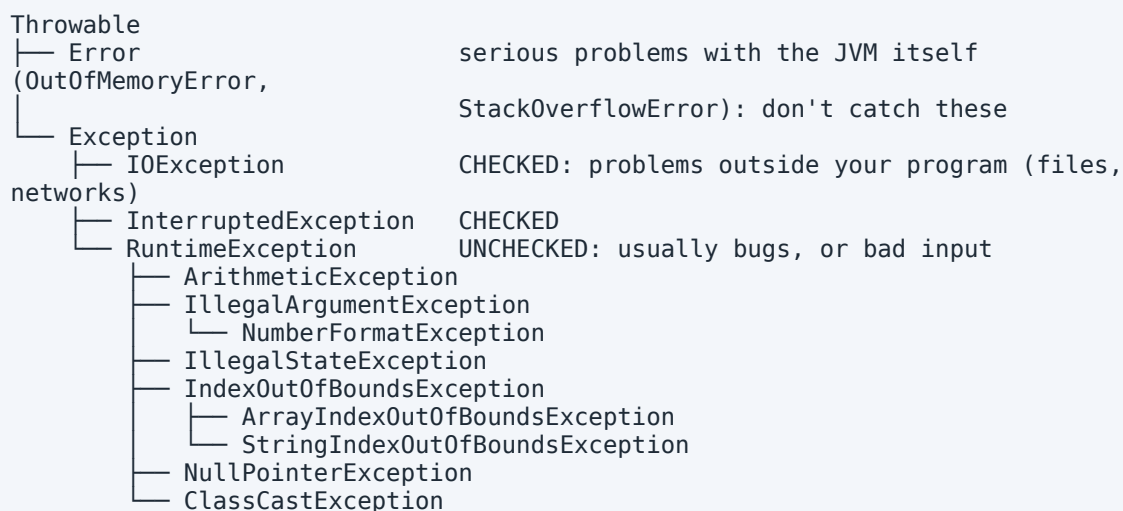
To handle several kinds the same way, join them with `|`: `catch (ClassCastException | NullPointerException e)`. See [examples/Ex03MultipleCatches.java](#).

finally

A **finally** block runs **whatever happens**: after the `try` succeeds, after a `catch` handles a problem, even after a `return`. It's where clean-up goes: switching equipment off, closing a file (Module 14), printing a footer. See [examples/Ex04Finally.java](#).

The exception family tree

Exceptions are objects, and their classes form an inheritance hierarchy (Module 11):



Because of inheritance, a `catch` catches its type **and every subclass**: `catch (IllegalArgumentException e)` also catches `NumberFormatException`; `catch (Exception e)` catches almost everything. So put specific catches **before** general ones (the compiler complains if a general catch makes a later one unreachable). Catch the most specific type you can: `catch (Exception e)` everywhere hides bugs. See [examples/Ex05Hierarchy.java](#).

Throwing exceptions

You've thrown exceptions since Module 10 to stop invalid objects being created. The two you'll use most:

```

if (ohms <= 0) {
    throw new IllegalArgumentException("Resistance must be positive, not " + ohms);
}
if (amount > charge) {
    throw new IllegalStateException("Only " + charge + "% left");
}

```

- **IllegalArgumentException**: the **argument** is bad (a negative resistance, an empty name).
- **IllegalStateException**: the argument is fine, but the **object** can't do it right now (the battery is flat, the queue is empty).

Throwing early, with a precise message, is far better than carrying on with bad data and failing mysteriously later. A method that detects a problem it can't sensibly fix should **throw**; the code that *can* decide what to do (usually the part talking to the user) should **catch**. See [examples/Ex06ThrowingExceptions.java](#).

Checked and unchecked exceptions

Look at the family tree again. Java treats two groups differently:

- **Unchecked** exceptions (**RuntimeException** and its subclasses) usually mean a bug or bad input. You *may* catch them, but the compiler doesn't insist.
- **Checked** exceptions (every other **Exception**, such as **IOException**) are problems outside your program's control: a missing file, a network failure. The compiler **insists** that you deal with them, in one of two ways: catch them, or **declare** that your method passes them on, with **throws**:

```

static String firstLine(String fileName) throws IOException {
    return Files.readAllLines(Path.of(fileName)).get(0);
}

```

Forget, and the code doesn't compile: "unreported exception IOException; must be caught or declared to be thrown". This is why Module 5's countdown needed **throws InterruptedException** on **main**: **Thread.sleep** throws a checked exception. See [examples/Ex07CheckedExceptions.java](#).

Your own exceptions

When the built-in types don't say exactly what went wrong, write your own. An exception class is an ordinary class that extends **Exception** (checked) or **RuntimeException** (unchecked):

```

class InsufficientFundsException extends Exception {
    private final double shortBy;

    InsufficientFundsException(double shortBy) {
        super(String.format("Insufficient funds: short by GHS %.2f", shortBy));
        this.shortBy = shortBy;
    }

    double getShortBy() {
        return shortBy;
    }
}

```

`super(message)` passes the message up to `Exception`, so `getMessage()` works. Extra fields carry details the caller can use, such as how much more money is needed. Then a method can say exactly what can go wrong, and callers can catch exactly that:

```
void withdraw(double amount) throws InsufficientFundsException { ... }

try {
    account.withdraw(120);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage() + " (top up at least " + e.getShortBy() +
    ")");
}
```

Make an exception **checked** when the caller can reasonably be expected to recover (not enough money, a wrong PIN). Make it **unchecked** when it signals a mistake in the calling code. See [examples/Ex08CustomExceptions.java](#).

Crash-proof input helpers

[examples/Ex09InputHelpers.java](#) wraps the "keep asking" pattern in reusable methods: `readInt(prompt, min, max)`, `readDouble(...)` and `readNonEmpty(...)`. Write them once, and every program you write from now on can read numbers without crashing. You'll build your own versions in Exercise 1.

The debugger: finding bugs that don't crash

The worst bugs don't throw anything. The program runs happily and gives the **wrong answer**. [examples/Ex10BuggyAverage.java](#) says the average of 70, 85 and 90 is 87.50. It should be 81.67. Where's the bug?

You could add `println`s everywhere. Or you could use the **debugger**, which lets you pause a running program, look inside every variable, and run it one line at a time. Here's how, in VS Code:

1. Open [Ex10BuggyAverage.java](#). Click in the margin **just left of the line number** of the line `int total = 0;`. A red dot appears: a **breakpoint**, a place where the program will pause.
2. Click **Debug** in the little menu that appears above `main` (or press **F5**). The program starts, and stops at your breakpoint. That line is highlighted: it's about to run.
3. Look at the **Variables** panel on the left. You can see `values` (click the arrow to expand it: `[70, 85, 90]`).
4. Press **F10 (Step Over)** to run one line at a time. Watch `i` and `total` in the Variables panel as the loop goes round. The first value of `i` is `1`, so `values[0]`, the 70, is never added. Found it.
5. Press **Shift+F5** to stop. Fix the loop to start at 0, fix the division, and run it again.

The other controls on the debug toolbar:

Button	Key	Does
Continue	F5	Run until the next breakpoint
Step Over	F10	Run this line (including any method it calls), then pause
Step Into	F11	If this line calls one of <i>your</i> methods, go inside it

Button	Key	Does
Step Out	Shift+F11	Finish this method, and pause back in its caller
Restart / Stop	Ctrl+Shift+F5 / Shift+F5	

You can also **hover** over any variable in the code to see its value, and add expressions like `total / values.length` to the **Watch** panel. When a program *does* crash, put a breakpoint a few lines before the line in the stack trace and step up to it: you'll see exactly which value was wrong.

Common beginner mistakes

- **Catching an exception and doing nothing:** `catch (Exception e) { }`. The problem is hidden, not solved. At the very least, print a message.
- **Catching `Exception` everywhere.** Catch the specific type you expect, so real bugs still show up.
- **A general `catch` before a specific one,** making the specific one unreachable (a compile error).
- **Forgetting that the rest of the `try` block is skipped** after an exception. Variables assigned there may not have their values.
- **Declaring a variable inside `try` and using it after.** It's out of scope. Declare it before the `try`.
- **Throwing `Exception` itself** instead of a meaningful type. Use `IllegalArgumentException`, `IllegalStateException`, or your own class.
- **Using exceptions for ordinary decisions.** Checking `if (list.isEmpty())` is better than catching `IndexOutOfBoundsException`.
- **Guessing instead of debugging.** If you've changed the same line three times and it's still wrong, set a breakpoint and look.

Try it yourself

1. Run every file in `examples/`. Find the bug in `Ex10BuggyAverage.java` with the debugger, following the steps above.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: an ATM, a safe circuit calculator, or a vitals entry validator.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 13: exceptions and debugging"
git push
```

Module 14: Files & Data

Hook: a program that remembers you

Run `examples/Ex00Remember.java`:

```
java Ex00Remember.java
```

```
Hello! I don't think we've met. What's your name? Kojo  
Nice to meet you, Kojo. I'll remember you.
```

Now run it **again**:

```
Welcome back, Kojo!  
This is visit number 2. Last time was 4 Oct 2026 at 17:06.
```

And again. It counts every visit. Close VS Code, restart your computer, come back next week: it still knows you. Every program you've written so far has had **amnesia**: the moment it ended, everything in its variables was gone. This one writes what it knows to a **file**, `memory.txt`, and reads it back next time. Open the file in VS Code and look: three ordinary lines of text. Delete it, and the program forgets you.

Files are how programs keep data between runs: save games, settings, spreadsheets, logs, photos. This module covers reading and writing text files, the **CSV** format that spreadsheets and lab equipment use, and how to survive the broken data that real files always contain.

Paths: where a file is

A **Path** (from `java.nio.file`) says where a file is:

```
Path file = Path.of("greeting.txt");           // in the current folder  
Path data = Path.of("data", "students.csv");    // data/students.csv
```

`Path.of("data", "students.csv")` joins the parts with the right separator for the computer it's running on: `/` on macOS and Linux, `\` on Windows. So use separate parts rather than typing slashes yourself.

A path like `data/students.csv` is **relative**: it starts from the **working directory**, the folder you were in when you ran `java`. If Java says a file doesn't exist and you're sure it does, you're almost always running from a different folder. `Path.of("").toAbsolutePath()` tells you where Java is looking. (In VS Code, the  button usually runs from the folder you opened. Run the examples from the terminal, in the `examples` folder, to be sure.)

Writing and reading whole files

The `Files` class does most jobs in one line:

```
Files.writeString(file, "Akwaaba!\nWelcome to file handling.\n"); // write text
Files.write(Path.of("shopping.txt"), List.of("rice", "tomatoes")); // write lines

String everything = Files.readString(file); // the whole file as one
String
List<String> lines = Files.readAllLines(file); // one String per line
boolean there = Files.exists(file);
```

- Writing **replaces** whatever was in the file before. If the file doesn't exist, it's created.
- **Files.write** adds the line endings for you; **writeString** writes exactly what you give it, so include the `\ns`.
- Every one of these can throw an **IOException** (a checked exception, from Module 13): the file might be missing, locked by another program, or on a disk that's full. So you must either catch it or declare **throws IOException**. Reading a file that doesn't exist throws its subclass **NoSuchFileException**.

See `examples/Ex01WritingText.java` and `examples/Ex02ReadingText.java`.

Appending

To add to the end of a file instead of replacing it, pass **open options**:

```
Files.writeString(log, line, StandardOpenOption.CREATE, StandardOpenOption.APPEND);
```

CREATE makes the file if it's missing; **APPEND** adds to the end. This is how **logs** work: every run adds a line, and nothing is ever lost. Run `examples/Ex03Appending.java` a few times and watch `lab-log.txt` grow.

Line by line, with try-with-resources

readAllLines loads the whole file into memory at once. That's fine for the files in this course, but a sensor log can be gigabytes. To process a file **one line at a time**, open a **reader**:

```
try (BufferedReader reader = Files.newBufferedReader(input);
     PrintWriter writer = new PrintWriter(Files.newBufferedWriter(output))) {

    String line;
    while ((line = reader.readLine()) != null) { // null means "end of the file"
        writer.println(line.toUpperCase());
    }
}
```

A reader or writer holds the file **open** while you use it, and must be **closed** afterwards, or the operating system can run out of file handles, and written data may never actually reach the disk. The **try-with-resources** statement does it for you: anything opened in the brackets after **try** is closed automatically when the block ends, whether it finishes normally or with an exception. It's a neater, safer version of the **finally** from Module 13. Always open readers and writers this way.

PrintWriter gives a file the same **println** and **printf** you use with **System.out**. You can also read a file with a **Scanner**, exactly as you read the keyboard: `new Scanner(Path.of("data", "readings.txt"))`. See `examples/Ex04TryWithResources.java` and `examples/Ex05ScannerOverFile.java`.

CSV: data in rows and columns

A **CSV** (comma-separated values) file is a table in plain text. The first line is usually a **header** naming the columns, and each following line is one row:

```
id,name,programme,gpa
S001,Akua Mensah,BME,3.85
S002,Kwesi Appiah,EEE,3.10
```

Every spreadsheet can open and save CSV, and so can most lab equipment and databases. Reading one is a combination of things you know: read the lines, skip the header, `split(",")` each line, and turn the fields into an object:

```
for (int i = 1; i < lines.size(); i++) {           // start at 1: skip the header
    String[] f = lines.get(i).split(",");
    students.add(new Student(f[0], f[1], f[2], Double.parseDouble(f[3])));
}
```

Records (Module 12) are perfect for rows of data. Give the record a `toCsv()` method that turns it back into a line, and a static `fromCsv(line)` that builds one from a line, and saving and loading becomes a **round trip**: save a list, load it back, and you get equal objects. See `examples/Ex07WritingCSV.java`.

Commas inside the data (an address like `12 High St, Accra`) break simple splitting. Real CSV files put such fields in quotes, and libraries like OpenCSV handle that. In this course, reject commas in text fields instead.

Real data is messy

Real files contain mistakes: a word where a number should be, a missing field, a blank line. A program that crashes on the first bad line is useless. Wrap the processing of **each line** in its own `try/catch`, so a bad line is **skipped and reported**, and the rest still load:

```
try {
    if (f.length != 4) {
        throw new IllegalArgumentException("expected 4 fields, found " + f.length);
    }
    students.add(new Student(f[0], f[1], f[2], Double.parseDouble(f[3])));
} catch (IllegalArgumentException e) {           // NumberFormatException is one of
these
    System.out.println(" Skipping line " + (i + 1) + ": " + e.getMessage());
}
```

Telling the user **which line** and **why** turns a mystery into a two-second fix. Run `examples/Ex06ReadingCSV.java`: `data/students.csv` has a GPA written in words, a missing field and a blank line, and the program loads everything else.

Folders, and Properties

`Files` can also manage folders:

```
Files.createDirectories(Path.of("output", "reports")); // every missing folder in
the path
Path report = folder.resolve("january.txt");          // folder + file name
Files.list(folder)                                    // the folder's contents
Files.delete(report);                                // gone for good: no
recycle bin!
```

See `examples/Ex08Directories.java`. For simple settings, the `Properties` class reads and writes `key=value` files, like a `HashMap` that saves itself. See `examples/Ex09Settings.java`.

What about JSON? Most data on the web is in **JSON** format (`{"name": "Akua", "gpa": 3.85}`). Java's standard library can't read JSON by itself. You need a **library**, such as Gson or Jackson. In Module 17 you'll add Gson to a project with Maven in one line.

A note on text and bytes

Everything in this module reads and writes **text**: Java turns characters into bytes using an encoding (UTF-8, by default, on modern Java). Images, audio and compressed files aren't text. For those, use `Files.readAllBytes(path)` and `Files.write(path, bytes)`, which work with raw `byte[]` arrays. (In older Java code and textbooks you'll see `FileReader/FileWriter` for **character streams** and `FileInputStream/FileOutputStream` for **byte streams**: the same ideas, with more typing.)

Common beginner mistakes

- **The wrong working directory**, so relative paths point somewhere else. Check with `Path.of("").toAbsolutePath()`.
- **Forgetting that writing replaces the file**. Use `APPEND` to add to it.
- **Not closing readers and writers**. Use try-with-resources, always.
- **Typing `"data\\students.csv"` or `"data/students.csv"`** instead of `Path.of("data", "students.csv")`, so it breaks on another operating system.
- **Forgetting to skip the header line** of a CSV, so the first row fails to parse.
- **Letting one bad line crash the whole load**. Catch errors **per line**.
- **`split(",")` and empty last fields**. `"a,b,".split(",")` gives just two fields; Java drops empty strings at the end. Check the length.
- **Saving only when the program quits**. If it crashes, or the power goes off, everything is lost. Save after every change.

Try it yourself

1. Run every file in `examples/`, from the `examples` folder. Open every file they create, and look inside.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: an expense tracker, a bench data analyser, or a patient vitals log.

4. Commit and push your work. Data files in `data/` folders are part of your project, so commit them too. Files your programs *generate* don't belong in Git: add a line saying `output/` to your coursework repository's `.gitignore` file, and they'll be left out.

```
git add .  
git commit -m "Complete Module 14: files and data"  
git push
```

Module 14b: Databases with SQLite & JDBC (optional)

This module is optional. Module 15 doesn't depend on it. Do it if you'll build software that keeps lots of records (student records, stock, readings, patients), or if a course assessment asks for a database. It follows on directly from Module 14.

Hook: two thousand students, five questions

From this module's folder, run `examples/Ex00StudentDatabase.java`:

```
java -cp "lib/*" examples/Ex00StudentDatabase.java
```

```
Built school.db with 2,000 students in 1211 ms.
```

```
How many students in each programme?
```

```
SQL: SELECT programme, COUNT(*) FROM students GROUP BY programme ORDER BY programme
```

```
BME 375
CIV 395
CSC 429
EEE 407
MEC 394
```

```
The top five students?
```

```
SQL: SELECT id, name, programme, gpa FROM students ORDER BY gpa DESC, id LIMIT 5
S0950 Nii Appiah CSC 4.0
S1043 Esi Darko BME 4.0
...
```

```
How many are on probation (GPA below 2.0)?
```

```
SQL: SELECT COUNT(*) FROM students WHERE gpa < 2.0
386
```

The program built a **database** of 2,000 students in a file called `school.db`, and then asked it five questions. Each question is a single line in **SQL** (Structured Query Language), the language almost every database in the world understands. The database did the searching, sorting, counting and averaging; Java just asked and printed.

In Module 14 you stored records in CSV files. That works, until you have thousands of records, several programs using them at once, or data that must never be half-saved. That's what **databases** are for. This module uses **SQLite**, a complete database in a single file, through **JDBC**, Java's standard way of talking to any database.

What you need: the SQLite driver

Java's JDBC classes (in `java.sql`) know how to talk to a database, but each kind of database needs its own **driver**: a library, in a `.jar` file, that does the actual talking. The SQLite driver is `sqlite-jdbc`, and it's already in this module's `lib/` folder. It contains SQLite itself, so there's nothing else to install.

To use a library, put it on the **class path** with `-cp`:

```
java -cp "lib/*" examples/Ex01Connecting.java
```

"lib/*" means "every .jar file in the lib folder". Keep the quotation marks: they work the same in PowerShell, Command Prompt, and the macOS and Linux terminals. Run every example and exercise from this module's folder, so lib/ and data/ are where Java expects.

In VS Code: the Java extension automatically uses any .jar files in a lib folder at the top of the folder you opened. If you open this module's folder, the  button just works. In your own coursework folder, either put lib/ at the top, or add the jar under **Java Projects → Referenced Libraries**.

Once you've met **Maven** (Module 17), you won't need a lib folder: you'll add the driver to a project as a dependency in pom.xml, and Maven downloads it:

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.53.4.0</version>
</dependency>
```

Tables, rows and columns

A database holds **tables**. A table looks like a CSV file: **columns** across the top, one **row** per record. But unlike a CSV file, a table has **rules**:

```
CREATE TABLE products (
  code      TEXT PRIMARY KEY,           -- unique: no two products share a
code
  name      TEXT NOT NULL,             -- must be given
  category  TEXT NOT NULL DEFAULT 'Other',
  price     REAL NOT NULL CHECK (price >= 0), -- the database refuses negatives
  quantity  INTEGER NOT NULL DEFAULT 0
)
```

SQLite type	Holds	Java side
INTEGER	whole numbers	int, long
REAL	decimals	double
TEXT	text	String

The **constraints** are the database version of Module 10's validation: a **PRIMARY KEY** must be unique, **NOT NULL** means a value is required, and **CHECK** tests any condition. Break one, and the database refuses the change with an **SQLException**, so bad data never gets in, whichever program is writing. (SQLite has no **true/false** type: use an **INTEGER** holding 0 or 1. Store dates as **TEXT** like 2026-10-04, which sorts correctly.) See examples/Ex02CreateTable.java.

Connecting

```
try (Connection db = DriverManager.getConnection("jdbc:sqlite:school.db")) {
  // ... use db ...
}
```

- The URL `jdbc:sqlite:school.db` says "SQLite, the file `school.db`". If the file doesn't exist, it's created.
- `jdbc:sqlite::memory:` makes a database in memory, which vanishes when the connection closes. It's perfect for tests and exercises.
- A `Connection` holds a resource open, like a file reader, so open it in **try-with-resources** (Module 14), and it's always closed.
- Every JDBC method can throw `SQLException`, a checked exception (Module 13): catch it, or declare `throws SQLException`.

See `examples/Ex01Connecting.java`.

The four operations: CRUD

Almost everything a program does with stored data is one of four operations, known as **CRUD**:

Operation	SQL	JDBC method
Create	<code>INSERT INTO products (code, name) VALUES (?, ?)</code>	<code>executeUpdate()</code>
Read	<code>SELECT name, price FROM products WHERE price < ?</code>	<code>executeQuery()</code> → a <code>ResultSet</code>
Update	<code>UPDATE products SET quantity = ? WHERE code = ?</code>	<code>executeUpdate()</code>
Delete	<code>DELETE FROM products WHERE code = ?</code>	<code>executeUpdate()</code>

`executeUpdate()` returns **how many rows changed**, which is how you know whether an update or delete found anything to change.

Create: INSERT with a PreparedStatement

```
String sql = "INSERT INTO products (code, name, category, price, quantity) VALUES (?, ?, ?, ?, ?)";
try (PreparedStatement insert = db.prepareStatement(sql)) {
    insert.setString(1, "A01");           // fill in the first ?
    insert.setString(2, "Arduino Uno");
    insert.setString(3, "Boards");
    insert.setDouble(4, 180.0);
    insert.setInt(5, 12);
    insert.executeUpdate();
}
```

The SQL is written once, with a **?** placeholder for each value; then `setString`, `setInt` and `setDouble` fill them in, numbered **from 1**. The same prepared statement can be filled in and run again and again, which is how `examples/Ex03Insert.java` inserts a whole list of records.

Read: **SELECT** and the **ResultSet**

```
try (PreparedStatement query = db.prepareStatement(
    "SELECT code, name, price FROM products WHERE price < ? ORDER BY price
DESC")) {
    query.setDouble(1, 50.0);
    try (ResultSet rows = query.executeQuery()) {
        while (rows.next()) {
            String name = rows.getString("name");
            double price = rows.getDouble("price");
            ...
        }
    }
}
```

A **ResultSet** is a **cursor** that starts *before* the first row. Each **next()** moves to the next row, and returns **false** when there are no more, so **while (rows.next())** visits every row. Read the current row's columns with **getString**, **getInt** and **getDouble**, by name or by position (from 1). To look up one row by its key, check whether **next()** found anything: **if (rows.next()) { ... } else { not found }**.

The parts of a **SELECT**:

Part	Does	Example
SELECT columns	which columns (* for all)	SELECT name, price
FROM table	which table	FROM products
WHERE condition	which rows (AND , OR , NOT , LIKE)	WHERE price < ? AND category = ?
ORDER BY column	sorting (DESC for biggest first)	ORDER BY price DESC
LIMIT n	at most n rows	LIMIT 5

LIKE matches patterns: **%** means "any characters", so **name LIKE '%sensor%'** finds every name containing "sensor". See [examples/Ex04Query.java](#).

Update and delete

```
try (PreparedStatement sell = db.prepareStatement(
    "UPDATE products SET quantity = quantity - ? WHERE code = ? AND quantity >=
?")) {
    ...
    int changed = sell.executeUpdate();    // 0 means nothing matched
}
```

Always include a WHERE clause. **UPDATE products SET price = 0** with no **WHERE** changes every row, and **DELETE FROM products** empties the whole table. Putting the rule in the **WHERE** (here, "only if there's enough stock") makes the check and the change happen in one step. See [examples/Ex05UpdateDelete.java](#).

SQL injection: why **PreparedStatement** matters

It's tempting to build SQL by joining text, as you've joined text since Module 1:

```
// NEVER DO THIS
String sql = "SELECT COUNT(*) FROM users WHERE name = '" + user + "' AND password = '" + password + "'";
```

Run `examples/Ex06SqlInjection.java`. An attacker who types the password `anything' OR '1'='1` turns the query into:

```
SELECT COUNT(*) FROM users WHERE name = 'admin' AND password = 'anything' OR '1'='1'
```

The quote mark in their "password" ends the text early, and the rest of what they typed becomes **SQL code**. `'1'='1'` is always true, so they log in without knowing the password. This is **SQL injection**, one of the most common ways real systems are broken into, and the same trick can read, change or delete any data in the database.

With a **PreparedStatement**, the SQL is fixed **before** the user's text arrives, and the text is sent separately, **only ever as data**. The attack becomes nothing more than a strange password that doesn't match. So the rule is simple: **every value that comes from a user, a file or anywhere outside your code goes in through a ? placeholder**. No exceptions, even when you're "sure" the value is safe.

Transactions: all or nothing

Some changes belong together. Moving money from Ama to Kojo is two updates: take it from Ama, give it to Kojo. If the program crashed between them, the money would vanish. A **transaction** makes a group of changes happen **all together, or not at all**:

```
db.setAutoCommit(false);           // start a transaction
try {
    ... take from Ama ...
    ... give to Kojo ...
    db.commit();                     // make every change permanent, together
} catch (SQLException e) {
    db.rollback();                   // undo everything since setAutoCommit(false)
} finally {
    db.setAutoCommit(true);
}
```

Normally every statement is its own transaction ("auto-commit"), and each one waits for the data to be safely written to the disk. So transactions also make **big imports dramatically faster**:

`examples/Ex07Transactions.java` inserts 5,000 rows one at a time (several seconds) and then in a single transaction (a fraction of a second). Import files in one transaction.

Let the database do the arithmetic

SQL can summarise data directly, like Module 15's collectors:

```
SELECT programme,
       COUNT(*)      AS students,
       ROUND(AVG(gpa), 2) AS average
FROM students
GROUP BY programme
ORDER BY average DESC
```

Function	Gives
<code>COUNT(*)</code>	how many rows
<code>SUM(x), AVG(x)</code>	total, average
<code>MIN(x), MAX(x)</code>	smallest, largest
<code>GROUP BY column</code>	one result row per group
<code>HAVING condition</code>	filters the <i>groups</i> (<code>WHERE</code> filters rows before grouping)
<code>CASE WHEN ... THEN ... ELSE ... END</code>	an if-else inside SQL

See `examples/Ex08Aggregates.java` (run the hook first, since it uses `school.db`).

Keep the SQL in one place: the repository

If SQL is scattered through a program, every change to the table means hunting through every class. Instead, put **all** of it in one **repository** class, whose methods take and return ordinary objects:

```
class ContactRepository implements AutoCloseable {
    Contact add(String name, String phone) throws SQLException { ... }
    List<Contact> search(String text) throws SQLException { ... }
    Optional<Contact> find(int id) throws SQLException { ... }
    boolean changePhone(int id, String phone) throws SQLException { ... }
    boolean delete(int id) throws SQLException { ... }
    public void close() throws SQLException { db.close(); }
}
```

The menu code never sees a `Connection` or a `ResultSet`: it just calls `contacts.add(...)` and gets a `Contact` record back. Implementing `AutoCloseable` lets the repository itself go in try-with-resources. An `INTEGER PRIMARY KEY AUTOINCREMENT` column numbers rows for you, and `Statement.RETURN_GENERATED_KEYS` gets the new number back. See `examples/Ex09Repository.java`, a complete contacts manager. The rest of the program never sees any SQL, so the storage could change without touching it. That's how Capstone 2 teams can swap their JSON storage for a database.

Common beginner mistakes

- **Joining user input into SQL.** Use `?` placeholders, always.
- **Forgetting the class path:** `No suitable driver found for jdbc:sqlite:...` means the jar isn't on the class path. Use `java -cp "lib/*" ...`, from the module folder.
- **Numbering placeholders from 0.** The first `?` is 1, and so is the first column of a `ResultSet`.
- **Reading a `ResultSet` without calling `next()` first.** It starts before the first row.
- **`UPDATE` or `DELETE` without a `WHERE`,** changing every row.
- **Not closing statements and result sets.** Use try-with-resources for all of them, not just the connection.
- **Inserting thousands of rows without a transaction,** and waiting.

- **Committing the .db file to Git.** Add `*.db` to your `.gitignore`: rebuild the database from your code and data files.

Try it yourself

1. Run every file in `examples/`, from this module's folder. Then open `school.db` in **DB Browser for SQLite** (free, for every operating system) and look inside.
2. Complete `exercises/Exercise1.java` (a library database, CRUD, with checks) and `exercises/Exercise2.java` (a CSV import and reports in SQL).
3. Build the module project: your Module 14 track, on a database.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 14b: SQLite and JDBC"
git push
```

Module 15: Generics, Collections & Lambdas

Hook: six questions, six statements

Run `examples/Ex000neLineAnswers.java`:

```
java Ex000neLineAnswers.java
```

```
1. Total revenue: GHS 31,357.00
2. Top three by revenue: [Raspberry Pi 5, ESP32, Arduino Uno]
3. Revenue by category: {Boards=12405.0, Components=6580.0, Motors=5096.0, ...}
4. Pocket-money items: [Jumper wires, LED pack, Soil moisture sensor, Ultrasonic sensor]
5. Units sold: 657
6. Average Boards price: GHS 421.25
...
```

A month of sales in a makerspace shop, and six questions about it. With the loops from Part 1, question 3 alone (revenue for each category) would need a map, a loop, `getOrDefault`, and a dozen lines. Here, each answer is **one statement** that reads almost like the question:

```
sales.stream()
    .sorted(Comparator.comparingDouble(Sale::revenue).reversed())
    .limit(3)
    .map(Sale::product)
    .toList();
```

"Take the sales, sort them by revenue, biggest first, keep the first three, turn each into its product name, and give me a list." This is a **stream**, and it's how modern Java processes data. Streams are built from two ideas this module explains properly: **generics** (the `<T>` in `List<T>`, which you've used since Module 7) and **lambdas** (the `->` you met in Module 12).

Generics: one class, any type

Since Module 7 you've written `ArrayList<String>` and `HashMap<String, Integer>`. The part in angle brackets is a **type argument**: it tells the list what it holds, so Java can stop you adding the wrong thing and you never need to cast what comes out. Classes that take type arguments are **generic**, and you can write your own:

```
class Box<T> {
    private T contents;

    void put(T item) { contents = item; }
    T get() { return contents; }
}

Box<String> words = new Box<>();
words.put("Akwaaba");
String w = words.get();           // no cast: Java knows it's a String
words.put(42);                    // compile error: a Box<String> only holds Strings
```

T is a **type parameter**: a placeholder that's filled in when the class is used. One generic class works for every type, with full type-checking, instead of one class per type. By convention type parameters are single capital letters: **T** (type), **E** (element), **K** and **V** (key and value), **A** and **B**. A class can have several: `class Pair<A, B>`. Generic types work with objects only, which is why you write `Box<Integer>`, not `Box<int>`. See `examples/Ex01GenericClass.java`.

Generic methods and bounds

A single method can be generic too. The type parameter goes before the return type:

```
static <T> void swap(T[] array, int i, int j) { ... }
```

Often a generic method needs to *do* something with its items, such as compare them. A **bound** says what the type must be able to do:

```
static <T extends Comparable<T>> T max(List<T> items) {
    T best = items.get(0);
    for (T item : items) {
        if (item.compareTo(best) > 0) {
            best = item;
        }
    }
    return best;
}

max(List.of(3, 41, 7))           // 41
max(List.of("mango", "apple"))  // "mango"
```

T extends Comparable<T> means "any type **T**, as long as **T** objects can be compared with each other", which is exactly what `compareTo` needs. One method finds the maximum of `Integers`, `Strings`, `Doubles`, or your own `Comparable` classes. (`extends` is used here for interfaces too.)

A **wildcard**, `?`, means "some type I don't need to name": `List<? extends Number>` accepts a `List<Integer>`, a `List<Double>`, or any list of numbers. See `examples/Ex02GenericMethods.java`.

The Collections Framework

The collections you've used are part of a family of **interfaces** (Module 12) and the classes that implement them:

Interface	Is...	Main classes
<code>List<E></code>	ordered, duplicates allowed, access by index	<code>ArrayList</code> , <code>LinkedList</code>
<code>Set<E></code>	no duplicates	<code>HashSet</code> , <code>LinkedHashSet</code> , <code>TreeSet</code>
<code>Queue<E></code>	first in, first out	<code>ArrayDeque</code> , <code>LinkedList</code> , <code>PriorityQueue</code>
<code>Deque<E></code>	a double-ended queue: also works as a stack	<code>ArrayDeque</code>
<code>Map<K, V></code>	keys to values	<code>HashMap</code> , <code>LinkedHashMap</code> , <code>TreeMap</code>

Two you haven't used yet:

- A **Deque** used as a **stack** (**push**, **pop**, **peek**) gives you last in, first out, like the **ArrayStack** in Module 12. Used as a **queue** (**offer** to join the back, **poll** to leave from the front), it gives first in, first out.
- A **PriorityQueue** always hands out the **smallest** item first (or the first by a **Comparator** you give it), however the items were added. It's how you model a hospital queue, a to-do list by priority, or the next event in a simulation.

See [examples/Ex03ChoosingACollection.java](#). Any class that implements **Iterable<T>** works in a for-each loop, as [examples/Ex04IterableClass.java](#) shows.

Lambdas and functional interfaces

In Module 12 you wrote lambdas for your own one-method interfaces. Java provides ready-made ones in [java.util.function](#), and the whole of the streams library is built on them:

Interface	Takes → gives	Method	Example
Predicate<T>	T → boolean (a test)	test	n -> n % 2 == 0
Function<T, R>	T → R (a conversion)	apply	s -> s.length()
Consumer<T>	T → nothing (an action)	accept	s -> System.out.println(s)
Supplier<T>	nothing → T (a source)	get	() -> new ArrayList<>()
BiFunction<T, U, R>	T, U → R	apply	(v, i) -> v / i
UnaryOperator<T>	T → T	apply	s -> s.toUpperCase()

The real power is that **methods can take functions as parameters**:

```
static <T> int countMatching(List<T> items, Predicate<T> test) {
    int count = 0;
    for (T item : items) {
        if (test.test(item)) {
            count++;
        }
    }
    return count;
}

countMatching(numbers, n -> n % 2 == 0)           // how many are even?
countMatching(words, w -> w.length() > 5)        // how many are long?
```

One method, any question. Predicates can also be combined with **and**, **or** and **negate**. See [examples/Ex05FunctionalInterfaces.java](#).

Method references

When a lambda does nothing but call one existing method, you can **name the method** instead, with **::**:

Method reference	Is short for
Integer::parseInt	s -> Integer.parseInt(s) (a static method)
String::toUpperCase	s -> s.toUpperCase() (a method of the item)
System.out::println	x -> System.out.println(x) (a method of one object)

Method reference	Is short for
<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code> (a constructor)

You've already used `Sale::revenue` and `Integer::sum`. See [examples/Ex06MethodReferences.java](#).

Streams

A **stream** is a pipeline for processing a collection: a **source**, any number of **intermediate** steps, and one **terminal** step that produces the result:

```
List<Integer> topThree = marks.stream()    // source: every mark
    .filter(m -> m >= 50)                 // keep the passes
    .sorted(Comparator.reverseOrder())    // biggest first
    .limit(3)                             // just three
    .toList();                            // terminal: collect into a list
```

Intermediate step	Does
<code>filter(predicate)</code>	Keeps only the items that pass the test
<code>map(function)</code>	Turns each item into something else
<code>sorted()</code> / <code>sorted(comparator)</code>	Puts the items in order
<code>distinct()</code>	Removes duplicates
<code>limit(n)</code> / <code>skip(n)</code>	Keeps the first <code>n</code> / skips the first <code>n</code>
<code>mapToInt(f)</code> / <code>mapToDouble(f)</code>	Turns items into numbers, for <code>sum</code> , <code>average</code> , <code>max</code>

Terminal step	Gives
<code>toList()</code>	A list of the results
<code>count()</code>	How many items are left
<code>sum()</code> , <code>average()</code> , <code>max()</code> , <code>min()</code>	Numbers (after <code>mapToInt</code> / <code>mapToDouble</code>)
<code>anyMatch(p)</code> , <code>allMatch(p)</code> , <code>noneMatch(p)</code>	A boolean
<code>findFirst()</code> , <code>max(comparator)</code>	An <code>Optional</code> (see below)
<code>forEach(action)</code>	Does something with each item
<code>collect(collector)</code>	Anything else: maps, joined strings, groups

Three things to know:

1. **Nothing happens until the terminal step.** The intermediate steps just build up the recipe.
2. **Streams never change the original collection.** They produce new results.
3. **A stream can only be used once.** Call `.stream()` again for a new one.

See [examples/Ex07StreamBasics.java](#).

Collecting into maps

The `Collectors` class turns a stream into almost anything. The most useful collector is `groupingBy`, which builds a map from a key to something about each group:

```
// Map<programme, List<Student>>
students.stream().collect(Collectors.groupingBy(Student::programme));

// Map<level, how many>, with the keys sorted
students.stream().collect(Collectors.groupingBy(Student::level, TreeMap::new,
Collectors.counting()));

// Map<programme, average GPA>
students.stream().collect(Collectors.groupingBy(Student::programme,
Collectors.averagingDouble(Student::gpa)));

// One String: "Akua, Esi, Kofi"
students.stream().map(Student::name).sorted().collect(Collectors.joining(", "));
```

`partitioningBy` splits into exactly two groups (`true` and `false`), and `summaryStatistics()` gives count, min, max and average in one go. See [examples/Ex08StreamGrouping.java](#).

Optional: a result that might not be there

What's the maximum of an empty list? There isn't one. Instead of returning `null` (and risking a `NullPointerException`), methods like `max` and `findFirst` return an **Optional**: a box that either holds a value or is empty.

```
Optional<Student> best = students.stream().filter(s -> s.level() == 400)
    .max(Comparator.comparingDouble(Student::gpa));

best.map(Student::name).orElse("(none)") // the name, or "(none)" if empty
best.isPresent() // is there a value?
best.ifPresent(s -> System.out.println(s)) // do something only if there is
```

Lambdas built into collections

Collections have their own lambda-taking methods, which replace some of the fiddliest loops from Part 1:

```
tasks.removeIf(t -> t.startsWith("buy")); // no
ConcurrentModificationException
tasks.replaceAll(String::toUpperCase);
map.forEach((key, value) -> System.out.println(key + ": " + value));
groups.computeIfAbsent(letter, k -> new ArrayList<>()).add(name); // create the
list if needed
```

See [examples/Ex09CollectionShortcuts.java](#). Now go back to the hook and read each answer aloud as a sentence. You can explain every one.

Loops or streams?

Streams are excellent for **transforming and summarising** data: filter this, group by that, add these up. Loops are still better when you need to **stop early in complicated ways**, change several variables at once,

or when a stream would need so many steps that it's hard to read. Good Java code uses both. If a stream doesn't read like a sentence, consider a loop.

Common beginner mistakes

- **Generic types with primitives:** `List<int>`. Use `List<Integer>`.
- **Using a stream twice:** "stream has already been operated upon or closed". Call `.stream()` again.
- **Forgetting the terminal step**, so nothing happens at all.
- **Expecting a stream to change the original list.** Use the result, or use `removeIf` / `replaceAll`.
- **`.get()` on an empty `Optional`:** `NoSuchElementException`. Use `orElse` or `ifPresent`.
- **`groupingBy` without `TreeMap::new`** when you want the keys in order.
- **Changing outside variables inside a lambda.** Lambdas can only use local variables that never change ("effectively final"). Let the stream produce the result instead.
- **Over-clever one-liners.** If you can't read it tomorrow, split it up.

Try it yourself

1. Run every file in `examples/`. In the hook, add a seventh question of your own and answer it with one stream.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: a film club explorer, a parts inventory, or a clinic flow analyser.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 15: generics, collections and lambdas"
git push
```

Module 16: Recursion & Algorithms

Hook: a tree from one method that calls itself

Run `examples/Ex00FractalTree.java`:

```
java Ex00FractalTree.java
```

A window opens with a tree: a trunk, two branches, four smaller branches, eight smaller still, all the way out to over a thousand green twigs. Here is the code that draws **all** of it:

```
void branch(Graphics2D pen, double x, double y, double length, double heading, int
depth) {
    if (depth == 0) {
        return;                                // too small: stop
    }
    ... draw one line from (x, y), length long, in the direction heading ...
    branch(pen, endX, endY, length * SHRINK, heading + ANGLE, depth - 1);
    branch(pen, endX, endY, length * SHRINK, heading - ANGLE, depth - 1);
}
```

`branch` draws one line, then **calls itself twice**, to draw two smaller trees growing from the tip. Each of those draws a line and calls itself twice more, and so on, until `depth` reaches 0. A method that calls itself is **recursive**. Change `ANGLE` to 60, or `SHRINK` to 0.8, and the whole tree changes shape.

Recursion is the natural way to handle anything that contains smaller copies of itself: trees, folders inside folders, mazes, family trees, circuits built from smaller circuits. This module teaches you to think recursively, and then uses that skill on two of the most important jobs in computing: **searching** and **sorting**, and how to measure which way of doing them is fastest.

Recursion: a smaller version of the same problem

Every recursive method has two parts:

1. A **base case**: a version of the problem so small you can answer it directly, without recursion.
2. A **recursive case**: a way to solve a bigger version using the answer to a **smaller** version.

```
/** n! = n x (n-1)! and 0! = 1 */
static long factorial(int n) {
    if (n == 0) {
        return 1;                                // base case
    }
    return n * factorial(n - 1);                  // recursive case: a smaller problem
}
```

`factorial(4)` is `4 * factorial(3)`, which is `4 * 3 * factorial(2)`... down to `factorial(0)`, which is `1`. The base case is what stops it. Every recursive call must move **closer** to the base case; otherwise the recursion never ends. See `examples/Ex01CountdownAndFactorial.java`, which also adds up the digits of a number (`sumDigits(4071)` is `1 + sumDigits(407)`) and works out powers.

What really happens: the call stack

Remember the call stack from Module 8: each method call waits for the calls it makes to finish. Recursion is no different. Each call gets its own copy of its parameters and local variables, and waits for the call below it:

```
factorial(4) called
  factorial(3) called
    factorial(2) called
      factorial(1) called
        factorial(0) called
          base case: returning 1
        factorial(1) returns 1 x 1 = 1
      factorial(2) returns 2 x 1 = 2
    factorial(3) returns 3 x 2 = 6
  factorial(4) returns 4 x 6 = 24
```

That's the output of `examples/Ex02CallStackTrace.java`: five calls go **down** to the base case, then the answers come back **up**, each call finishing its multiplication.

Thinking recursively

The trick is to **trust the recursion**. To write `reverse(s)`, don't try to picture every call. Just ask: "if I already had a method that reversed a *shorter* string, how would I use it?"

```
static String reverse(String s) {
    if (s.length() <= 1) {
        return s; // base case: nothing to reverse
    }
    return reverse(s.substring(1)) + s.charAt(0); // reverse the rest, then add the
    first letter
}
```

The same pattern, "deal with the first item, let recursion deal with the rest", works for checking palindromes, adding up arrays and counting characters. A method can also make **several** recursive calls, as the tree does, and as the permutations method in

`examples/Ex03RecursionOnStringsAndArrays.java` does to list every way of ordering `ABC`.

When recursion goes wrong

Leave out the base case, or make a call that isn't smaller, and the calls never stop. Each one takes a little memory on the call stack, and eventually it runs out: a **StackOverflowError**. Run `examples/Ex05StackOverflow.java`: it gets through about twenty thousand calls first. That limit also means very deep recursion (a million levels) isn't practical in Java, even when it's correct. A loop is better there.

Making recursion fast: memoisation

The Fibonacci numbers (0, 1, 1, 2, 3, 5, 8, 13..., each the sum of the two before) have a beautifully simple recursive definition:

```
static long fib(int n) {
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

But run `examples/Ex04Fibonacci.java`:

```
fibSlow(30) =      832,040      2,692,537 calls      3.9 ms
fibSlow(40) = 102,334,155  331,160,281 calls    479.0 ms
fibFast(40) = 102,334,155    79 calls
```

Over 300 million calls for `fib(40)`, because the same values are calculated again and again: `fib(40)` calls `fib(38)` twice, `fib(37)` three times, and so on. The fix is **memoisation**: store each answer in a `HashMap` the first time you work it out, and look it up after that. 79 calls instead of 331 million.

Searching

Linear search

The search you've written since Module 6 checks every item in turn. For `n` items it might need `n` comparisons. Double the data, double the work.

Binary search

If the data is **sorted**, there's a much better way: look at the **middle** item. If it's the one you want, you're done. If the target is bigger, it can only be in the **upper half**; if smaller, the **lower half**. Throw away the other half, and repeat:

```
static int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (a[mid] == target) {
            return mid;
        } else if (a[mid] < target) {
            low = mid + 1;           // it must be in the upper half
        } else {
            high = mid - 1;         // it must be in the lower half
        }
    }
    return -1;
}
```

Each step halves what's left, so a million items need only about 20 steps, and a hundred million about 27. That's how the computer guessed your number in at most 7 tries in Module 5. Run `examples/Ex06LinearVsBinarySearch.java`:

```
1,000 items: linear      1,000 comparisons, binary 10
1,000,000 items: linear  1,000,000 comparisons, binary 20
100,000,000 items: linear 100,000,000 comparisons, binary 27
```

Binary search can also be written recursively (the example shows how). Java has it built in:

`Arrays.binarySearch` and `Collections.binarySearch`.

Sorting

Binary search needs sorted data, so how do you sort? There are dozens of algorithms. Three show the main ideas:

- **Selection sort** (Ex07): find the smallest item and swap it to the front; then the smallest of the rest, and so on.
- **Insertion sort** (Ex08): take each item in turn and slide it left into its place among the items already sorted, the way you'd sort a hand of cards.
- **Merge sort** (Ex09), which is **recursive**: split the array in half, sort each half (by merge sort!), then **merge** the two sorted halves by repeatedly taking the smaller of their front items. The base case is an array of 0 or 1 items, which is already sorted.

Each example prints its progress step by step. Follow them with a pen and paper on a short array: it's the best way to understand them.

Big-O: how the work grows

How long an algorithm takes depends on the computer. A better question is: **how does the work grow** as the input grows? That's what **Big-O notation** describes:

Big-O	Name	Double the input, and the work...	Example
O(1)	constant	stays the same	<code>array[i], map.get(key)</code>
O(log n)	logarithmic	goes up by one step	binary search
O(n)	linear	doubles	linear search, adding up a list
O(n log n)	"n log n"	a little more than doubles	merge sort, <code>Arrays.sort</code>
O(n ²)	quadratic	quadruples	selection sort, insertion sort, nested loops over the data
O(2 ⁿ)	exponential	squares (adding just one more item doubles it)	the slow <code>fib</code> , the Towers of Hanoi

Run `examples/Ex10SortingRace.java`:

size	insertion ms	merge ms	<code>Arrays.sort</code> ms
16,000	24.8	1.4	1.4
32,000	90.1	2.8	1.6
64,000	362.7	7.6	4.4

Every time the size doubles, insertion sort takes about **four times** as long (O(n²)), while merge sort only roughly doubles (O(n log n)). For a million items, that's the difference between a couple of minutes and a fraction of a second. For small inputs almost anything is fast; Big-O matters when the data gets big. (`Arrays.sort` uses a highly tuned O(n log n) algorithm. In real programs, use it.)

Finally, `examples/Ex11TowersOfHanoi.java` solves a puzzle that's very hard to solve with loops and takes three lines with recursion, and shows what O(2ⁿ) means in practice.

Common beginner mistakes

- **No base case**, or a base case that's never reached: `StackOverflowError`.
- **A recursive call that isn't smaller**, such as `factorial(n)` calling `factorial(n)`.
- **Forgetting to return the recursive call's result**. Writing `factorial(n - 1);` on its own throws the answer away.

- **Binary search on unsorted data.** It gives wrong answers without any error. Sort first.
- **Off-by-one errors in binary search:** `low < high` instead of `low <= high`, or `mid` instead of `mid + 1`. Test with the first item, the last item, and a missing item.
- **Timing tiny inputs once.** Use big inputs, warm up first, and repeat.
- **Recursion where a loop is simpler.** Adding up a list is clearer as a loop. Use recursion when the problem itself is recursive.

Try it yourself

1. Run every file in `examples/`. In the hook, give the tree three branches at each level instead of two.
2. Complete `exercises/Exercise1.java` and `exercises/Exercise2.java`.
3. Build the module project in the track of your choice: a maze solver, resistor networks, or a DNA toolkit.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 16: recursion and algorithms"
git push
```

Module 17: Packages, Maven & JUnit Testing

Hook: a program that checks itself

Open a terminal in `examples/00-leap-year-bug` and type one command:

```
cd examples/00-leap-year-bug
./mvnw test # Windows: mvnw test
```

The first time, it downloads a tool called **Maven** and a testing library called **JUnit** (it takes a minute, and needs the internet). Then it compiles the project and runs seven **automated tests**. Two fail:

```
[ERROR] Failures:
[ERROR]   DateToolsTest.lastDayOfALeapYear:40 expected: <366> but was: <365>
[ERROR]   DateToolsTest.yearDivisibleBy400IsLeap:30 2000 divides by 400, so it IS a
leap year ==> expected: <true> but was: <false>
[ERROR] Tests run: 7, Failures: 2, Errors: 0, Skipped: 0
[ERROR] BUILD FAILURE
```

Somebody wrote the leap-year rule from Module 4 and forgot the 400 rule. Nobody had to run the program and notice a wrong date: the tests **caught it automatically**, said exactly which check failed, what was expected and what actually happened, and even pointed to the line.

Now open `src/main/java/com/makerspace/dates/DateTools.java`, fix `isLeapYear`, and run `./mvnw test` again:

```
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

That's how professional software is built. This module covers the three things that hook used: **packages** (how large programs are organised), **Maven** (the tool that builds them and fetches libraries), and **JUnit** (automated tests).

Packages

Every class so far has lived in the same folder, with no package. That's fine for a dozen classes, but real projects have hundreds, and they use libraries with thousands more. Two classes called `Reading` or `Utils` would clash. **Packages** solve this: they group related classes, like folders, and give each class a full name.

```
package com.makerspace.util; // the FIRST line of the file

public class TextTools {
    ...
}
```

- The class's full name is now `com.makerspace.util.TextTools`.
- The file **must** be in a matching folder: `com/makerspace/util/TextTools.java`.

- Package names are lower case, and by convention start with a reversed web domain (`com.makerspace`, `gh.edu.ug`), so they're unique worldwide. Java's own classes are in `java.util`, `java.nio.file`, and so on.

To use a class from **another** package, import it, exactly as you've imported `java.util.Scanner` since Module 3:

```
package com.makerspace.app;

import com.makerspace.util.TextTools;

public class Main { ... TextTools.titleCase("ama") ... }
```

Classes in `java.lang` (`String`, `Math`, `System`, `Integer`) are imported automatically. And now the **package-private** access level from Module 11 makes sense: a member with no modifier is visible only inside its own package.

Compiling packages by hand, and JAR files

`examples/01-packages` has two classes in two packages, under a `src` folder:

```
cd examples/01-packages
javac -d out src/com/makerspace/util/TextTools.java src/com/makerspace/app/Main.java
java -cp out com.makerspace.app.Main
```

`-d out` puts the `.class` files in `out`, in matching package folders. To run, give `java` the class's **full** name.

To hand a program to someone, you bundle its classes into one **JAR** file (Java ARchive: a zip file with a description inside):

```
jar --create --file greeter.jar --main-class com.makerspace.app.Main -C out .
java -jar greeter.jar
```

Typing out every file and every library by hand quickly becomes impossible. That's what build tools are for.

Maven: the build tool

Maven is the most widely used Java build tool. You describe your project once, in a file called `pom.xml`, and Maven does the rest: compiling, downloading libraries, running tests, and building JARs.

The standard layout

Every Maven project has the same shape, so anyone can find their way around it:

```
my-project/
├── pom.xml                what the project is, and what it needs
├── mvnw, mvnw.cmd, .mvn/  the Maven Wrapper (see below)
├── src/
│   ├── main/java/         your code, in package folders
│   │   └── com/makerspace/dates/DateTools.java
│   └── test/java/         your tests, in the SAME packages
│       └── com/makerspace/dates/DateToolsTest.java
```

Maven puts everything it builds in a `target` folder, which can always be deleted and rebuilt. (Don't commit it to Git.)

`pom.xml`

The **POM** (Project Object Model) is an XML file. The important parts:

```
<groupId>com.makerspace</groupId>      <!-- who made it -->
<artifactId>leap-year-bug</artifactId>  <!-- what it's called -->
<version>1.0</version>

<properties>
  <maven.compiler.release>17</maven.compiler.release>  <!-- the Java version -->
</properties>

<dependencies>
  <dependency>                                <!-- a library this project needs -->
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.11.4</version>
    <scope>test</scope>                      <!-- only needed for testing -->
  </dependency>
</dependencies>
```

Dependencies are the big win. Name a library, and Maven downloads it, and everything *it* needs, from **Maven Central**, a huge public store of Java libraries, and puts it on the classpath for you. `examples/02-maven-json` adds the **Gson** library with five lines of XML, and uses it to save a list of records as **JSON** and load them back, the job Module 14 said needed a library. To find a library's `<dependency>` lines, search central.sonatype.com.

The Maven Wrapper and the lifecycle

Every project in this module includes the **Maven Wrapper**: the `mvnw` script (`mvnw.cmd` on Windows) and the `.mvn` folder. It downloads the right version of Maven the first time you use it, so you never need to install Maven yourself. (If you *do* install it, `mvn` works in place of `./mvnw`.)

You ask Maven to carry out **phases** of a standard **lifecycle**. Each phase runs all the ones before it:

Command	Does
<code>./mvnw compile</code>	Compiles <code>src/main/java</code> into <code>target/classes</code>
<code>./mvnw test</code>	Compiles, then runs every test in <code>src/test/java</code>
<code>./mvnw package</code>	Compiles, tests, and builds a JAR in <code>target/</code> (it won't build one if a test fails!)
<code>./mvnw clean</code>	Deletes the <code>target</code> folder
<code>./mvnw -q compile exec:java</code>	Compiles and runs the main class (set up in the course's <code>pom.xml</code> files)

On macOS and Linux, type `./mvnw`; on Windows, type `mvnw`. If macOS or Linux says "Permission denied", run `chmod +x mvnw` once. In VS Code, open the folder containing `pom.xml`: the Java extension recognises the Maven project, and a **Maven** panel lets you run these phases with a click.

JUnit: automated testing

In Module 8 you wrote a little `check` method that printed PASS or FAIL. **JUnit** is the professional version: the standard testing library for Java. A test is a method marked `@Test` that calls your code and **asserts** what should be true:

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class DateToolsTest {

    @Test
    void februaryInALeapYear() {
        assertEquals(29, DateTools.daysInMonth(2, 2024));    // (expected, actual)
    }
}
```

If every assertion holds, the test **passes**. If one doesn't, the test **fails**, and JUnit reports the test's name, what it expected, and what it got. Each test is independent: a failure in one doesn't stop the others running.

Assertion	Passes when...
<code>assertEquals(expected, actual)</code>	they're equal (using <code>equals</code>)
<code>assertEquals(expected, actual, delta)</code>	two doubles are within <code>delta</code> of each other. Always use a delta for doubles (Module 2's <code>0.1 + 0.2!</code>)
<code>assertTrue(condition)</code> / <code>assertFalse(condition)</code>	the condition is true / false
<code>assertNull(x)</code> / <code>assertNotNull(x)</code>	<code>x</code> is / isn't <code>null</code>
<code>assertThrows(Type.class, () -> code)</code>	the code throws that exception (it gives you the exception, so you can check its message too)
<code>assertAll(() -> ..., () -> ...)</code>	every assertion inside passes (and it reports every failure, not just the first)

Every assertion can take a last argument: a message to show if it fails, as the hook's `yearDivisibleBy400IsLeap` test does.

`examples/03-junit-basics` tests a shopping cart, and shows the other features you'll use most:

- **@BeforeEach** marks a method that runs before **every** test, usually to create a fresh object, so tests can't affect each other.
- **@DisplayName("...")** gives a test a readable name in reports.
- **@ParameterizedTest** with **@CsvSource** runs the same test with a whole table of inputs and expected outputs: perfect for boundaries.

```

@ParameterizedTest
@CsvSource({
    "499.99, 0",
    "500, 0.05",           // boundaries: test exactly AT them
    "1000, 0.10",
})
void discountRates(double subtotal, double expectedRate) {
    assertEquals(expectedRate, ShoppingCart.discountRate(subtotal), 0.0001);
}

```

Test classes live in `src/test/java`, in the **same package** as the class they test (so they can even test package-private methods), and are named after it: `ShoppingCart` → `ShoppingCartTest`. Name each test method after **what it checks**: `zeroQuantityIsRefused` tells you what broke before you even open the file. In VS Code, the **Testing** panel (the flask icon) lists every test, with a  to run one, and a green tick or a red cross for the result.

What to test

A good set of tests checks three kinds of case for every method:

1. **Ordinary cases**: typical inputs, with answers you've worked out by hand.
2. **Boundaries and edge cases**: exactly 500, not just 450 and 550; an empty list; a single item; zero; the largest allowed value. Most bugs live at boundaries.
3. **Errors**: invalid input must be refused (`assertThrows`).

Tests can only check what you think to check. They show the **presence** of bugs, never their absence. But a project with good tests can be changed with confidence: change something, run `./mvnw test`, and in seconds you know whether you broke anything.

Test-driven development

Test-driven development (TDD) turns testing round: write a test **first**, for something the code doesn't do yet. Watch it fail (**red**). Write just enough code to make it pass (**green**). Then tidy the code, running the tests to check you haven't broken anything (**refactor**). Repeat. Writing the test first forces you to decide exactly what the code should do before you write it. Exercise 2 is a TDD exercise: the tests are written, and you make them pass.

Common beginner mistakes

- A **package** line that doesn't match the folder. `package com.makerspace.util;` must be in `com/makerspace/util/`.
- **Forgetting to import** a class from another package: "cannot find symbol".
- **Running a class without its full name**: `java -cp out Main` instead of `java -cp out com.makerspace.app.Main`.
- **Putting tests in `src/main/java`**. They go in `src/test/java`.
- **`assertEquals(actual, expected)`**: the wrong way round. The test still works, but failure messages say "expected" for the wrong value, which is very confusing. Expected comes first.
- **Comparing doubles without a delta**.
- **Tests that depend on each other**, or on the order they run in. Use `@BeforeEach` to start each test fresh.

- Committing the **target** folder. Add **target/** to **.gitignore**.
- Testing only the easy cases. Test the boundaries and the errors.

Try it yourself

1. Fix the bug in **examples/00-leap-year-bug** and get **BUILD SUCCESS**. Then run **./mvnw package** and **java -jar target/leap-year-bug-1.0.jar**.
2. Compile and run **examples/01-packages** by hand, and build its JAR. Run **examples/02-maven-json** and open the **stock.json** it writes. Run the tests in **examples/03-junit-basics**, then break **ShoppingCart** on purpose and watch which tests catch it.
3. Complete **exercises/exercise1-find-the-bugs** (write tests that catch four hidden bugs) and **exercises/exercise2-test-driven** (make the given tests pass).
4. Build the module project in the track of your choice: a tested text, electronics, or clinical toolkit.
5. Commit and push your work (without any **target** folders):

```
git add .
git commit -m "Complete Module 17: packages, Maven and JUnit"
git push
```

Module 18: GUIs with JavaFX

Hook: a game in a window

Open a terminal in `examples` and type:

```
cd examples
./mvnw javafx:run           # Windows: mvnw javafx:run
```

The first time, Maven downloads JavaFX (it takes a minute, and needs the internet). Then a window opens with a golden dot on a dark background, and a clock counting down from 20. Click the dot. It jumps somewhere else, a little smaller, and it starts moving faster. How many can you catch before time runs out? When the clock hits zero, a dialog asks whether you want to play again.

Look at how differently this program works. Every program since Module 1 ran from the top of `main` to the bottom, stopping to ask questions in a fixed order. This one sets things up and then **waits**: it reacts to a mouse click whenever one happens, to a timer firing every 900 milliseconds, and to another firing every second. This is **event-driven programming**, and it's how every app on your phone and laptop works. This module shows you how to build windowed programs with **JavaFX**, Java's modern GUI toolkit.

JavaFX or Swing? Since Module 1 you've drawn pictures with **Swing**, the older toolkit built into every JDK. JavaFX is newer, and has better controls, charts, styling with CSS, and animation, so it's the one to use for new desktop applications. It isn't part of the JDK, so you add it to a project like any other library. Swing is covered in the optional Appendix A, if you want to see how your Module 1 drawings work.

Getting JavaFX

With Maven (recommended)

Every project in this module is a Maven project (Module 17). JavaFX is two dependencies in `pom.xml`, and a plugin that runs the program:

```
<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-controls</artifactId>
  <version>21.0.12</version>
</dependency>
...
<plugin>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-maven-plugin</artifactId>
  <version>0.0.8</version>
  <configuration>
    <mainClass>com.makerspace.fx/com.makerspace.fx.${main}</mainClass>
  </configuration>
</plugin>
```

Maven downloads the right JavaFX for your computer (Windows, macOS or Linux). `./mvnw javafx:run` compiles and runs the program. The examples project holds 15 programs, so you choose one with `-Dmain:`

```
./mvnw javafx:run -Dmain=Ex03Layouts
```

Each project also has a `module-info.java`, which says which parts of JavaFX it uses:

```
module com.makerspace.fx {
    requires javafx.controls;      // windows, buttons, layouts, charts
    requires javafx.fxml;         // FXML (Example 13)
    opens com.makerspace.fx to javafx.fxml;
    exports com.makerspace.fx;
}
```

You can treat it as given. (Java **modules** group packages, the way packages group classes. JavaFX is built from modules, and this file lets it find your classes.)

In **VS Code**, open the folder containing `pom.xml` (for example `18-guis-with-javafx/examples`). Once the Java extension has loaded the project, open any example and click **Run** above its `main` method.

With the JavaFX SDK (no Maven)

You can also download the **JavaFX SDK** from gluonhq.com/products/javafx, unzip it, and tell `javac` and `java` where its `lib` folder is with two **VM options**:

```
javac --module-path /path/to/javafx-sdk-21/lib --add-modules javafx.controls -d out Hello.java
java --module-path /path/to/javafx-sdk-21/lib --add-modules javafx.controls -cp out Hello
```

(On Windows the path looks like `C:\javafx-sdk-21\lib`. Add `,javafx.fxml` after `javafx.controls` if you use FXML.) In an IDE, these two options go in the run configuration's **VM options** box. The single-file shortcut `java Hello.java` doesn't work with JavaFX, so compile first. Maven does all of this for you, which is why the course uses it.

"JavaFX runtime components are missing"

If you see this error, Java was started without JavaFX on the module path: you ran a JavaFX program with plain `java`, or with an IDE's Run button outside a Maven project, or without the VM options. Use `./mvnw javafx:run`, or add the `--module-path` and `--add-modules` options.

Stage, scene and nodes

A JavaFX program is a class that **extends** `Application` and overrides `start`:

```

public class Ex01FirstWindow extends Application {

    @Override
    public void start(Stage stage) {
        Label hello = new Label("Akwaaba! This is a JavaFX window.");
        StackPane root = new StackPane(hello);

        stage.setTitle("My first window");
        stage.setScene(new Scene(root, 480, 120));
        stage.show(); // windows start hidden!
    }

    public static void main(String[] args) {
        launch(args); // sets up JavaFX, then calls start
    }
}

```

The words come from the theatre:

Class	Is...
Stage	The window , with a title bar and a close button. JavaFX creates the first one and passes it to <code>start</code>
Scene	What's shown in the window: one tree of nodes, with a size
Nodes	Everything in the scene: controls (<code>Label</code> , <code>Button</code> , <code>TextField</code> ...), layout panes (<code>HBox</code> , <code>BorderPane</code> ...), and shapes (<code>Circle</code> , <code>Line</code> ...)

The nodes form a tree called the **scene graph**: a layout pane holds other nodes in its `getChildren()` list, and they can be panes too. `launch` runs `start` on the **JavaFX application thread**, and closing the last window ends the program. See `Ex01FirstWindow.java`.

Events

A program finds out what the user did through **events**. You give a node a **handler**, and JavaFX calls it when the event happens:

```

Button plus = new Button("+");
plus.setOnAction(e -> {
    count++;
    display.setText(String.valueOf(count));
});

```

The handler is a functional interface (Module 12), so a lambda is the neat way to write one. This line **doesn't** run the lambda: it **registers** it, and JavaFX runs it each time the button is clicked, possibly hours later. The `e` is an `ActionEvent` describing what happened. Pressing Enter in a `TextField` also fires its `setOnAction`. Other events have their own handlers: `setOnMouseClicked`, `setOnKeyPressed`, and so on. See `Ex02ButtonsAndEvents.java`.

Layout panes

You don't usually place nodes at exact pixel positions: windows get resized, and fonts differ between computers. Instead, put them in a **layout pane** that decides where they go and how they stretch:

Pane	Arranges its children...	Good for
<code>HBox(spacing, ..)</code>	in a row	rows of buttons
<code>VBox(spacing, ..)</code>	in a column	forms, stacks of controls
<code>BorderPane</code>	in five regions: top, bottom, left, right, and centre, which gets all the spare space	the overall shape of a window
<code>GridPane</code>	in rows and columns: <code>grid.add(node, column, row)</code>	forms with labels, keypads
<code>StackPane</code>	on top of each other, centred	a message over a picture
<code>Pane</code>	exactly where you put them (<code>setLayoutX</code> , or a shape's own coordinates)	drawings and games

```
BorderPane root = new BorderPane();
root.setTop(title);
root.setCenter(keypad);
root.setBottom(new HBox(8, save, cancel));
root.setPadding(new Insets(10)); // space round the edge
HBox.setHgrow(searchField, Priority.ALWAYS); // this one stretches to fill a row
```

Real windows **nest** panes: a `BorderPane` for the overall shape, an `HBox` of buttons at the bottom, and a `GridPane` keypad in the centre. Run `Ex03Layouts.java` and resize the window to see how each pane behaves.

Text input and choices

`textField.getText()` gives what the user typed, as a `String`, and `label.setText(...)` changes what a label shows. Everything you know about converting and validating input (Modules 3 and 13) applies:

```
try {
    double c = Double.parseDouble(celsius.getText().trim());
    result.setText(String.format("%.1f C = %.1f F", c, c * 9 / 5 + 32));
} catch (NumberFormatException e) {
    result.setTextFill(Color.RED);
    result.setText("'" + celsius.getText() + "' isn't a number");
}
```

In a GUI, an uncaught exception doesn't end the program. It prints a stack trace in the terminal, where the user can't see it, and the button just seems to do nothing. So **always** catch bad input, and say what went wrong *in the window*. See `Ex04TextInput.java`.

Control	Read it with
<code>CheckBox</code>	<code>isSelected()</code>
<code>RadioButton</code> , each with <code>setToggleGroup(group)</code> so only one can be chosen	<code>isSelected()</code> , or <code>group.getSelectedToggle()</code>
<code>ComboBox<T></code> , a drop-down list	<code>getValue()</code> (it shows each item's <code>toString()</code>)
<code>Slider(min, max, start)</code>	<code>getValue()</code>

Control	Read it with
<code>DatePicker</code>	<code>getValue()</code> , a <code>LocalDate</code> (Module 14)
<code>TextArea</code>	<code>getText()</code> ; it scrolls by itself

`Ex05ChoiceControls.java` builds a lab-booking form from the first four, laid out with a `GridPane`.

Properties and binding

Most things about a JavaFX node are **properties**: a value you can get and set, that also tells anyone who's interested when it changes. A `Slider`'s value is `valueProperty()`, a `TextField`'s text is `textProperty()`, a `Label`'s text is `textProperty()`.

You can **listen** to a property:

```
slider.valueProperty().addListener((obs, oldValue, newValue) ->
    System.out.println("now " + newValue));
```

Or, better, **bind** one property to another, so it always follows it, with no handler code at all:

```
level.textProperty().bind(slider.valueProperty().asString("Level: %.0f"));
save.disableProperty().bind(name.textProperty().isEmpty()); // greyed out until
there's a name
```

The second line is something you'll use in every form: the Save button is disabled while the name field is empty, and enables itself the moment you type. See `Ex06PropertiesAndBinding.java`.

Dialogs, menus and files

Alert has ready-made dialogs, so you don't need to build a window for every message or question:

```
new Alert(Alert.AlertType.INFORMATION, "Saved!").showAndWait();

Alert sure = new Alert(Alert.AlertType.CONFIRMATION, "Delete this task?");
if (sure.showAndWait().orElse(ButtonType.CANCEL) == ButtonType.OK) { ... }

Optional<String> name = new TextInputDialog().showAndWait(); // empty if cancelled
```

`showAndWait` returns an `Optional` (Module 15), because the user might close the dialog without choosing. See `Ex07Dialogs.java`.

`Ex08MenusAndFiles.java` builds a small text editor with a **menu bar** (`MenuBar`, `Menu`, `MenuItem`), keyboard shortcuts (`setAccelerator(KeyCombination.keyCombination("Shortcut+0"))`), which means Ctrl on Windows and Cmd on a Mac), and `FileChooser`, the standard Open and Save dialogs, connected to the file code from Module 14.

Shapes and the canvas

JavaFX can draw in two ways:

- **Shapes** are nodes: `Circle`, `Rectangle`, `Line`, `Polygon`, `Text`. Add them to a `Pane`, and they stay there. To move one or change its colour, change its properties (`circle.setCenterX(...)`, `rect.setFill(Color.RED)`), and JavaFX redraws it for you. Shapes can be clicked, like the dot in the hook.
- A **Canvas** is a single node you paint on with a `GraphicsContext`, like the `Graphics2D` pen of Module 1's drawings: `gc.setStroke(...)`, `gc.strokeLine(...)`, `gc.fillOval(...)`. It keeps only the pixels, so to change the picture you clear it and paint it again. It's best for drawings with thousands of pieces, such as a graph or a game background.

`Ex09ShapesAndCanvas.java` shows both side by side: shapes you can click, and a sine wave painted on a canvas whose frequency follows a slider.

Lists and tables

A `ListView` or a `TableView` shows the items in an **ObservableList**: a list that tells the control when it changes. Add, remove, or replace an item in the list, and the control updates by itself:

```
ObservableList<Student> students = FXCollections.observableArrayList();
TableView<Student> table = new TableView<>(students);

TableColumn<Student, String> name = new TableColumn<>("Name");
name.setCellValueFactory(row -> new SimpleStringProperty(row.getValue().name()));
table.getColumns().add(name);

students.add(new Student("S001", "Akua Mensah", "BME", 300, 3.85)); // the table
shows it at once
Student chosen = table.getSelectionModel().getSelectedItem();
```

Each column's **cell value factory** is a lambda that, given a row, returns the value to show in that column, as a property. Two wrappers make tables even more useful:

- **FilteredList** shows only the items that pass a test. Change the test (`filtered.setPredicate(s -> ...)`) whenever a search box changes, and the table updates.
- **SortedList**, bound to the table's comparator, keeps the column headers clickable for sorting.

`Ex10TableView.java` is a student table with a live search box, sorting, and Add and Delete buttons.

Animation, and the JavaFX thread

For anything that happens **over time**, use a **Timeline** or an **AnimationTimer**. Both run your code on the JavaFX thread:

```
Timeline clock = new Timeline(new KeyFrame(Duration.seconds(1), e -> tick()));
clock.setCycleCount(Animation.INDEFINITE);
clock.play();

new AnimationTimer() {
    @Override
    public void handle(long now) { // about 60 times a second
        moveTheBalls();
    }
}.start();
```

Never use `Thread.sleep` (or a long loop) inside a handler. While a handler runs, JavaFX can't do anything else: no drawing, no clicks. The window freezes. See the bouncing balls in `Ex11Animation.java`. Now re-read the hook: two `Timelines` (one for the dot, one for the clock), a `Circle` with `setOnMouseClicked`, and an `Alert`.

Keep the model separate from the view

A common beginner design puts everything inside the window code: the data, the rules, and the buttons, tangled together. It works, until you want to test the rules, save the data, or build a different interface. Instead, split the program in two:

- The **model**: plain classes holding the data and enforcing the rules. No JavaFX at all. It can be unit tested (Module 17), and reused.
- The **view**: the window, which *displays* the model and *passes on* the user's actions to it.

`Ex12ToDoList.java` does this with a to-do list: a `TaskList` class that knows nothing about windows, and a window around it that saves to a file after every change. Exercise 2's calculator does it too, which is why `./mvnw test` can test it without opening a window at all.

FXML, Scene Builder and CSS

Bigger applications often describe the **layout** in a separate file, written in **FXML** (an XML language for scene graphs), and keep the Java code for what happens:

```
<VBox xmlns:fx="http://javafx.com/fxml"
fx:controller="com.makerspace.fx.TemperatureController"
    spacing="12" styleClass="card">
    <TextField fx:id="celsius" onAction="#convert"/>
    <Button text="Convert" onAction="#convert"/>
    <Label fx:id="result"/>
</VBox>
```

The **controller** class gets the controls through fields marked `@FXML`, with the same names as their `fx:ids`, and `#convert` calls its `convert()` method. `FXMLLoader.load(...)` builds the scene graph from the file. You don't have to write FXML by hand: **Scene Builder** (free, from gluonhq.com/products/scene-builder) lets you design the window by dragging controls, and saves FXML.

JavaFX nodes can also be styled with **CSS**, much as web pages are, with `-fx-` in front of each property:

```
.button { -fx-background-color: #F89820; -fx-text-fill: white; }
.bad    { -fx-text-fill: #C0392B; }
```

`scene.getStylesheets().add(...)` loads a stylesheet, and `node.getStyleClass().add("bad")` gives a node a class. See `Ex13FXMLAndCSS.java`, with `temperature.fxml` and `app.css` in `src/main/resources`.

Charts

JavaFX has charts built in: `LineChart`, `BarChart`, `PieChart`, `ScatterChart`, `AreaChart`. A chart shows one or more **series** of data points, and like a table, it updates by itself when you add a point:

```

LineChart<Number, Number> chart = new LineChart<>(new NumberAxis(), new
NumberAxis());
XYChart.Series<Number, Number> heartRate = new XYChart.Series<>();
heartRate.setName("Heart rate");
chart.getData().add(heartRate);

heartRate.getData().add(new XYChart.Data<>(1, 72));    // appears straight away

```

See `Ex14Charts.java`.

Common beginner mistakes

- **"JavaFX runtime components are missing."** Run with `./mvnw javafx:run`, or add the `--module-path` and `--add-modules` VM options.
- **Forgetting `stage.show()`**, or never giving the stage a scene.
- **Creating a node and never adding it** to a pane's `getChildren()` (or a `BorderPane` region), so it doesn't appear.
- **Adding the same node in two places.** A node can only have one parent: the second `add` moves it (or throws "duplicate children").
- **Using `Thread.sleep` in a handler**, which freezes the window. Use a `Timeline`.
- **Letting exceptions escape from a handler.** They vanish into the terminal. Catch them and show a message.
- **Changing a copy of the list** instead of the `ObservableList` the table shows, so the table doesn't update.
- **Calling `setText` on a bound property.** A bound property follows its binding: "A bound value cannot be set". Unbind it, or change what it's bound to.
- **A misspelt `fx:id` or `#handler`** in FXML: the `@FXML` field stays `null`, or loading fails. The names must match exactly.
- **Putting all the program's logic in the handlers.** Keep a separate model class.

Try it yourself

1. Run every example in `examples` with `./mvnw javafx:run -Dmain=...`. In the hook, add a second dot worth more points that appears only now and then.
2. Complete the exercises in `exercises`: Exercise 1 (a unit converter, `./mvnw javafx:run`) and Exercise 2 (a calculator whose model you make pass `./mvnw test` first, then `./mvnw javafx:run -Dmain=Exercise2`).
3. Build the module project in the track of your choice: a study planner, a resistor colour code calculator, or a vitals dashboard.
4. Commit and push your work (without any `target` folders):

```

git add .
git commit -m "Complete Module 18: GUIs with JavaFX"
git push

```

Module 19: Capstone Project 2

This is where Part 2 comes together: classes and objects, inheritance and polymorphism, interfaces, enums and records, exceptions, files, generics and streams, recursion, packages, Maven and JUnit, and (if you choose) a JavaFX interface, combined into **one real application**, built by a team, tested, documented, and tracked in Git.

Capstone 1 was a game that forgot everything when it closed. Capstone 2 is the kind of software people use at work: it **remembers** its data, **refuses** to crash, **proves** it works with automated tests, and **turns its data into charts**.

There are three tracks, as in every Part 2 project:

Track	Project
A: Generic	MakerStore : an inventory and lending system for a makerspace
B: Electrical/Electronic Engineering	SensorLab : a multi-sensor data logger and analyser
C: Biomedical Engineering	WardWatch : a patient vitals monitor with early-warning alerts

1. Teams

The rules are the same as Capstone 1:

Class size	Group size	Ratio
Up to 20 students	Pairs	2:1
More than 20 students	Groups of 4	4:1
Approved solo ("daring") students, any class size	Individual	1:1

Solo students complete the same core requirements (\$3) **plus every stretch goal** for their track (\$5). Ask your instructor before Stage 0.

2. Repository and setup

1. One team member creates a **private** repository named `capstone2-<track>-<teamname>` (e.g. `capstone2-ee-voltage-vipers`), with the Java `.gitignore` (then add `target/` to it), adds teammates and the instructor as collaborators, and everyone clones it.
2. Copy your track's starter folder (`generic_starter/`, `ee_starter/`, or `biomedical_starter/`) into the repository root (including the hidden `.mvn` folder), and commit it as your Stage 0 starting point.
3. **Every** team member checks that it builds and tests on their own computer:

```
./mvnw test                # Windows: mvnw test
./mvnw -q compile exec:java # run it
```

4. Split the work **by package and class**: one person owns `model`, another `storage`, and so on, with everyone writing tests for their own code. `git pull` before you start, commit small, and push often. **Never push code that doesn't compile or whose tests fail**: run `./mvnw test` before every push.

Each starter is a Maven project with **Gson** (for JSON) and **JUnit 5** already in its `pom.xml`, organised into four packages:

Package	Holds	May print or read input?
<code>model</code>	The classes, their rules, and the custom exceptions	No
<code>storage</code>	Saving and loading: JSON (with Gson), CSV import and export	No
<code>analysis</code>	Statistics, the recursive feature, and the charts (drawn to PNG files)	No
<code>ui</code>	The menu (and the optional JavaFX window)	Yes : the only package that talks to the user

Keeping input and output in `ui` is what makes everything else testable.

3. Requirements for every track

Object-oriented design (Modules 10–12)

- **At least six classes** across the `model` package, including an **abstract superclass with at least three subclasses** that override its abstract methods and are used polymorphically (no `instanceof` chains where an overridden method would do).
- **At least one interface** with more than one implementation (or used with lambdas), **at least one enum with fields**, and **at least two records**.
- **Encapsulation** everywhere: private fields, validation in constructors and methods, and copies (not the real lists) returned from getters.
- `equals` and `hashCode` overridden where objects are compared or used as keys, and `toString` everywhere it helps.
- A **custom exception hierarchy** (the starter's base exception and at least two more subclasses), thrown by the model and handled in `ui`.

Data that persists (Modules 13–14, 17)

- The application's full state is **saved to JSON** with Gson after every change, and **reloaded** on start-up. The starter's `SubtypeAdapterFactory` lets Gson save and load your class family: register each subclass with it.
- **CSV import** of the messy data file in your starter's `sample_data/` folder (skip and report every bad row), and **CSV export** of at least two reports.
- **Nothing crashes**: not on bad input, a missing or **corrupt** save file, a messy CSV row, or an empty data set.

Modern Java (Modules 15–16)

- **Streams** with `Comparators` and `Collectors` for the searching, sorting, grouping and statistics, and generics where they make code reusable.

- **At least one recursive method** that fits the problem (your track names one), with its base case and recursive case in its Javadoc.

Analysis

- **At least five summary statistics** shown in the application.
- **At least three different charts** generated from the application's data and saved as PNG files, each with a title and labelled axes (with units). The starter includes one working chart class drawn with Java2D, the same drawing tools as Module 1, onto an image instead of a window. Use it as a model for the others.

Testing (Module 17)

- **At least 40 JUnit tests**, all passing, covering the model (normal, boundary and invalid cases), storage (using `@TempDir`, as the starter's `StorageTest` does), the recursive method, and the statistics. Use `assertThrows`, a delta for doubles, and at least two `@ParameterizedTests`.

Documentation and teamwork

- The starter's `README.md`, completed: what the application does; **setup instructions** for a fresh computer; how to use it, with an example session; an **architecture** section listing every package and class and what it's responsible for; how to run the tests; and **who built what**.
- **Javadoc** on every public class and method.
- Regular, meaningful commits from **every** team member, under their own name.

4. The tracks

Track A: MakerStore (inventory and lending for a makerspace)

A makerspace lends tools, sells components, and hands out project kits.

- **Classes:** the abstract `Item` (ID, name, category, `value()`) with subclasses `Tool` (borrowed and returned; has a condition), `Component` (a consumable with a quantity and a reorder level), and `Kit` (contains other items, **including other kits**). Plus a `Member` class, the `Loan` record (with borrow and due dates, using `LocalDate`), and the `Inventory` that holds everything.
- **Features:** add, edit and remove items; lend and return tools, with due dates and **overdue detection**; issue components, with a **low-stock alert** at the reorder level; search by name or category; members' borrowing history.
- **Recursion:** a kit's total `value()` and its full parts list (`allParts()`) must include the contents of any kits inside it, at any depth.
- **Import/export:** import stock from `sample_data/stock.csv` (it has bad rows); export a stock report and an overdue list to CSV.
- **Charts:** stock value by category, loans per week, and the most borrowed tools.

Track B: SensorLab (multi-sensor data logger and analyser)

A lab bench with several sensors (voltage rails, a temperature probe, a current shunt) that need logging, alarms, and analysis.

- **Classes:** the abstract `Sensor` (ID, label, unit, alarm limits, calibration offset), whose subclasses each turn a **raw** logger value into a real one in their own way: `VoltageSensor` (raw volts, given), `TemperatureSensor` (an LM35: 10 mV per °C) and `CurrentSensor` (raw millivolts across a

shunt resistor: $I = V / R$). Plus the `Reading` record, a `Session` holding readings, and the `Lab` holding sensors and sessions.

- **Features:** configure sensors (limits and calibration, saved to JSON); **import** a messy log (`sample_data/bench_log.csv`: 480 good readings from four sensors over two hours, plus six bad rows); **simulate** a live session with a random walk, including occasional faults; raise an **alarm** for every out-of-limit reading; per-sensor statistics; a **moving average**; and **power and energy** for the matched voltage/current pair (V5 and I1).
- **Recursion:** the longest run of consecutive alarm readings (`Statistics.longestAlarmRun`), or a recursive equivalent-resistance calculator for the bench's resistor networks (Module 16).
- **Import/export:** import logs from CSV; export cleaned readings and an alarm report to CSV.
- **Charts:** a time series per sensor with its alarm limits drawn as lines (the starter's `LineChart` does this), a histogram of readings, and power over time.

The sample log has a story in it. Something happened to the load at around 10:10, and the 5 V rail didn't like it. Your alarms and charts should make it obvious.

Track C: WardWatch (patient vitals monitor with early-warning alerts)

A ward needs to record patients' vital signs, spot deterioration early, and summarise each patient's stay.

- **Classes:** the abstract `Measurement` with subclasses `HeartRate` (given), `BloodPressure`, `Temperature`, and `SpO2`, each with its own validation, "abnormal" rule and early-warning points. Plus the `Patient` record, a `PatientRecord` holding measurements, and the `Ward` holding patients and beds.
- **Features:** admit and discharge patients (with bed management); record vitals; import observations from `sample_data/ward_vitals.csv` (two days of four-hourly observations for the six patients in `sample_data/patients.csv`, plus six bad rows); the **simplified early-warning score** (below), with an **alert** when a patient's total reaches 5 or more; trends per patient; and a ward overview sorted by score.
- **Recursion:** the starter's `Location` class models the hospital as a tree (hospital → wards → bays → beds). Write the recursive `find` (where is a patient?) and `freeBeds` (how many empty beds?).
- **Import/export:** import vitals from CSV; export a discharge summary (text) and the alert log (CSV).
- **Charts:** each vital sign over time for a patient, with the normal range marked; early-warning scores across the ward; and a histogram of heart rates on the ward.

The sample data has a story too: one patient gets steadily worse over the two days, and another gets better. Your scores should show who.

Simplified early-warning score bands (for teaching only):

Points	Heart rate (bpm)	Temperature (°C)	SpO ₂ (%)	Systolic BP (mmHg)
3	≤ 40 or ≥ 131	≤ 35.0	≤ 91	≤ 90 or ≥ 220
2	111–130	≥ 39.1	92–93	91–100
1	41–50 or 91–110	35.1–36.0 or 38.1–39.0	94–95	101–110
0	51–90	36.1–38.0	≥ 96	111–219

This score is simplified and inspired by real early-warning systems. Thresholds are for teaching. This is not a clinical tool.

5. Stretch goals

Pick as many as you like. **Solo/daring students must complete every stretch goal for their track.**

All tracks:

1. **A JavaFX interface** (Module 18) as well as the console menu, using the **same** model, storage and analysis classes, with the charts drawn live in the window (JavaFX's own charts). Add the JavaFX dependencies and plugin from Module 18's `pom.xml`, and start the window with `./mvnw javafx:run`.
2. **SQLite storage** (the optional Module 14b) in place of, or as well as, the JSON files: a repository class in `storage` that the rest of the program uses exactly as it uses the JSON one.
3. **Command-line reports**: `java -jar target/<app>-1.0.jar report` prints a report and exits, without the menu.
4. **Test coverage**: add the JaCoCo plugin to the `pom.xml`, run `./mvnw test jacoco:report`, open `target/site/jacoco/index.html`, and reach 80% coverage of the `model`, `storage` and `analysis` packages.

Track A: fines for overdue tools (charged per day, by tool type); a reservation queue (a `Queue`) for a busy tool.

Track B: automatic detection of sensor **drift** (a moving average moving steadily away from the calibrated value); a one-page PNG dashboard showing every sensor.

Track C: a **trend alert** when a patient's score rises by 3 or more between consecutive observations; a per-patient PNG "chart sheet" showing all four vital signs.

6. Stages and timeline

Stage	Focus	Git evidence expected
Stage 0	Team formed, repository set up, everyone can build and test, design on paper : classes (with fields and methods), packages, and who owns what	Starter committed; design and work split in the README
Stage 1	<code>model</code> : classes, validation, exceptions, <code>equals/hashCode/toString</code> , with tests	Commits adding each class together with its tests
Stage 2	<code>storage</code> : JSON save/load, CSV import (with cleaning) and export, with @TempDir tests	A commit where data survives a restart
Stage 3	<code>ui</code> : the menu, input helpers, error handling, clean exit	The application is usable end to end
Stage 4	<code>analysis</code> : statistics, charts, the recursive feature	Charts generated from real application data
Stage 5	Hardening and polish: attack your own app (as in Module 13), reach 40+ tests, stretch goals	Commits named after the bugs found and fixed
Final	README complete, every test passing, <code>./mvnw package</code> builds a runnable JAR, final push before the deadline	Your final commit is your submission

Swap and review: before the final stage, swap with another team. Clone their repository, run `./mvnw test`, try to break their application, and read their code. Write them three specific, kind, useful suggestions as a GitHub issue on their repository.

7. How it's assessed

Area	What we look for
Functionality	Every feature in §3 and §4 works. Data persists. Nothing crashes.
Design	Sensible classes and packages with clear responsibilities. Inheritance used for real "is a" relationships, composition elsewhere. Logic separated from input and output.
Code quality	Readable names, Javadoc, no duplicated blocks, streams and records used where they help.
Testing	40+ meaningful tests covering normal, boundary, and invalid cases, all passing.
Analysis	Correct statistics and clear, honest, well-labelled charts.
Documentation	A stranger can set up, run, and test the project from the README alone.
Teamwork and Git	Regular commits from every member. Contributions match the README's "who built what". Be ready to explain any part of the code.

Try it yourself

1. Confirm your team and track with your instructor.
2. Create the repository, copy in your starter, and check that every member can run `./mvnw test`.
3. Design on paper, and agree who owns which package.
4. Work through the stages, testing as you go, committing and pushing regularly.
5. Submit by pushing your final commit before the deadline.

You started this course unable to write a single line of Java. You're finishing it with a tested, documented, multi-package application in your GitHub portfolio. Well done. Put the link on your CV.

Optional Material

Material you can skip, or come back to: Appendix A builds windowed programs with Swing, the GUI toolkit built into the JDK, and explains every line of the drawing programs from Module 1.

Appendix A: GUIs with Swing (optional)

This appendix is optional. The course builds GUIs with **JavaFX** in Module 18. Swing is the older GUI toolkit, and it's built into every JDK, so it needs no setup: that's why Module 1's drawing programs use it. Work through this appendix if you want to understand every line of those programs, or you need to maintain one of the many Swing applications still in use. It assumes Modules 10 to 13, and Module 14 for the file examples.

Hook: a game in a window

Run `examples/Ex00CatchTheDot.java`:

```
java Ex00CatchTheDot.java
```

A window opens with a golden dot on a dark background, and a clock counting down from 20. Click the dot. It jumps somewhere else, a little smaller, and it starts moving faster. How many can you catch before time runs out? When the clock hits zero, a dialog asks whether you want to play again.

Look at how differently this program works. Every program since Module 1 ran from the top of `main` to the bottom, stopping to ask questions in a fixed order. This one sets things up and then **waits**: it reacts to a mouse click whenever one happens, to a timer firing every 900 milliseconds, and to another firing every second. This is **event-driven programming**, and it's how every app on your phone and laptop works. This appendix shows you how to build windowed programs with **Swing**, the GUI toolkit that comes with every JDK.

Windows, components and containers

A Swing program is built from **components**, the visible pieces, put inside **containers**:

Class	Is...
<code>JFrame</code>	A window , with a title bar and a close button
<code>JPanel</code>	An invisible container that groups components and arranges them
<code>JLabel</code>	A piece of text (or an image)
<code>JButton</code>	A button that can be clicked
<code>TextField</code> / <code>JTextArea</code>	One line / several lines of text the user can type
<code>JCheckBox</code> , <code>JRadioButton</code> , <code>JComboBox</code> , <code>JSlider</code> , <code>JList</code>	Choices

```

JPanel panel = new JPanel();
panel.add(new JLabel("Akwaaba! This is a Swing window.));

JFrame window = new JFrame("My first window");
window.add(panel);
window.setSize(480, 120);
window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // closing the window ends
the program
window.setVisible(true); // windows start hidden!

```

Three rules you'll follow in every Swing program:

1. **Start Swing on its own thread.** Swing isn't safe to use from more than one thread at once, so it has a special one, the **event dispatch thread**. Wrap the code that builds your window in `SwingUtilities.invokeLater(() -> { ... });`, and Swing runs it there.
2. Call `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`, or the program keeps running invisibly after the window is closed.
3. Call `setVisible(true)` last, after adding everything.

`window.pack()` (instead of `setSize`) sizes the window to fit its contents exactly. See [examples/Ex01FirstWindow.java](#).

Events and listeners

A program finds out what the user did through **events**. You attach a **listener** to a component, and Swing calls it when the event happens:

```

JButton plus = new JButton("+");
plus.addActionListener(e -> {
    count++;
    display.setText(String.valueOf(count));
});

```

`ActionListener` is a functional interface (Module 12), so a lambda is the neat way to write one. Clicking the button **doesn't** run the lambda immediately when this line runs: it **registers** it, and Swing runs it each time the button is clicked, possibly hours later. The `e` is an `ActionEvent` object describing what happened. Pressing Enter in a `JTextField` also fires an action event, so the same listener can serve a button and a text field. See [examples/Ex02ButtonsAndEvents.java](#).

Layout managers

You don't usually place components at exact pixel positions: windows get resized, and fonts differ between computers. Instead, each container has a **layout manager** that decides where its components go and how they stretch:

Layout	Arranges components...	Good for
<code>FlowLayout</code> (a <code>JPanel</code> 's default)	in a row, left to right, wrapping like words	rows of buttons
<code>BorderLayout</code> (a <code>JFrame</code> 's default)	in five regions: <code>NORTH</code> , <code>SOUTH</code> , <code>EAST</code> , <code>WEST</code> and <code>CENTER</code> , which gets all the spare space	the overall shape of a window
<code>GridLayout(rows, cols)</code>	in a grid of equal cells	keypads, forms

Layout	Arranges components...	Good for
<code>BoxLayout</code>	in one column or one row	stacks of controls

```
JPanel main = new JPanel(new BorderLayout());
main.add(title, BorderLayout.NORTH);
main.add(keypad, BorderLayout.CENTER);
main.add(toolbar, BorderLayout.SOUTH);
```

Real windows **nest** panels: a `BorderLayout` for the overall shape, with a `FlowLayout` panel of buttons in the south and a `GridLayout` keypad in the centre. Run `examples/Ex03Layouts.java` and resize the window to see how each layout behaves. Borders such as `BorderFactory.createEmptyBorder(10, 10, 10, 10)` (padding) and `createTitledBorder("...")` make the groups clear.

Text input

`textField.getText()` gives what the user typed, as a `String`, and `label.setText(...)` changes what a label shows. Everything you know about converting and validating input (Modules 3 and 13) applies:

```
try {
    double c = Double.parseDouble(celsius.getText().trim());
    result.setText(String.format("%.1f C = %.1f F", c, c * 9 / 5 + 32));
} catch (NumberFormatException e) {
    result.setForeground(Color.RED);
    result.setText("'" + celsius.getText() + "' isn't a number");
}
```

In a GUI, an uncaught exception doesn't end the program. It prints a stack trace in the terminal, where the user can't see it, and the button just seems to do nothing. So **always** catch bad input, and say what went wrong *in the window*. Put a `JTextArea` inside a `JScrollPane` to give it scroll bars. See `examples/Ex04TextInput.java`.

Choices: check boxes, radio buttons, combo boxes, sliders

Component	Read it with
<code>JCheckBox</code>	<code>isSelected()</code>
<code>JRadioButton</code> , added to a <code>ButtonGroup</code> so only one can be chosen	<code>isSelected()</code>
<code>JComboBox<T></code> , a drop-down list	<code>getSelectedItem()</code> (and it shows each item's <code>toString()</code>)
<code>JSlider(min, max, start)</code>	<code>getValue()</code>

`examples/Ex05ChoiceComponents.java` builds a lab-booking form from all four, laid out with a `GridLayout`.

Dialogs and menus

`JOptionPane` has ready-made dialogs, so you don't need to build a window for every message or question:

```
JOptionPane.showMessageDialog(window, "Saved!");
String name = JOptionPane.showInputDialog(window, "What's your name?"); // null if
cancelled
int answer = JOptionPane.showConfirmDialog(window, "Delete this task?"); //
YES_OPTION, NO_OPTION...
```

See `examples/Ex06Dialogs.java`. `examples/Ex07Menus.java` builds a small text editor with a **menu bar** (`JMenuBar`, `JMenu`, `JMenuItem`), keyboard shortcuts (`setAccelerator`), and `JFileChooser`, the standard Open and Save dialogs, connected to the file code from Module 14.

Custom painting, explained at last

Since Module 1 you've drawn pictures by copying a pattern: a class that `extends JPanel` with a `paintComponent` method. Now every line makes sense:

```
class WaveGraph extends JPanel { // a JPanel that draws itself
(Module 11)
    @Override
    protected void paintComponent(Graphics g) { // override JPanel's version
        super.paintComponent(g); // let JPanel paint the background
first
        Graphics2D pen = (Graphics2D) g; // the richer Graphics2D (a cast,
(Module 11)
        ... draw, using getWidth() and getHeight() for the CURRENT size ...
    }
}
```

Swing decides when to call `paintComponent`: when the window first appears, when it's resized or uncovered, or when you ask. You **never** call it yourself. When your data changes, call `repaint()`, which asks Swing to repaint the component soon. Everything you draw must come from the object's fields, because Swing may ask you to redraw at any moment. `examples/Ex08CustomPainting.java` draws a sine wave whose frequency follows a slider.

Mouse events and animation

To react to the mouse, add a `MouseListener` (presses, releases, clicks) or a `MouseMotionListener` (moving and dragging). The `MouseAdapter` class provides empty versions of every method, so you only override the ones you need:

```
addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        ... e.getX(), e.getY() ...
    }
});
```

`new MouseAdapter() { ... }` creates an **anonymous class**: a subclass with no name, written right where it's needed. (Lambdas don't work here, because `MouseListener` has several methods.) See the sketch pad in `examples/Ex09MouseEvents.java`.

For anything that happens **over time**, use a `javax.swing.Timer`. It fires an action event every so many milliseconds, on Swing's own thread:

```
new Timer(16, e -> {           // about 60 times a second
    moveTheBalls();
    repaint();
}).start();
```

Never use `Thread.sleep` (or a long loop) inside a Swing listener. While a listener runs, Swing can't do anything else: no painting, no clicks. The window freezes. See the bouncing balls in `examples/Ex10Animation.java`. Now re-read the hook: two `Timers` (one for the dot, one for the clock), a `MouseAdapter`, and `paintComponent`.

Keep the model separate from the view

A common beginner design puts everything inside the window code: the data, the rules, and the buttons, tangled together. It works, until you want to test the rules, save the data, or build a different interface. Instead, split the program in two:

- The **model**: plain classes holding the data and enforcing the rules. No Swing at all. It can be unit tested (Module 17), and reused.
- The **view**: the window, which *displays* the model and *passes on* the user's actions to it.

`examples/Ex11ToDoList.java` does this with a to-do list: a `TaskList` class that knows nothing about windows, and a GUI around it that saves to a file after every change. Exercise 2's calculator does it too, which is why you can test it with `java Exercise2.java test`, without opening a window at all.

Common beginner mistakes

- Forgetting `setVisible(true)`, or calling it before adding the components.
- Forgetting `EXIT_ON_CLOSE`, so the program keeps running after the window closes.
- Calling `paintComponent` yourself. Call `repaint()`.
- Changing the data and not calling `repaint()`, so the picture doesn't update.
- Using `Thread.sleep` in a listener, which freezes the window. Use a `javax.swing.Timer`.
- Letting exceptions escape from a listener. They vanish into the terminal. Catch them and show a message.
- `setSize` on a component inside a layout. Layouts ignore it. Use `setPreferredSize`, or let the layout decide.
- Using `java.util.Timer` instead of `javax.swing.Timer`. Only the Swing one runs on the event thread. Import the right one.
- Putting all the program's logic in the listeners. Keep a separate model class.

Try it yourself

1. Run every file in `examples/`. In the hook, add a second dot worth more points that appears only now and then.

2. Complete `exercises/Exercise1.java` (a unit converter) and `exercises/Exercise2.java` (a calculator whose model you can test without a window).
3. Build the appendix project in the track of your choice: a study planner, a resistor colour code calculator, or a vitals dashboard.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Appendix A: GUIs with Swing"  
git push
```